



基础篇之-Linux 系统基础入门

Embedded Learning Fans Board

Instruction Manual

Rev. 1.1

2025/08/18



更新记录

日期	手册版本	更新内容
20241120	V1.0	初版
20250818	V1.1	做通用化调整

前言

📖说明：本文档旨在介绍 ubuntu 系统的常用命令及如何搭建个人开发环境，以 ubuntu 18.04 为例，详细展示操作系统的安装和配置过程，内容通用且适用于各类 ubuntu 发行版。

第一章介绍了 Linux 系统的起源、特征及组成。

第二章介绍使用虚拟机软件安装 ubuntu 系统。

第三章介绍 Linux 下常用的命令。

第四章介绍 Linux 下的 Vim 和 Gedit 编辑器。

第五章介绍了 ubuntu 下 GCC 编译器的相关知识，并以一个“Hello,World!”小例程来展示在 Linux 下如何进行编译。

第六章讲解了在 ubuntu 系统与 windows 之间互传文件以及常用的网络服务的搭建和使用。

⚠️**注意：**如果需要搭建特定平台的开发环境，请参考该平台的编译手册。

目录

更新记录.....	- 1 -
前言	- 2 -
目录	- 3 -
第一章 Linux 的前世今生	- 6 -
1.1 Linux 的简介	- 6 -
1.1.1 Linux 的由来.....	- 6 -
1.1.2 Linux 的主要特性	- 7 -
1.2 Linux 内核.....	- 7 -
1.2.1 Linux 内核简介	- 7 -
1.2.2 Linux 内核的组成	- 7 -
1.3 Linux 发行版	- 8 -
1.4 嵌入式 Linux	- 9 -
第二章 ubuntu 系统安装	- 11 -
2.1 操作系统选择	- 11 -
2.2 Vmware 虚拟机安装	- 11 -
2.2.1 VMware 软件的下载与购买	- 11 -
2.2.2 VMware 软件的安装.....	- 12 -
2.3 使用提供的 ubuntu 系统	- 16 -
2.4 搭建自己的 ubuntu 系统	- 20 -
2.4.1 创建 ubuntu 虚拟机	- 20 -
2.4.2 系统安装.....	- 31 -
2.4.3 VMware Tools 安装.....	- 42 -
2.4.4 ubuntu 网络设置	- 45 -
2.4.5 必要库安装	- 50 -
2.4.6 ubuntu 系统的主要目录	- 51 -
2.5 软件包的管理方式.....	- 51 -
2.5.1 APT 工具	- 51 -
2.5.2 dpkg 工具	- 52 -
第三章 shell 基本介绍及常用命令	- 53 -
3.1 Shell 是什么?	- 53 -
3.2 Shell 的特性.....	- 53 -
3.3 常用的 shell 命令	- 53 -
3.4 目录相关命令	- 53 -
3.4.1 pwd 查看当前目录.....	- 53 -
3.4.2 ls 查看当前目录的内容.....	- 54 -
3.4.3 cd 切换目录路径.....	- 55 -
3.4.4 mkdir 创建目录.....	- 56 -
3.4.5 rmdir 删除空目录.....	- 56 -
3.4.6 rm 删除命令.....	- 57 -
3.5 文件操作相关命令.....	- 58 -
3.5.1 touch 创建新文件	- 58 -
3.5.2 echo 创建有内容的文件.....	- 58 -

3.5.3 file 查看文件类型.....	58 -
3.5.4 cp 文件复制命令.....	59 -
3.5.5 mv 文件改名移动	60 -
3.5.6 chmod 权限设置命令	61 -
3.5.7 chown 文件属主设置命令	62 -
3.5.8 cat 文件查看合并命令	62 -
3.5.9 tar 文件压缩解压命令.....	63 -
3.5.10 ln 创建链接命令	64 -
3.5.11 find 文件查找命令	65 -
3.5.12 whereis 查找文件.....	66 -
3.5.13 grep 文本查找命令.....	66 -
3.5.14 uname 列出内核信息	67 -
3.6 网络相关命令	67 -
3.6.1 网络基本知识.....	67 -
3.6.2 ifconfig 网络配置命令.....	72 -
3.6.3 ping 网络通断检查命令	76 -
3.6.4 route 路由配置命令	80 -
3.6.5 nameserver 设置 DNS.....	81 -
3.7 其他命令	82 -
3.7.1 clear 清屏命令.....	82 -
3.7.2 man 帮助命令.....	83 -
3.7.3 su/sudo 用户切换命令	84 -
3.7.4 who.....	84 -
3.7.5 PS 查询进程命令	85 -
3.7.6 Kill 强制终止进程命令	86 -
3.7.7 top 动态显示命令	87 -
第四章 常用编辑器	89 -
4.1 Vim 编辑器.....	89 -
4.1.1 Vim 的安装	89 -
4.1.2 Vim 的工作模式.....	89 -
4.2 Vim 常用操作命令.....	90 -
4.2.1 系统终端常用的相关命令	90 -
4.2.2 Vim 一般模式常用命令.....	91 -
4.2.3 Vim 命令模式常用命令.....	93 -
4.3 Gedit 编辑器	98 -
第五章 GCC 编译器	100 -
5.1 GCC 的安装.....	100 -
5.2 GCC 的编译选项	100 -
5.3 GCC 的基本使用	101 -
5.4 静态链接和动态链接.....	104 -
第六章 文件传输与网络服务的搭建	108 -
6.1 windows 与 ubuntu 系统互传文件.....	108 -
6.1.1 共享文件夹	108 -
6.1.2 U 盘的加载	109 -

6.2 常用的网络服务的搭建和使用	- 110 -
6.2.1 TFTP 服务搭建.....	- 110 -
6.2.2 NFS 服务搭建	- 112 -
6.2.3 SSH 服务搭建	- 113 -
声明	- 116 -
更多帮助.....	- 117 -

第一章 Linux 的前世今生

本章节将对 Linux 展开介绍，包括 Linux 的由来、具有的主要特性、内核组成等等。并且对发行版 Linux 和嵌入式 Linux 也做了简单描述，便于大家能更好的理解

1.1 Linux 的简介

1.1.1 Linux 的由来

一、Multics 的诞生

（一）在上世纪 60 年代

计算机存在明显的几个劣势：未普及，只有少数人可以使用；只能做批处理，中途不能交互；不能多个用户同时使用，导致计算机资源的浪费。

（二）1965 年前后

贝尔实验室（Bell）、麻省理工学院（MIT）以及通用电气（GE）联合起来准备研发一个分时多任务处理系统（实现多人同时使用计算机），并把此系统取名为 Multics（多路信息计算系统）。

二、Unix 的诞生

（一）1969 年

Bell 实验室觉得项目可能不会成功，于是退出。当时实验室的 Ken Thompson 在研发 Multics 的时候，写了一个叫做太空旅行（Space Travel）的游戏，运行在 Multics 上。但由于 Bell 退出以后，Thompson 就没有了 Multics 的使用环境了，为了能够继续游戏，他花一个月的时间写了一个小型的操作系统。

（二）1970 年

游戏完成以后，他激动的拿出来给身边的同事分享，同事们普遍对他写的操作系统感兴趣，但是却只能支持两个用户，他们将这个系统命名为：“UNiplexed Information and Computing Service”，缩写为“UNICS”(与 Multics 相反)。后来大家取其谐音，就称其为“UNIX”了。这个时候已经是 1970 年了，于是就将 1970 年定为 Unix 元年，因此计算机上的时间就是从这一年开始计算的。

三、Minix 及 Linux 的诞生

（一）1974 年

Unix 的操作系统在 1974 年发展到第五个版本，Bell 实验室公开 Unix，引起了学术界的广泛兴趣以及其源码索取。所以，Unix 第五个版本就以“仅用于教育目的”的协议，提供给各大学作为教学使用。

（二）1978 年

后来由于受到美国《谢尔曼反托拉斯法》的影响，属于 AT&T 的 Bell 实验室无法销售除了电话电报机之外的产品，再后来 AT&T 分解，Bell 实验室出售 Unix，Unix 走向商业化，如果想使用此商品就需要购买授权，一份授权 4 万美元，很多大学由于高昂的授权费用，停止了研究。

（三）1987 年

于是，荷兰有个大学教授安德鲁写了一个 Minix，类似于 Unix，用于教学，世界各地的软件开发者纷纷开始使用并改进，希望将改进合并到 Minix 中，但是安德鲁不想破坏教学的纯净性就拒绝了。

（四）1991 年

由于 Minix 不好使用，也不让修改，Unix 又不开源，于是当时就读于赫尔辛基大学的 21 岁的 Linus Torvalds(林纳斯.托瓦兹)自己编写了一个操作系统内核，并在互联网上公之于众，而后发布了一个帖子表示自己写了一个操作系统的内核，但是还不够完善，大家可以随意使用，也可以一起修改，不收费。这个操作系统内核就是后来大名鼎鼎的 Linux。

（五）1994 年

在大家的努力下，Linux 的 1.0 版本正式发布。

1.1.2 Linux 的主要特性

一、基本思想

Linux 的基本思想是：一切都是文件。不管是真实的文件，还是抽象成文件的硬件设备，都提供统一的读写接口函数进行操作。至于说 Linux 是基于 Unix 的，很大程度上也是因为这两者的基本思想十分相近。

二、开源、free

Linux 是一款开源的操作系统，用户可以通过网络或其他途径自由获得，并可以任意修改其源代码。这是很多其他操作系统所做不到的。正是由于这一点，来自全世界的无数程序员参与了 Linux 的修改、编写工作，程序员可以根据自己的兴趣和灵感对其进行改变，这让 Linux 吸收了无数程序员的精华，不断壮大。

三、完全兼容 POSIX1.0 标准

这使得可以在 Linux 下通过相应的模拟器，运行常见的 DOS、Windows 的程序。这为用户从 Windows 转到 Linux 奠定了基础。许多用户在考虑使用 Linux 时，就想到以前在 Windows 下常见的程序是否能正常运行，这一点就消除了他们的疑虑。

POSIX(Portable Operating System Interface,可移植的操作系统接口)是由 IEEE 和 ISO/IEC 开发的一组标准。该标准基于现有的 UNIX 实践和经验完成，描述了操作系统的调用服务接口，用于保证编写的应用程序可以在源代码级上在多种操作系统中移植。该标准在推动 Linux 操作系统朝着正规化发展，是 Linux 前进的灯塔。

四、多用户、多任务

Linux 支持多用户，各个用户对于自己的文件设备有自己特殊的权限，保证了各用户之间互不影响。多任务则是现代电脑最主要的一个特点，Linux 可以使多个程序同时并独立地运行。

五、支持可视化界面

Linux 同时具有字符界面和图形界面。在字符界面用户可以通过键盘输入相应的指令来进行操作。它同时也提供了类似 Windows 图形界面的 X-Window 系统，用户可以使用鼠标对其进行操作。在 X-Window 环境中就和在 Windows 中相似，可以说是一个 Linux 版的 Windows。

六、支持多种平台

Linux 可以运行在多种硬件平台上，如具有 x86、ARM、ARM64、PowerPC 等处理器的平台。所以，大到大型企业的后台数据服务器，小到一般家庭里的机顶盒、游戏机、车载娱乐设备等，其操作系统内核很多都是 Linux。而且，2001 年 1 月份发布的 Linux 2.4 版内核已经能够完全支持 Intel64 位芯片架构。同时 Linux 也支持多处理器技术，多个处理器同时工作，使系统性能大大提高。

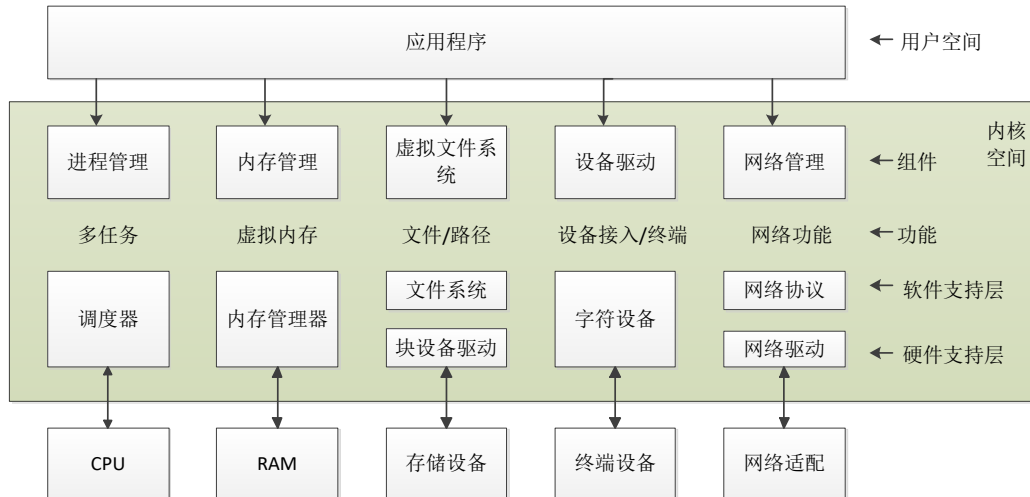
1.2 Linux 内核

1.2.1 Linux 内核简介

内核是操作系统的核心，具有许多最基本功能，它负责管理系统的进程、内存、设备驱动程序、虚拟文件系统 and 网络系统，并决定系统的性能和稳定性。

1.2.2 Linux 内核的组成

Linux 内核主要由如下几部分组成：进程管理、内存管理、虚拟文件系统、设备驱动和网络管理。如图：



上图说明了 Linux 内核的整体架构。根据内核的核心功能，Linux 内核提出了 5 个子系统，分别负责如下的功能：

一、**进程管理**（Process Scheduler）：也称进程调度。负责管理 CPU 资源，以便让各个进程可以以尽量公平的方式访问 CPU。

二、**内存管理**（Memory Manager）：管理 Memory（内存）资源，以便让各个进程可以安全的共享机器的内存资源。另外，内存管理会提供虚拟内存使进程地址空间隔离的机制，该机制可以让进程使用多于系统可用 Memory 的内存，不用的内存会通过文件系统保存在外部非易失存储器中，需要使用的時候，再取回到内存中。

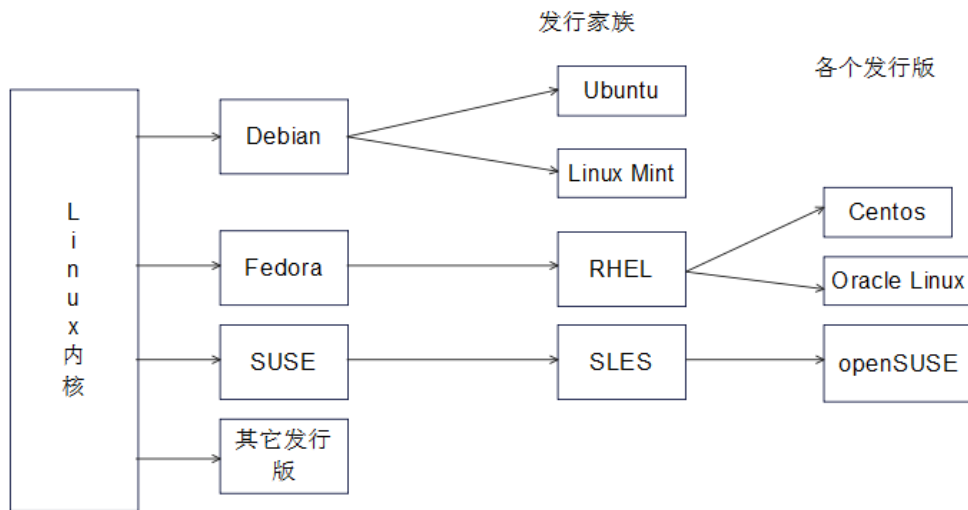
三、**虚拟文件系统**（Virtual File System）：即 VFS。Linux 内核将不同功能的外部设备（例如 Disk 设备（硬盘、磁盘、NAND Flash、Nor Flash 等）、输入输出设备、显示设备等等）抽象为统一的文件操作接口（open、close、read、write 等）。这就是 Linux 系统“一切皆是文件”的体现（其实 Linux 做的不彻底，因为 CPU、内存等还不是文件）。

四、**设备驱动**：负责管理第三方设备接入终端。

五、**网络管理**（Network）：负责管理系统的网络设备，各种硬件设备和外网设备的访问、驱动和控制等。

1.3 Linux 发行版

从技术上来说，Linus Torvalds（林纳斯·托瓦兹）开发的 Linux 只是一个内核。内核并不是一套完整的操作系统，它只是操作系统的核心。一些组织或厂商将 Linux 内核与各种软件和文档包装起来，并提供系统安装界面和系统配置、设定与管理工具，就构成了 Linux 的发行版本。Linux 的发行版说简单点就是将 Linux 内核与应用软件做一个打包。



目前市面上较知名的发行版有：Ubuntu、RedHat、CentOS、Debian、Fedora、SuSE、OpenSUSE、Arch Linux、SolusOS 等。

Linux 发行版主要有三个分支：Debian、Redhat、slackware。

一、Debian:（以社区的方式运作）

Ubuntu: 基于知名的 Debian Linux 发展而来，界面友好，容易上手，对硬件的支持非常全面，是目前最适合做桌面系统的 Linux 发行版本，Ubuntu 的所有发行版本操作系统是免费的，有一些服务和软件是收费的。

Linux Mint: 基于 Debian 和 Ubuntu 的 Linux 发行版，致力于桌面系统对个人用户每天的工作更易用，更高效，且目标是提供一种更完整的即刻可用体验。

二、Redhat:（商业公司维护的发行版本）

RedHatLinux: Red Hat 公司最早发行的个人版本的 Linux，自从 2013 年 RedHat9.0 版本发布后，Red Hat 公司停止了开发。

RHEL(red hat enterprise Linux): Red Hat 公司发布的面向企业用户的 Linux 操作系统，主要用在服务器上面，是收费版本。

Fedora: 基于 Red Hat Linux 终止发行后，Red Hat 公司计划以 Fedora 来取代 Red Hat Linux 在个人领域的应用，是免费版本。

Centos: 是一款企业级 Linux 发行版，它使用 Red hat Linux 中的免费源代码重新构建而成，相当于 RHEL 的免费社区克隆版。

三、slackware

suse: 基于 Slackware 二次开发的一款 Linux,主要用于商业桌面、服务器。

SLES(SUSE Linux Enterprise Server): 企业服务器操作系统，是唯一与微软系统兼容的 Linux 操作系统。

OpenSUSE: 由 suse 发展而来，旨在推进 Linux 的广泛使用，主要用于桌面环境，用户界面非常华丽，而且性能良好。

1.4 嵌入式 Linux

嵌入式 Linux 是嵌入式操作系统的一个新成员，其最大的特点是源代码公开并且遵循 GPL 协议，近几年来已成为研究热点。嵌入式 Linux 是将日益流行的 Linux 操作系统进行裁剪修改，使之能在嵌入式计算

机系统上运行的一种操作系统。ELF 1 开发板运行的系统就是一个嵌入式 Linux 系统。

一、嵌入式 Linux 的优势

- （一）承了 Internet 上无限的开放源代码资源，又具有嵌入式操作系统的可裁剪、低功耗等特性；
- （二）代码开放，技术支持为全世界的自由软件开发者；
- （三）性能优异，内核高效稳定；
- （四）丰富的硬件支持。目前已经成功移植到数十种硬件平台，几乎能够运行在所有流行的 CPU 上。驱动程序资源丰富，支持各种主流硬件设备和最新硬件技术；
- （五）应用产品开发周期短。软件丰富，有许多公开的代码可以参考和移植；

二、嵌入式 Linux 的应用

嵌入式 Linux 的应用领域非常广泛，主要的应用领域有信息家电数据网络远程通信、医疗电子、汽车远程信息处理、IoT 网关、人机界面、家庭能源管理系统、智能能源信息集中器、智能工业控制系统、电子 POS 设备、打印机和 2D 扫描仪等等，并且随着信息化、数字化、智能化的发展，不断向其他应用领域进行渗透和拓展。

第二章 ubuntu 系统安装

2.1 操作系统选择

Linux 操作系统有很多版本，我们选择使用的是 ubuntu18.04 发行版。ubuntu 是基于 Debian 派生的操作系统，兼容性极强，ubuntu 的特点是界面友好，容易上手。本文中的介绍及开发均是在 ubuntu18.04 上进行的。

运行 ubuntu 系统的环境除了 VMware 还有 virtualbox、hyper-v、kvm、WSL 等，我们选择了主流并且适合初学者的 VMware 来运行。

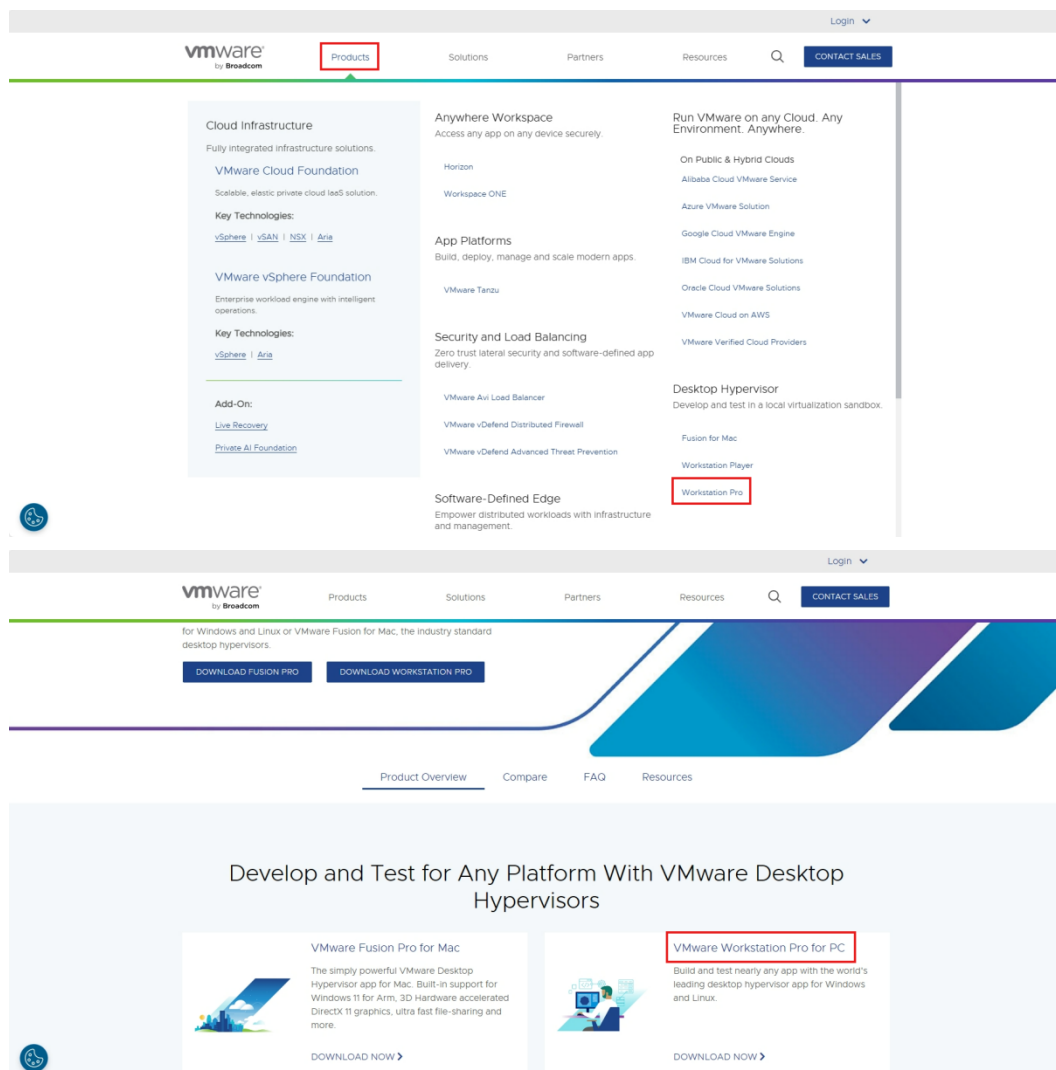
2.2 VMware 虚拟机安装

本章主要介绍 VMware 虚拟机的安装，以 VMware workstation 15 Pro v15 为例展示操作系统的安装配置过程。

2.2.1 VMware 软件的下载与购买

⚠注意：由于 VMware 官网内容可能会调整，链接和信息可能有变化。建议查阅时随时留意最新更新，并参考官方文档获取准确信息。

登陆 VMware 官网 <https://www.vmware.com/> 下载 Workstation Pro 并获取产品密钥。VMware 是付费软件，需要自行购买，或者可以选择使用试用。



目前官网最新的 Workstation Pro 已经更新到 Pro17，本文选择了主流并且稳定的 Pro15 版本进行使用和讲解。

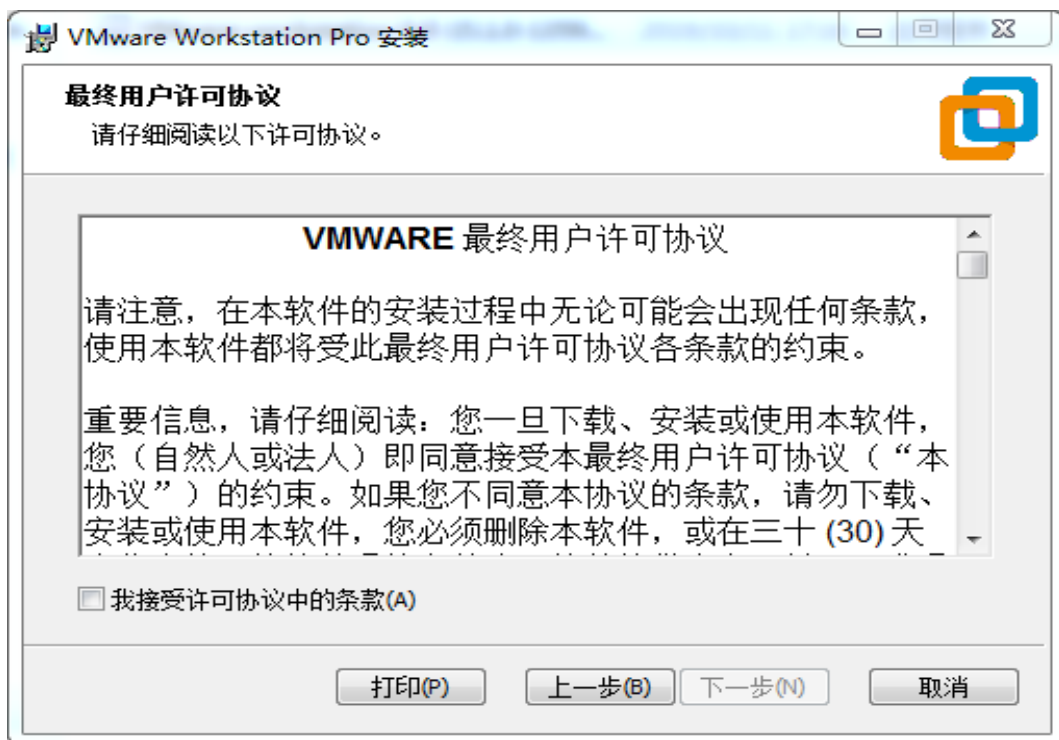
我们已经提供了 VMware Workstation 软件，大家可以直接使用，路径：[ELF 1 开发板资料包\06-常用工具\06-2 环境搭建工具\VMware-workstation-full-15.1.0-13591040.exe](#)

2.2.2 VMware 软件的安装

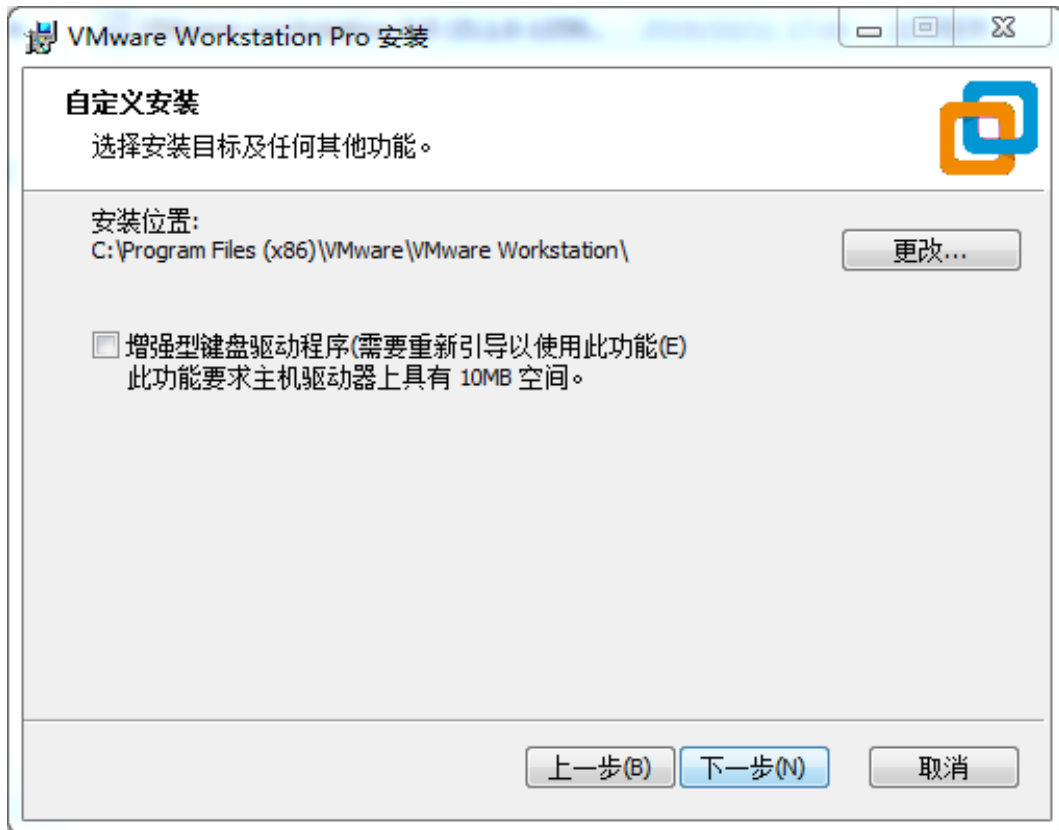
双击 VMware-workstation-full-15.1.0-13591040.exe 进入安装向导。



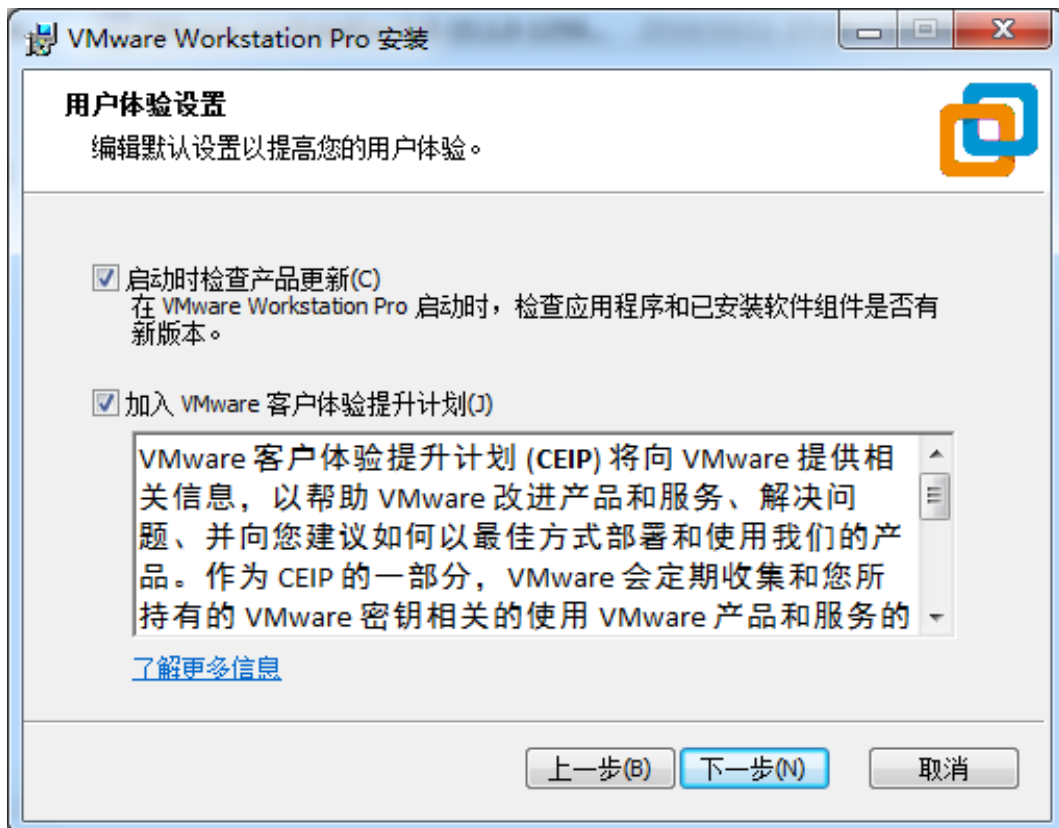
点击“下一步”。



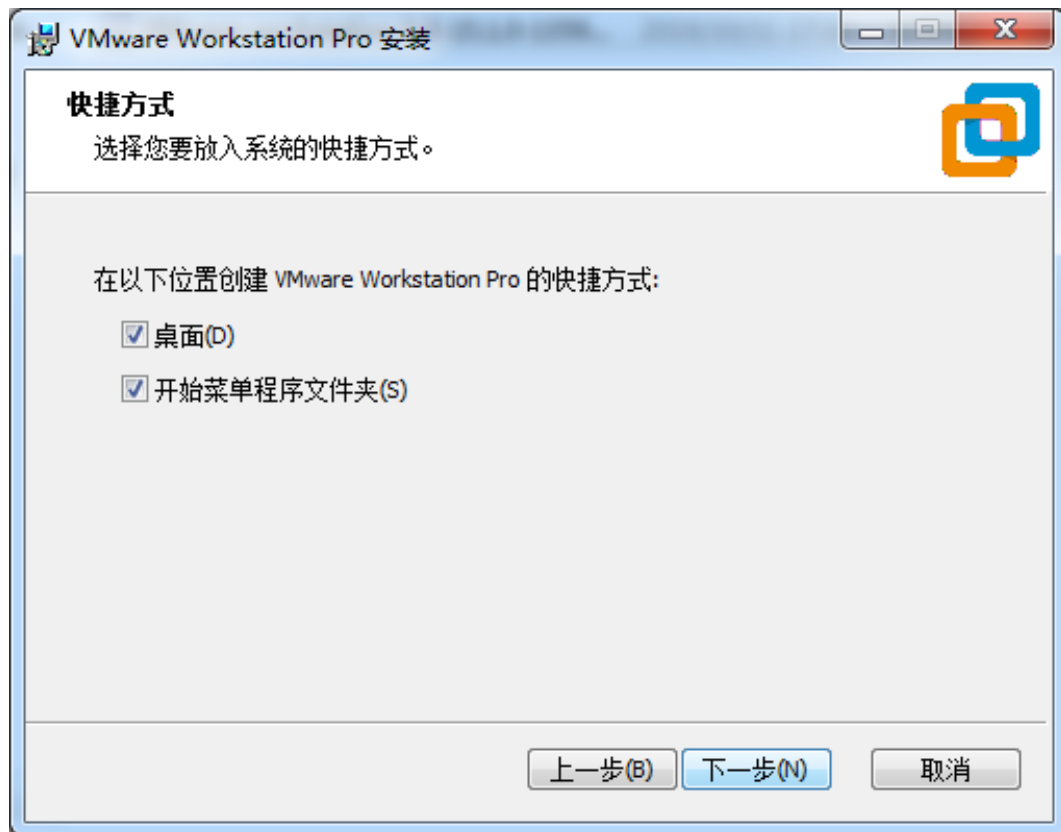
勾选我接受许可协议中的条款，点击“下一步”。



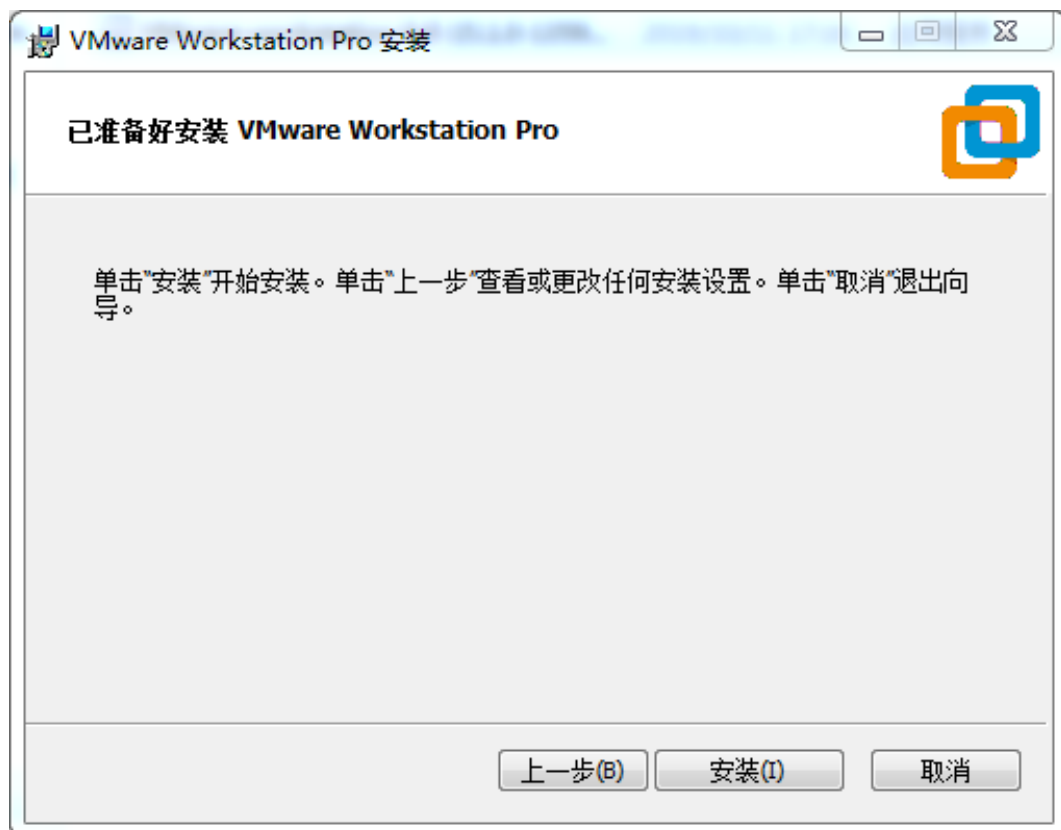
修改安装位置，装到自己电脑安装软件的分区，点击“下一步”。



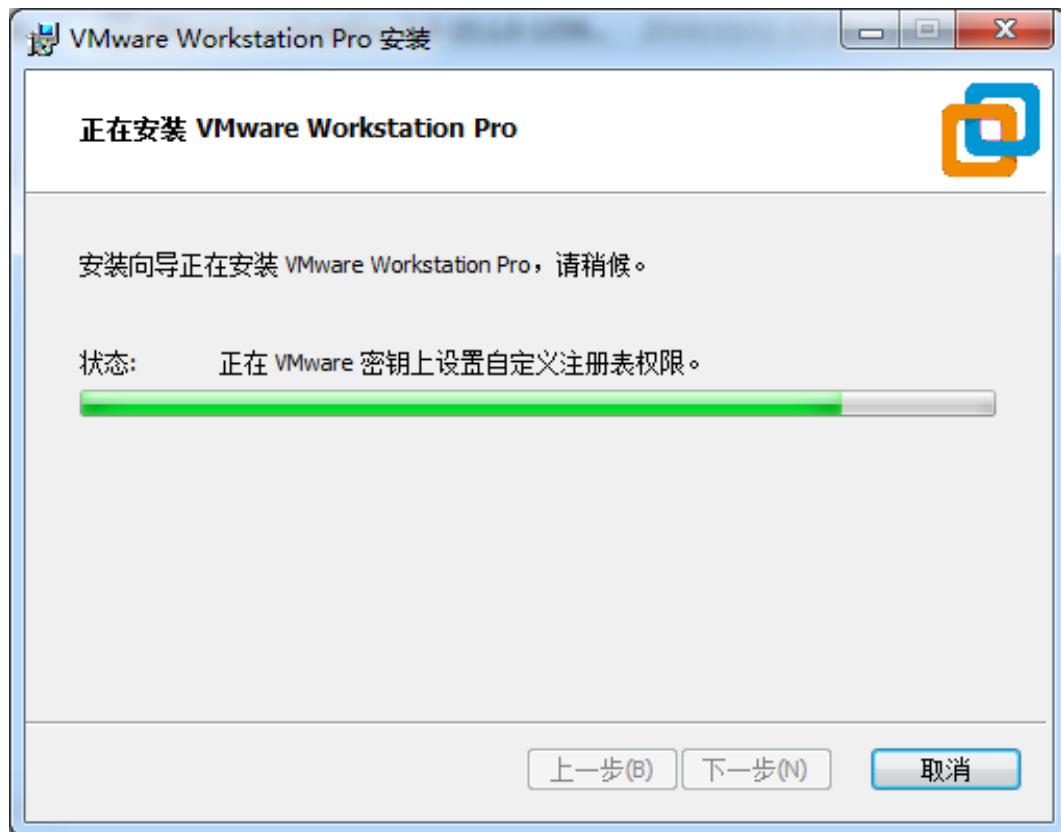
勾选，点击“下一步”。



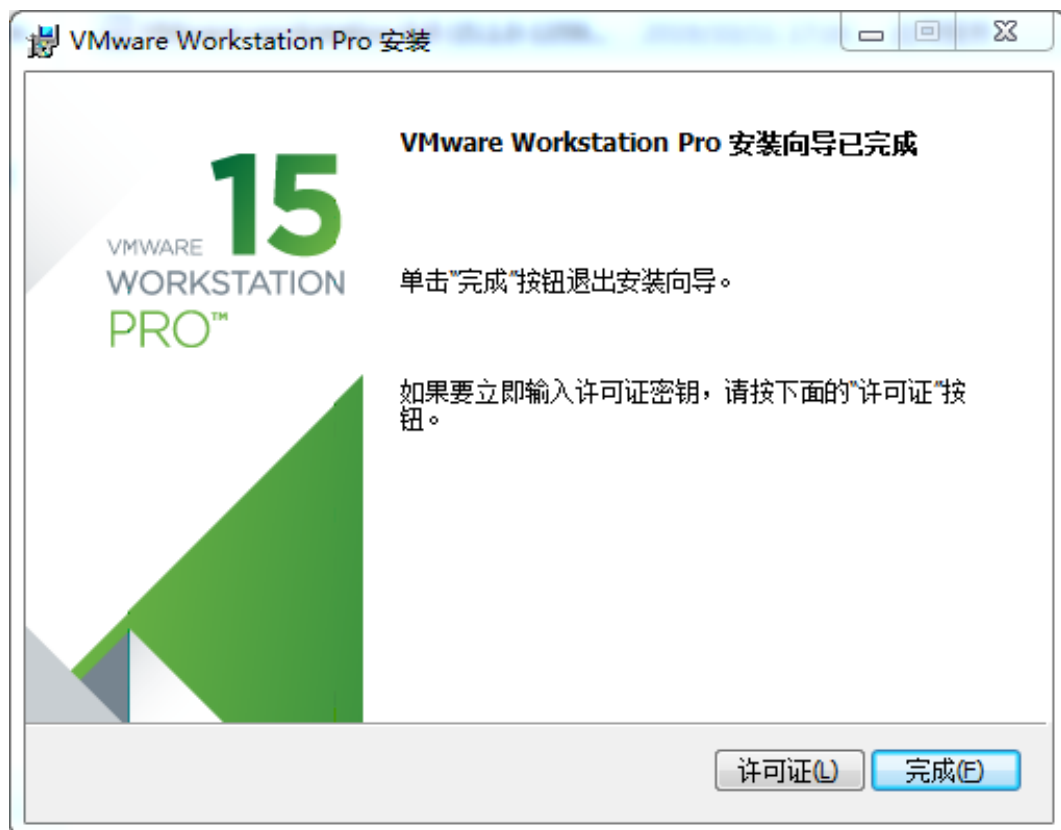
勾选添加快捷方式，点击“下一步”。

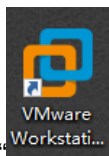


点击“安装”。

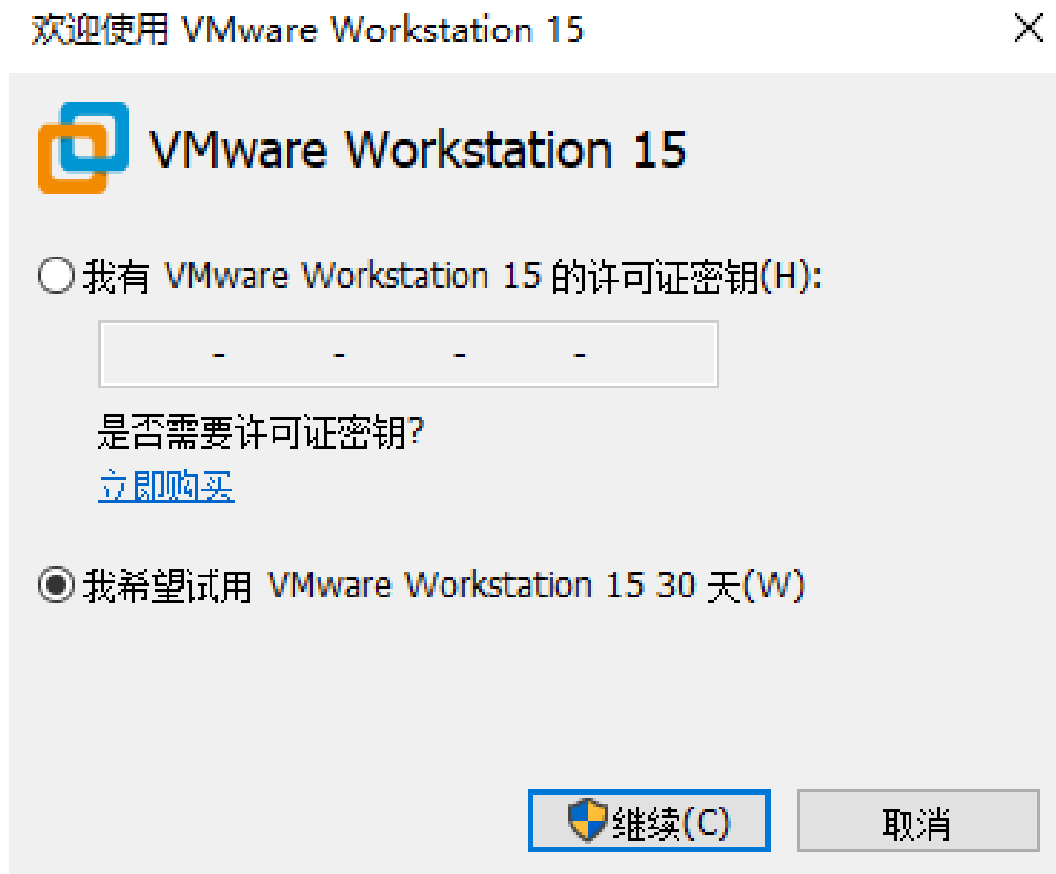


等待安装完成。





点击完成安装。双击桌面上的 VMware 图标“VMware Workstation 15”。



学习可以选择试用，若用户需要长期使用，需要到官方购买，填写许可证。

2.3 使用提供的 ubuntu 系统

⚠注意：EflBoard 提供的各种开发板的开发环境均可学习 ubuntu 系统的基本操作，学习者也可以下载所购买开发板的开发环境进行学习。

对于零基础的初学者，建议先使用搭建的 ubuntu 系统的学习基本操作，在对 Linux 系统有了基本了解之后再参考 2.4 章节搭建属于自己的 ubuntu 系统。本文以 ELF 1 开发板资料中提供的开发环境为例进行描述，该开发环境基于 ubuntu 系统进行构建，已经做了一些基本配置和网络服务的搭建（详细内容可见开发环境中的说明文档），学习者可以直接进行开发操作。

开发环境可通过以下链接获取：

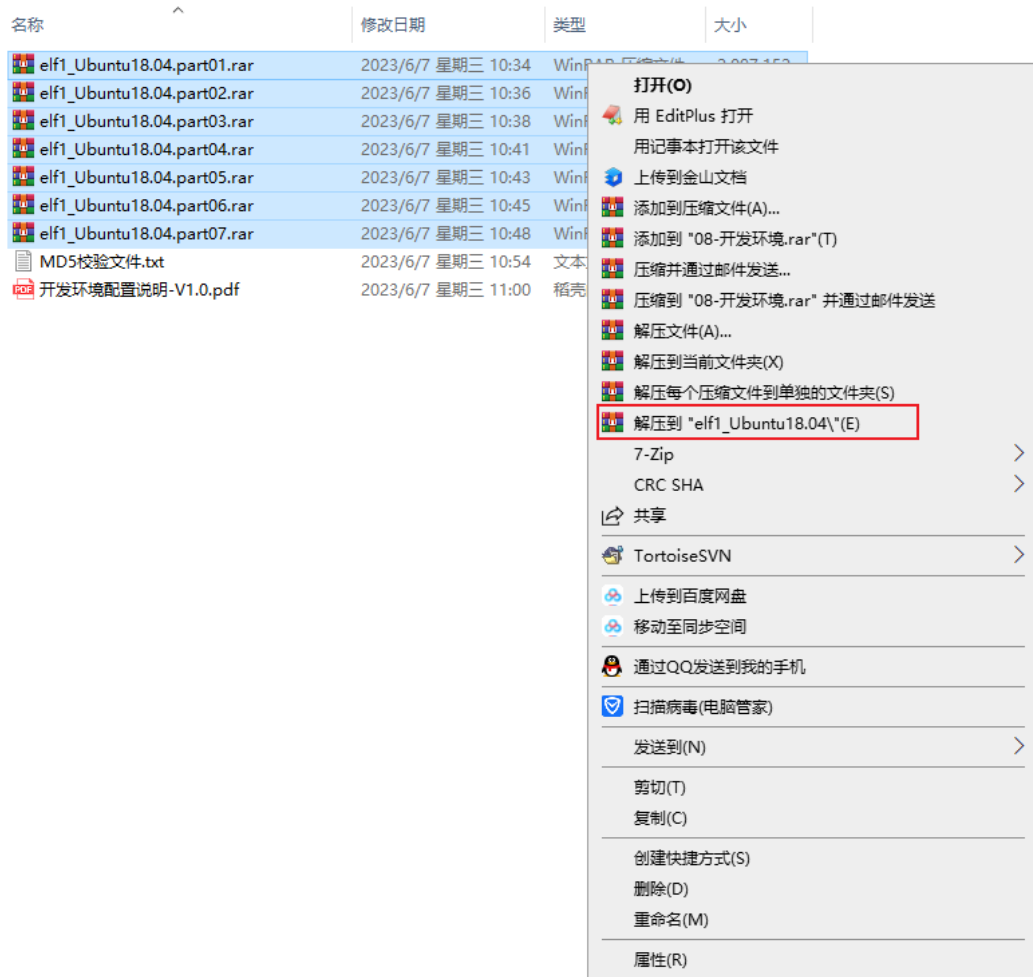
<https://pan.baidu.com/e/1-v7HFGMMhT2w2gweYN3PuA>

在 [ELF 1 开发板资料包\08-开发环境](#) 文件夹中有以下文件：

名称	修改日期	类型	大小	
elf1_Ubuntu18.04.part01.rar	2023/6/7 星期三 10:34	WinRAR 压缩文件	2,097,152...	开发环境压缩包
elf1_Ubuntu18.04.part02.rar	2023/6/7 星期三 10:36	WinRAR 压缩文件	2,097,152...	
elf1_Ubuntu18.04.part03.rar	2023/6/7 星期三 10:38	WinRAR 压缩文件	2,097,152...	
elf1_Ubuntu18.04.part04.rar	2023/6/7 星期三 10:41	WinRAR 压缩文件	2,097,152...	
elf1_Ubuntu18.04.part05.rar	2023/6/7 星期三 10:43	WinRAR 压缩文件	2,097,152...	
elf1_Ubuntu18.04.part06.rar	2023/6/7 星期三 10:45	WinRAR 压缩文件	2,097,152...	
elf1_Ubuntu18.04.part07.rar	2023/6/7 星期三 10:48	WinRAR 压缩文件	1,655,281...	
MD5校验文件.txt	2023/6/7 星期三 10:54	文本文档	1 KB	含MD5校验码文档
开发环境配置说明-V1.0.pdf	2023/6/7 星期三 11:00	稻壳阅读器 PDF ...	133 KB	

为了保证开发环境在下载过程中没有出现文件损坏，下载完后要对每一个文件进行 MD5 校验（可以通过网络进行 MD5 在线校验，也可以下载 MD5 校验工具），文件校验完生成的校验码与资料中提供的 MD5 一致，则文件下载正常，没有损坏；若不一致，则文件可能有破损，需要重新下载。

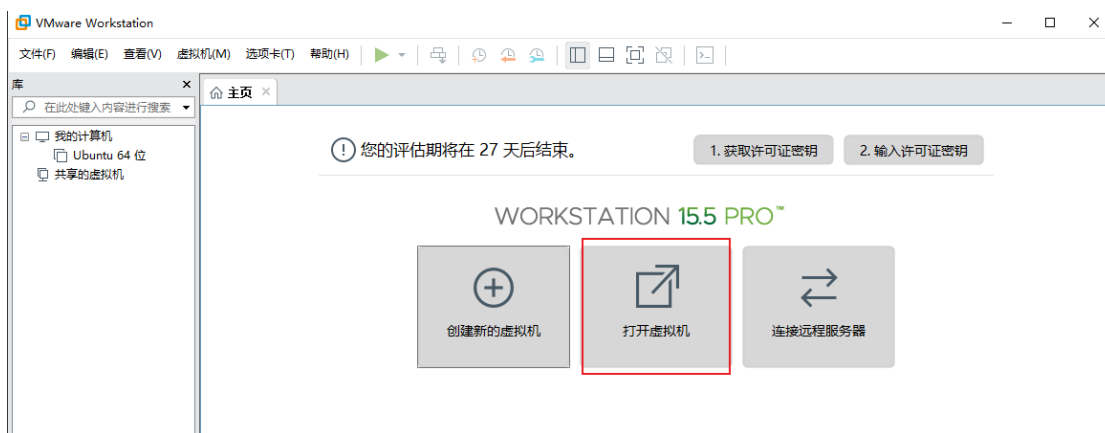
选中所有压缩包，右键解压到 elf1_Ubuntu18.04:



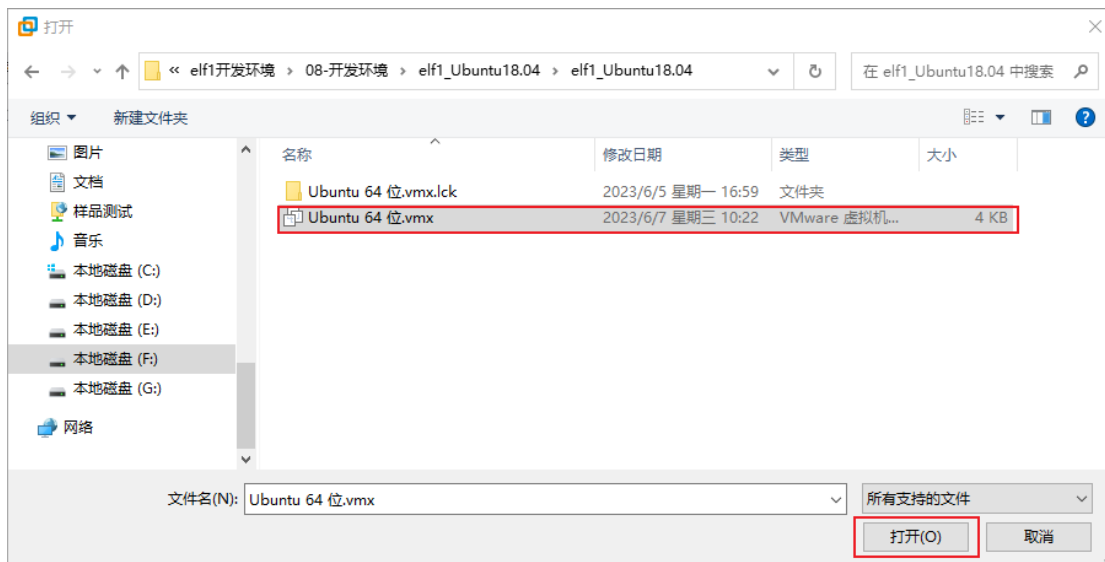
解压完成后，如下图:

名称	修改日期	类型	大小
elf1_Ubuntu18.04	2023/6/26 星期一 12:...	文件夹	
elf1_Ubuntu18.04.part01.rar	2023/6/7 星期三 10:34	WinRAR 压缩文件	2,097,152...
elf1_Ubuntu18.04.part02.rar	2023/6/7 星期三 10:36	WinRAR 压缩文件	2,097,152...
elf1_Ubuntu18.04.part03.rar	2023/6/7 星期三 10:38	WinRAR 压缩文件	2,097,152...
elf1_Ubuntu18.04.part04.rar	2023/6/7 星期三 10:41	WinRAR 压缩文件	2,097,152...
elf1_Ubuntu18.04.part05.rar	2023/6/7 星期三 10:43	WinRAR 压缩文件	2,097,152...
elf1_Ubuntu18.04.part06.rar	2023/6/7 星期三 10:45	WinRAR 压缩文件	2,097,152...
elf1_Ubuntu18.04.part07.rar	2023/6/7 星期三 10:48	WinRAR 压缩文件	1,655,281...
MD5校验文件.txt	2023/6/7 星期三 10:54	文本文档	1 KB
开发环境配置说明-V1.0.pdf	2023/6/7 星期三 11:00	稻壳阅读器 PDF ...	133 KB

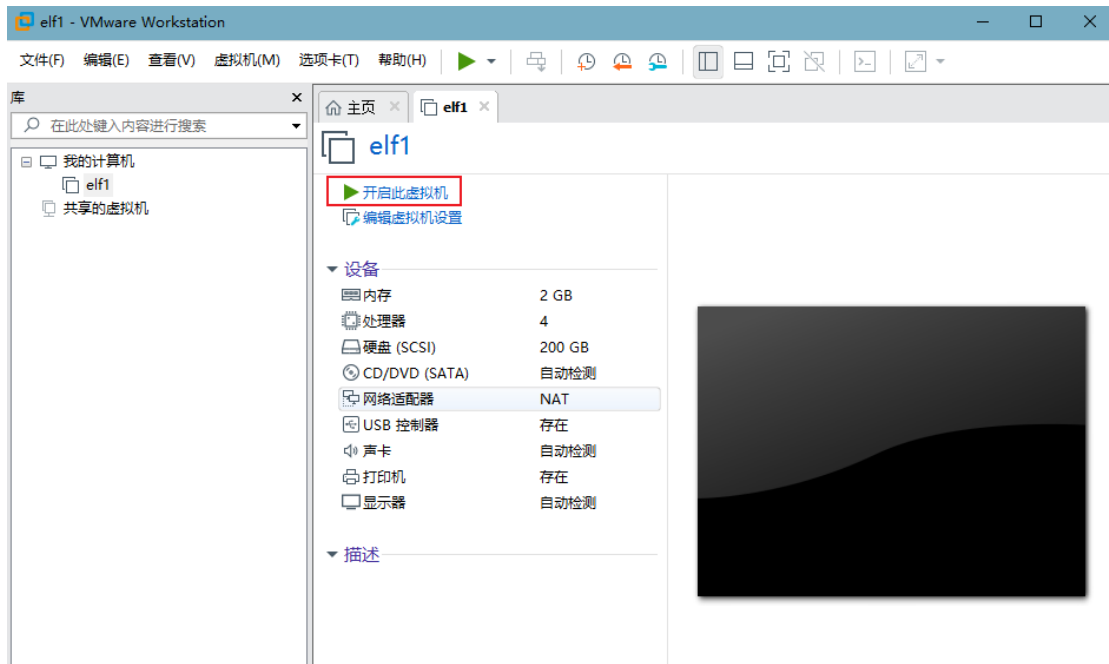
elf1_Ubuntu18.04 文件夹中的 Ubuntu 64 位.vmx 为虚拟机要打开的文件。
打开安装好的虚拟机。



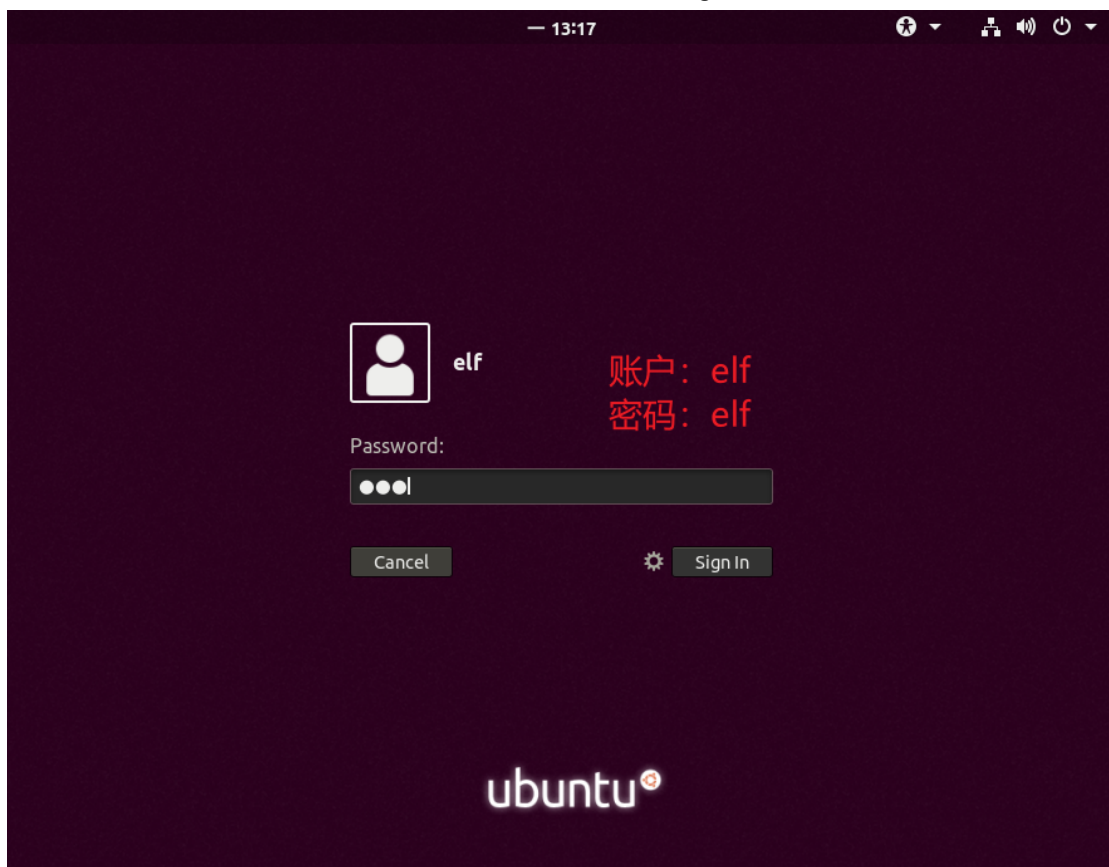
选择刚解压生成的 elf1_Ubuntu18.04 虚拟机文件所在的目录，双击打开启动文件。

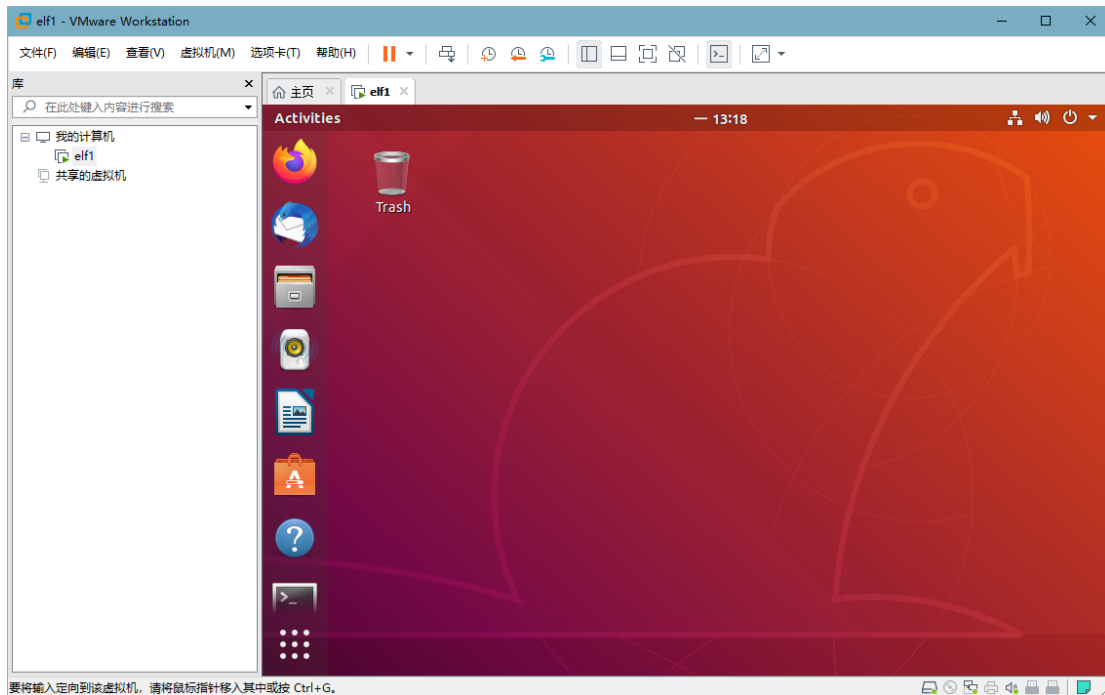


加载完成后点击开启此虚拟机，即可运行，进入系统的界面。



提供开发环境的账户为 elf，密码为 elf，填好密码后选择 Sign in 登录。



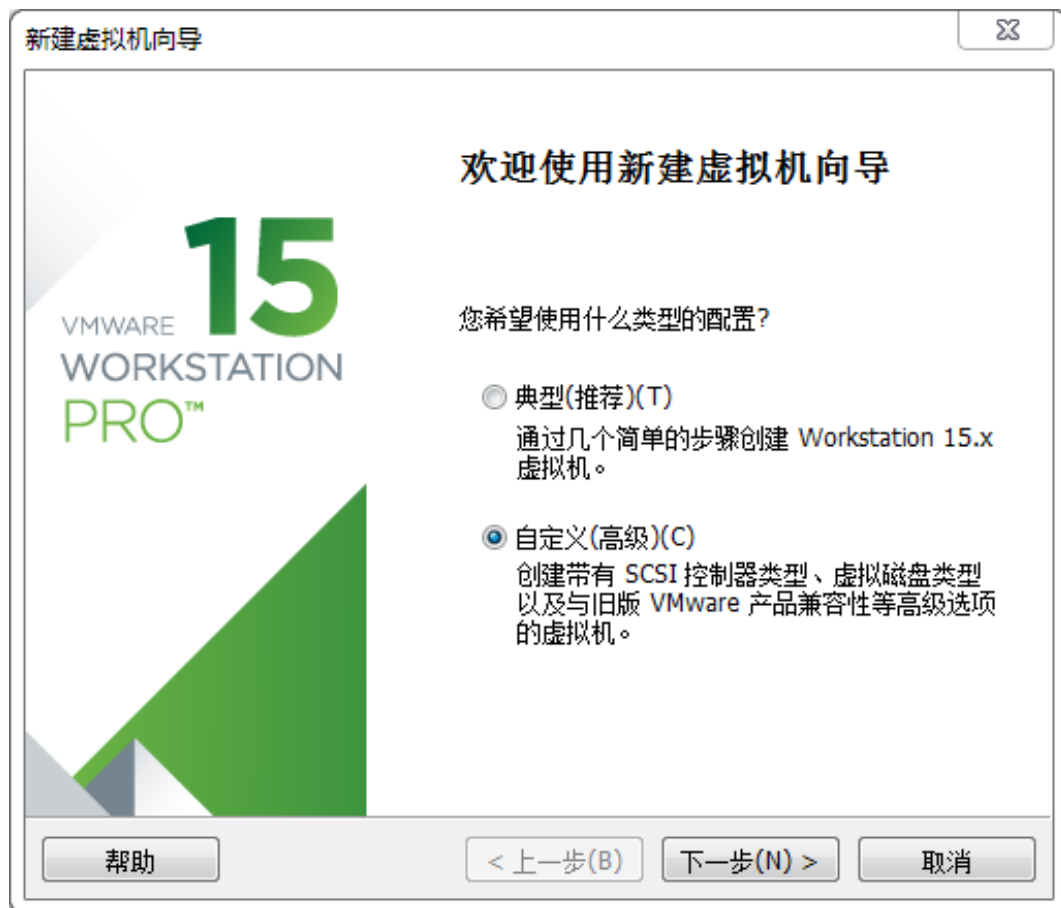


2.4 搭建自己的 ubuntu 系统

搭建系统建议安装英文版的 ubuntu，在实际使用中编译等操作都会遇到一些报错，中文版存在翻译不严谨，报错信息不好搜索解决方法的问题。此外 linux 相关开发更多的是英文资料，建议在初学的时候就养成看英文的习惯。

2.4.1 创建 ubuntu 虚拟机

打开 VMware 软件，点击创建新的虚拟机。进入以下界面：



选择自定义，点击“下一步”。



选择对应 VMware 版本的兼容性，版本可在帮助->关于 VMware Workstation 中查看，点击“下一步”。



选择稍后安装操作系统，点击“下一步”。



保持默认，点击“下一步”。



新建虚拟机向导

命名虚拟机
您希望该虚拟机使用什么名称?

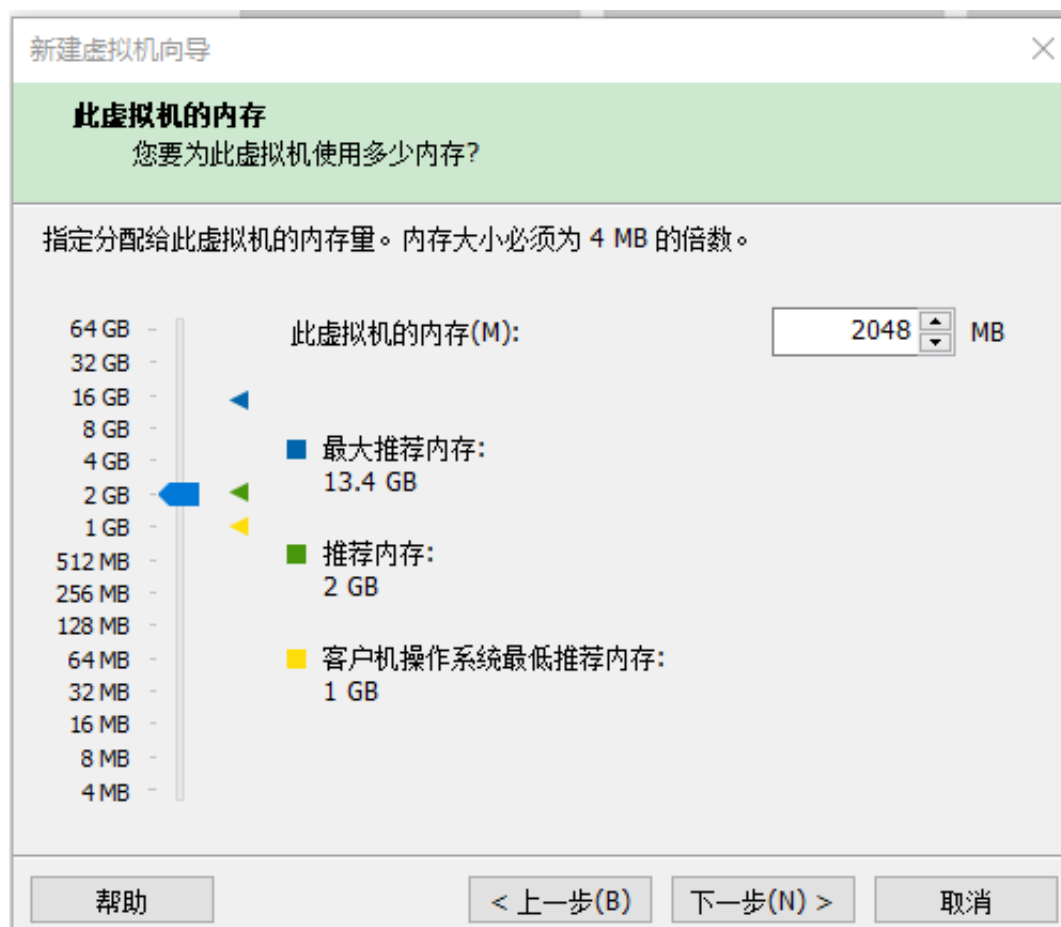
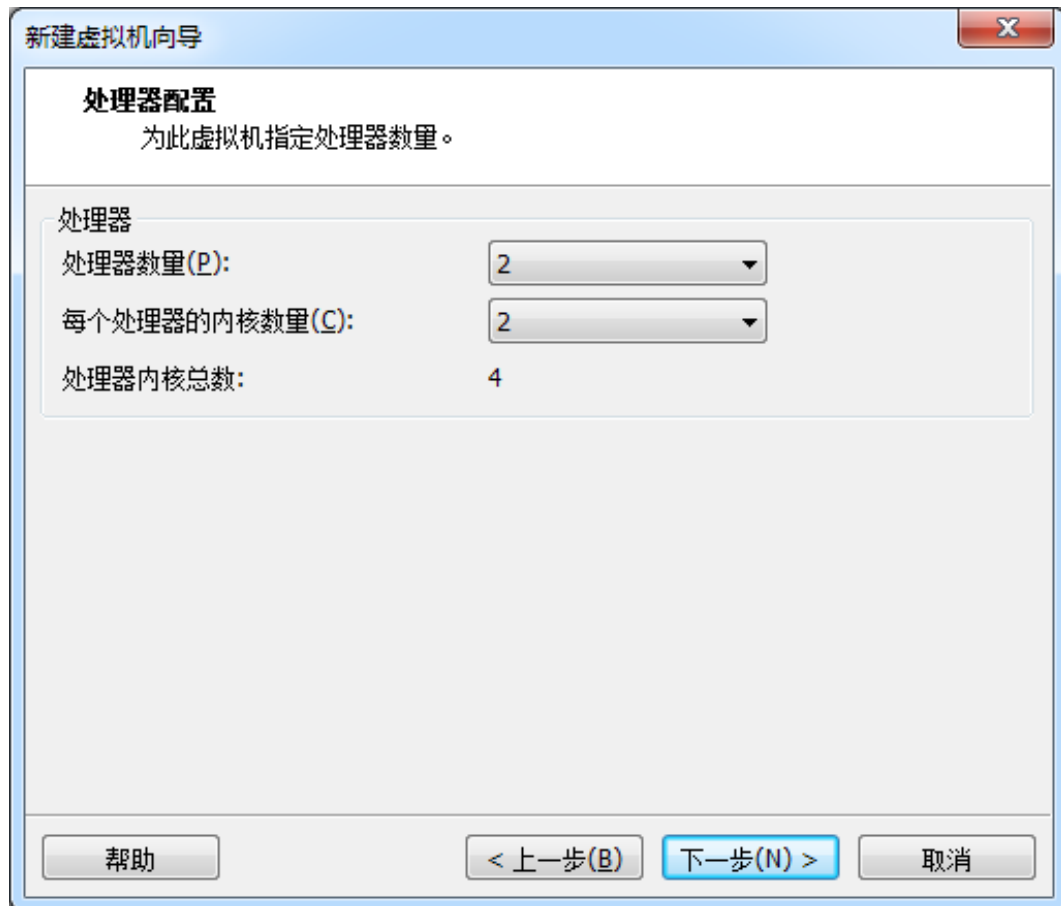
虚拟机名称(V):

位置(L):

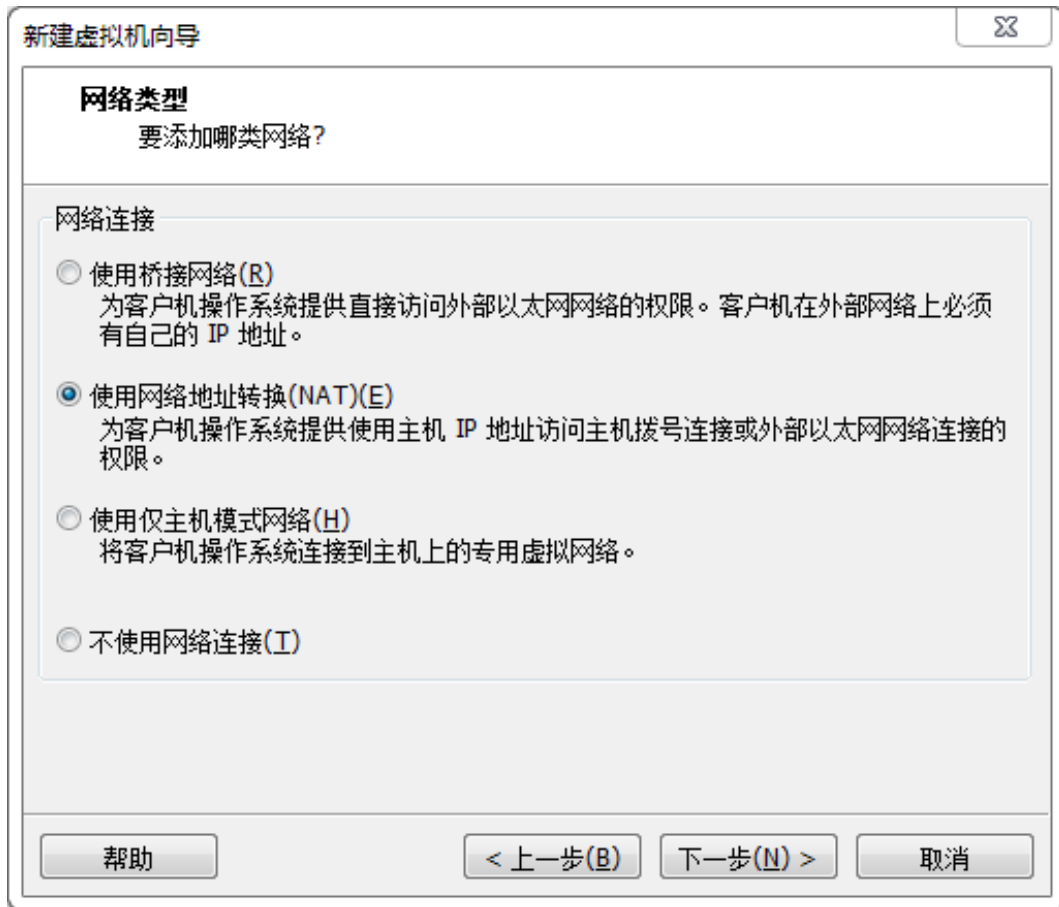
在“编辑”>“首选项”中可更改默认位置。

< 上一步(B) 下一步(N) > 取消

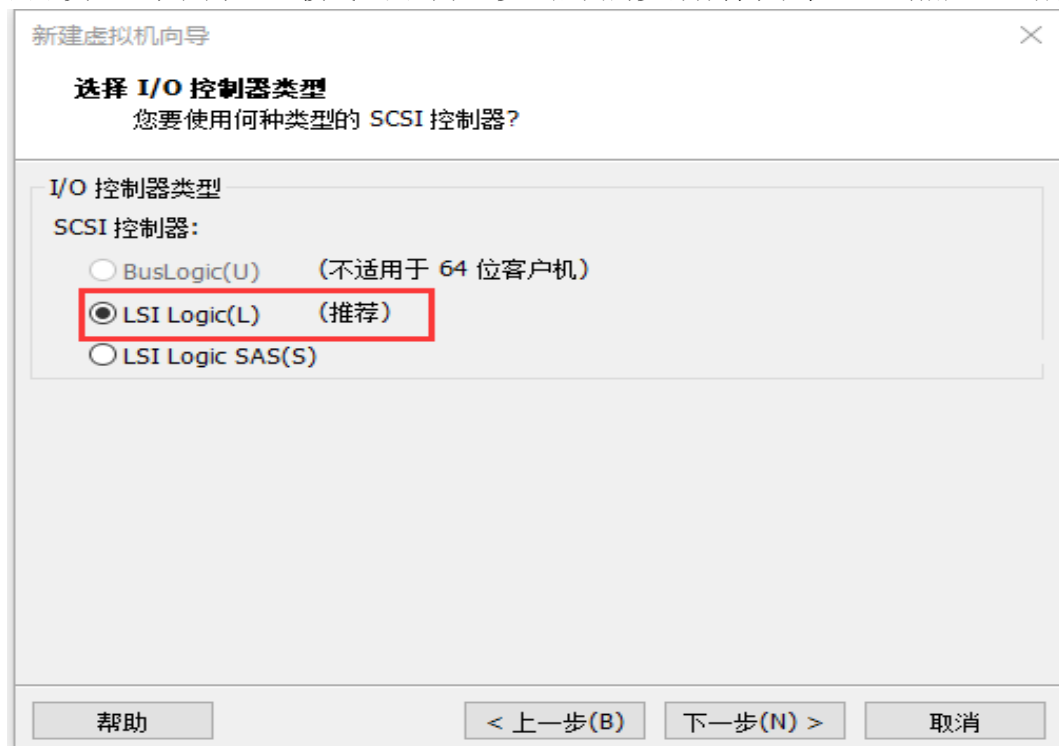
修改虚拟机名称及安装位置，点击“下一步”。



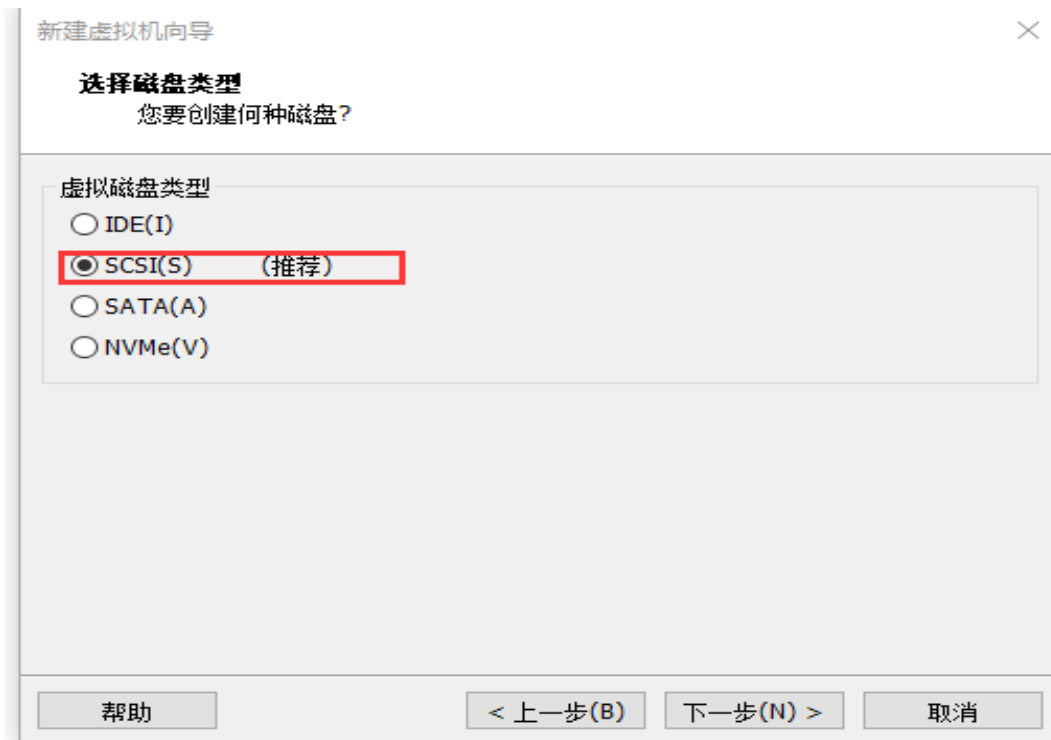
同样按照实际情况设置内存大小。



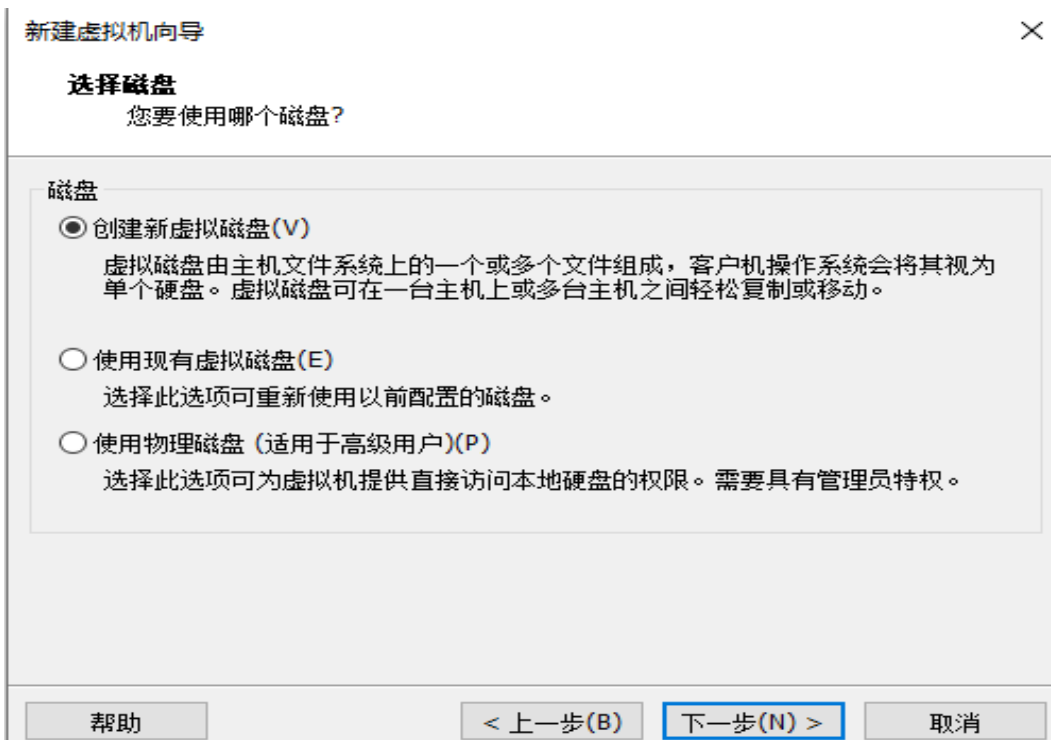
设置网络类型，默认为 NAT 模式，点击下一步。后面的步骤保持默认值，直到指定磁盘容量步骤。



IO 控制器类型这里默认选择 LSI 就可以：



这里同样是默认选择 SCSI。



这里选择创建新的虚拟磁盘：

新建虚拟机向导

指定磁盘容量
磁盘大小为多少?

最大磁盘大小 (GB)(S):

针对 Ubuntu 的建议大小: 20 GB

☐ 立即分配所有磁盘空间(A)。
分配所有容量可以提高性能，但要求所有物理磁盘空间立即可用。如果不立即分配所有空间，虚拟磁盘的空间最初很小，会随着您向其中添加数据而不断变大。

☐ 将虚拟磁盘存储为单个文件(O)

☒ 将虚拟磁盘拆分成多个文件(M)
拆分磁盘后，可以更轻松地在计算机之间移动虚拟机，但可能会降低大容量磁盘的性能。

帮助

< 上一步(B)

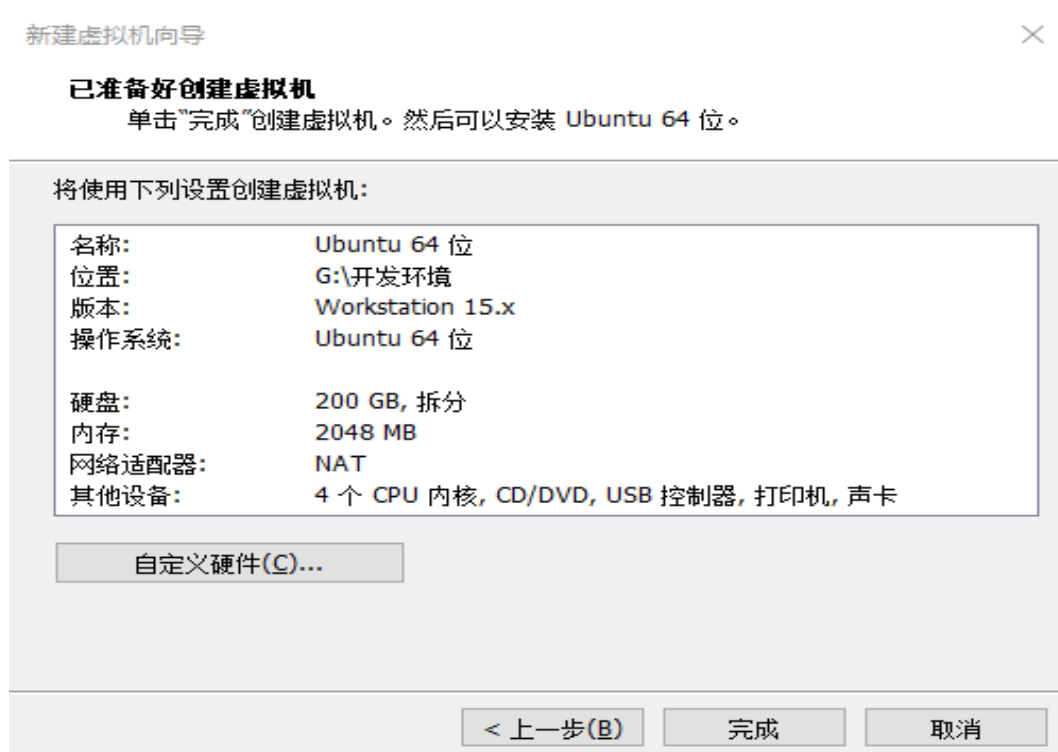
下一步(N) >

取消

设置磁盘大小为 200G，并选择磁盘的存在形式，然后点击“下一步”完成。



指定磁盘文件，这里默认即可。



默认点击“完成”即可。



至此，虚拟机创建完成。

下一小节中我们介绍系统在虚拟机中的安装，其在真机中的安装方法与虚拟机类似。这里我们介绍在虚拟机中安装 **ubuntu** 系统的方法。

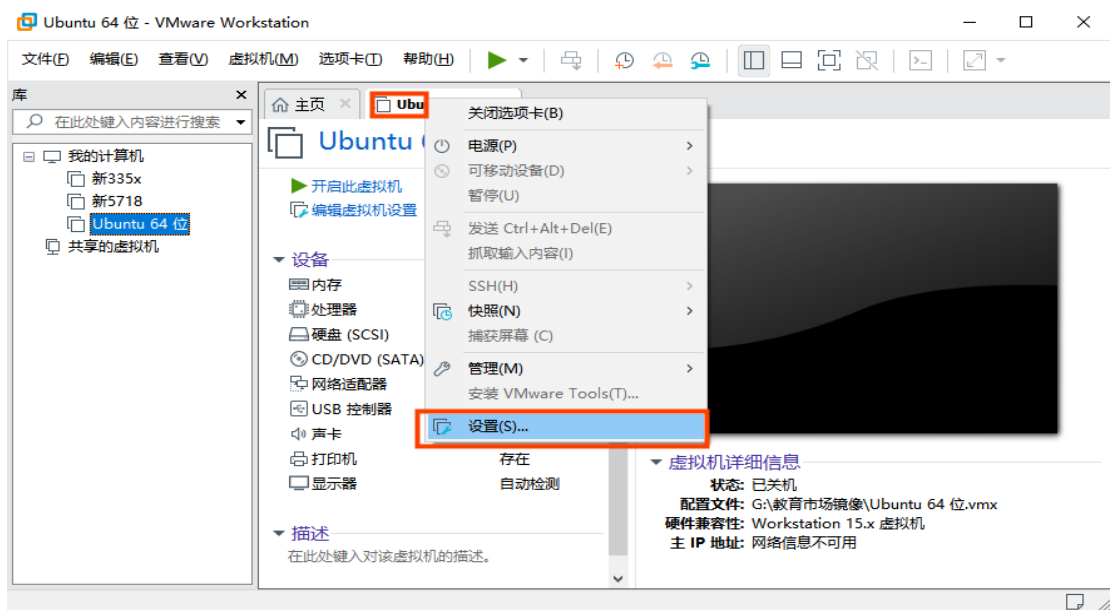
2.4.2 系统安装

注意：本文档以 **Ubuntu 18.04** 为例进行描述，其他版本的安装步骤可以参照此文进行相应操作。

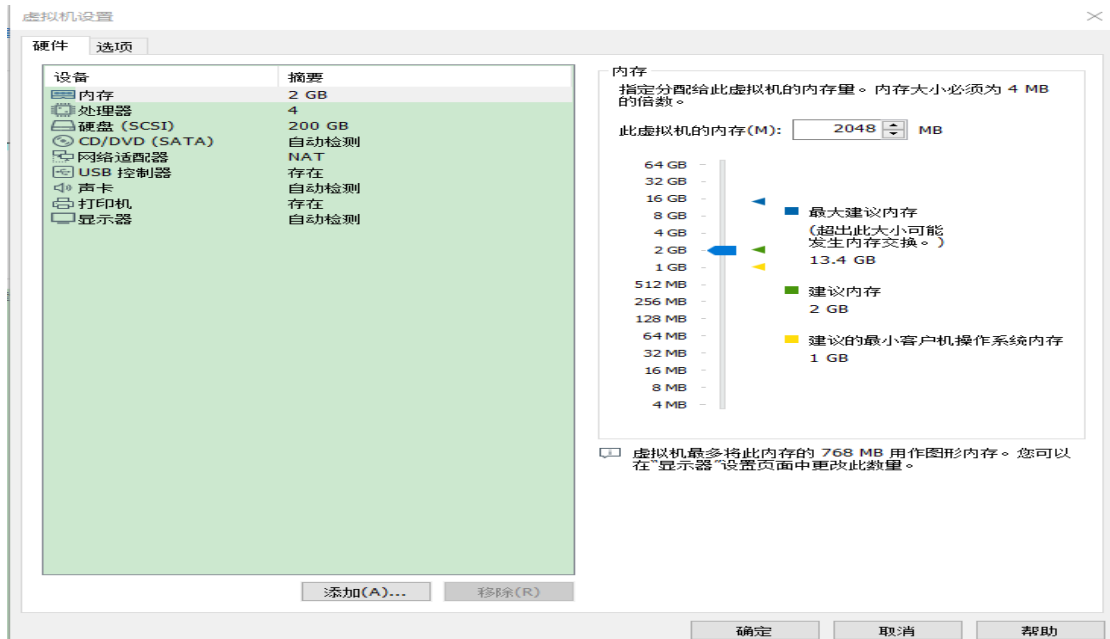
我们选择安装的 **ubuntu** 版本是 **18.04**，首先去 **ubuntu** 官网获取 **ubuntu18.04 64 位** 镜像，下载地址为：<http://releases.ubuntu.com/18.04/> 下载“**ubuntu-18.04.5-desktop-amd64.iso**”这个版本。

	ubuntu-18.04.6-desktop-amd64.iso	2021-09-15 20:42	2.3G	Desktop image for 64-bit PC (AMD64) computers (standard download)
---	--	------------------	------	---

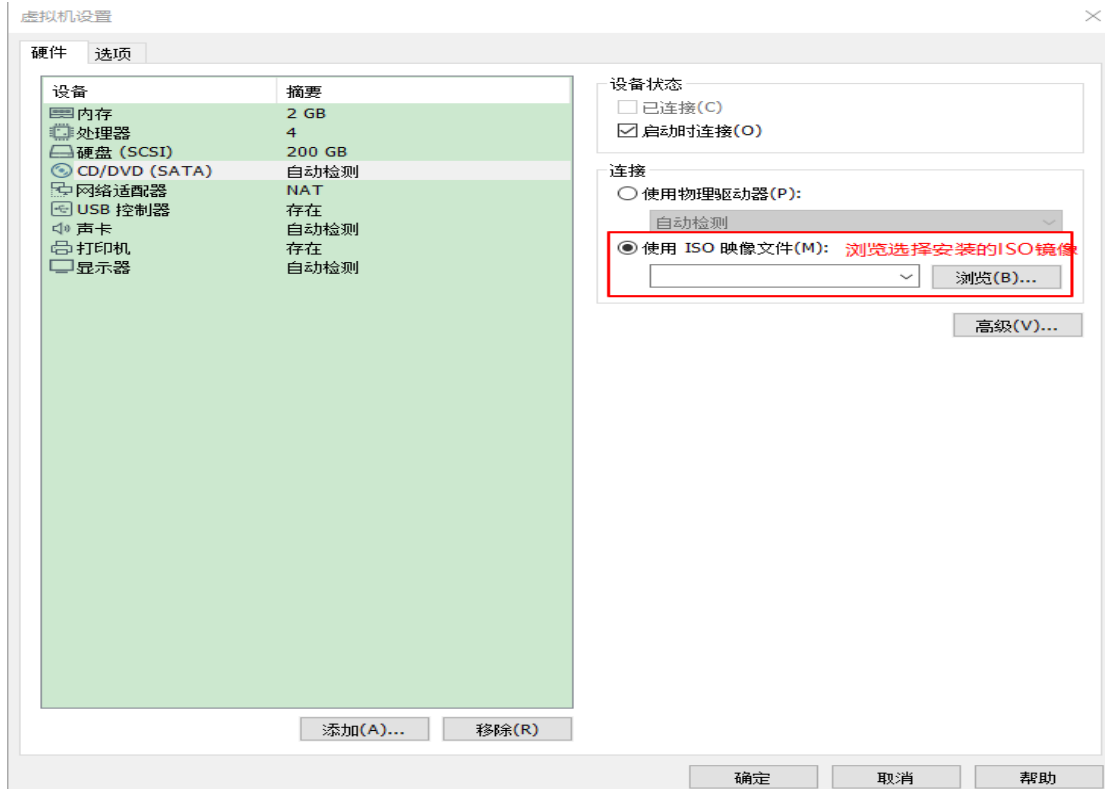
右击刚创建完成的 **ubuntu64 位** 在弹出菜单中选择设置：



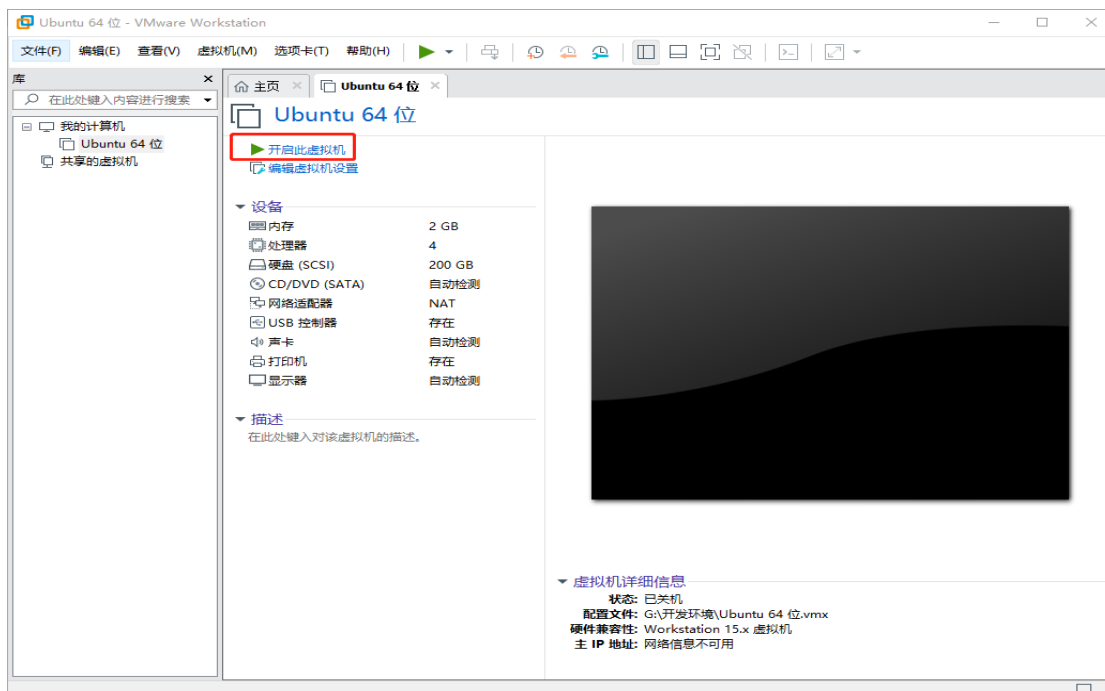
弹出“虚拟机设置菜单”根据如下图：



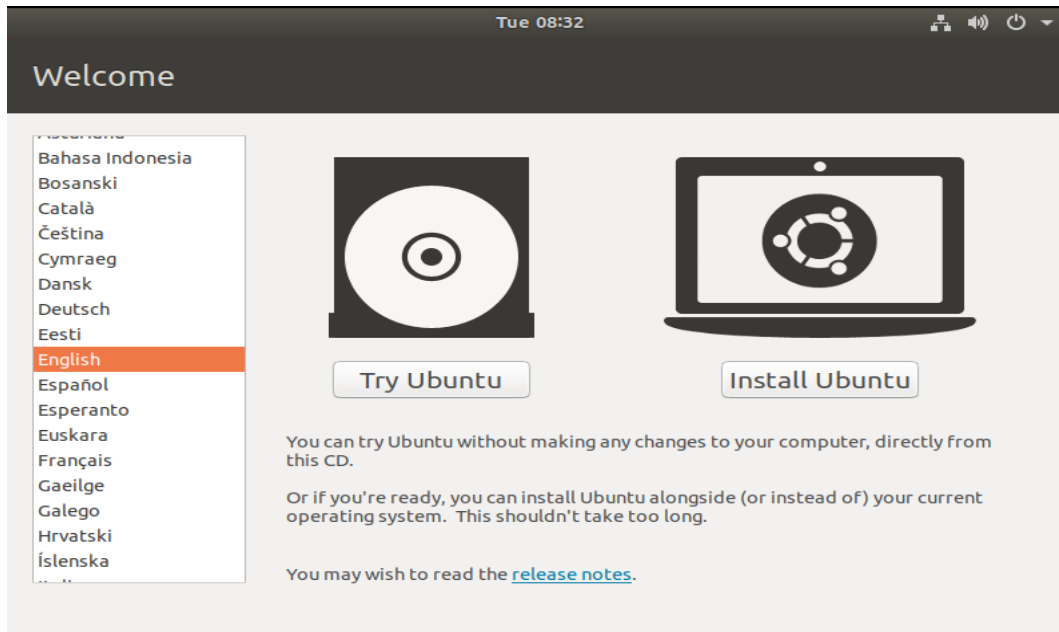
点击 CD/DVD (SATA)，选择使用 ISO 映像文件，浏览选择前面下载的 ubuntu 镜像，然后确定。



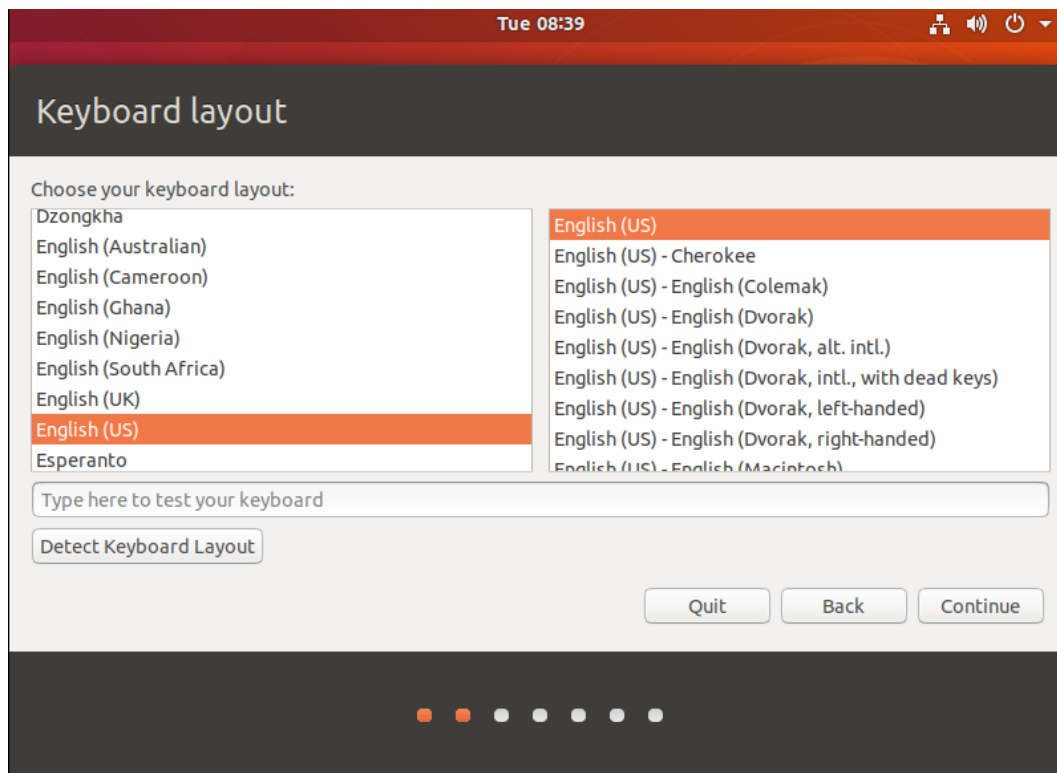
设置好镜像后，保证网络可用，然后开启虚拟机，进行 ubuntu 镜像的安装。



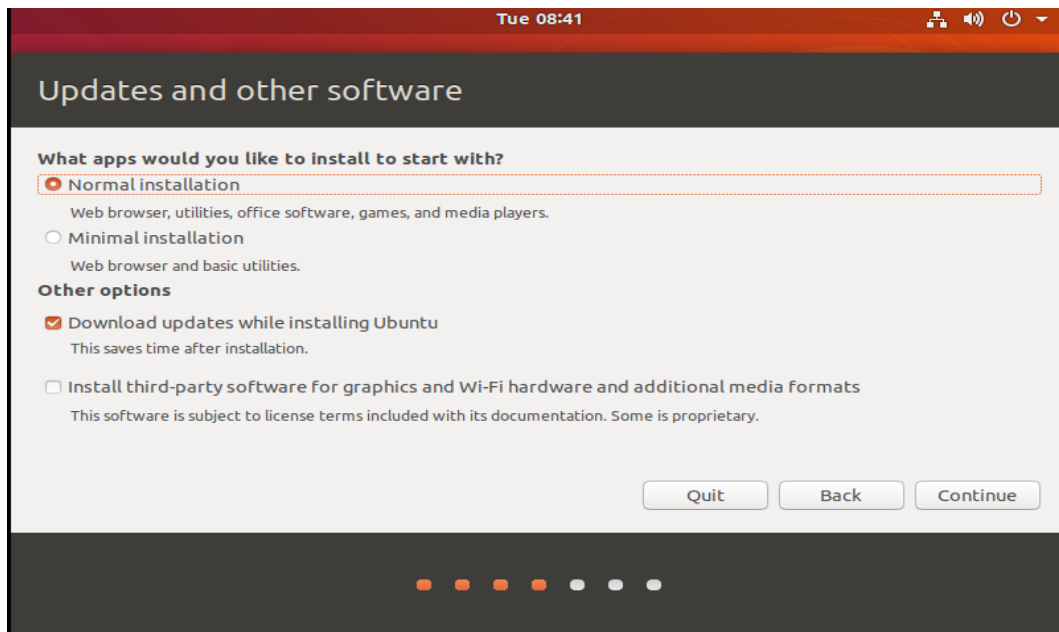
开启虚拟机后，等待出现安装界面如下：



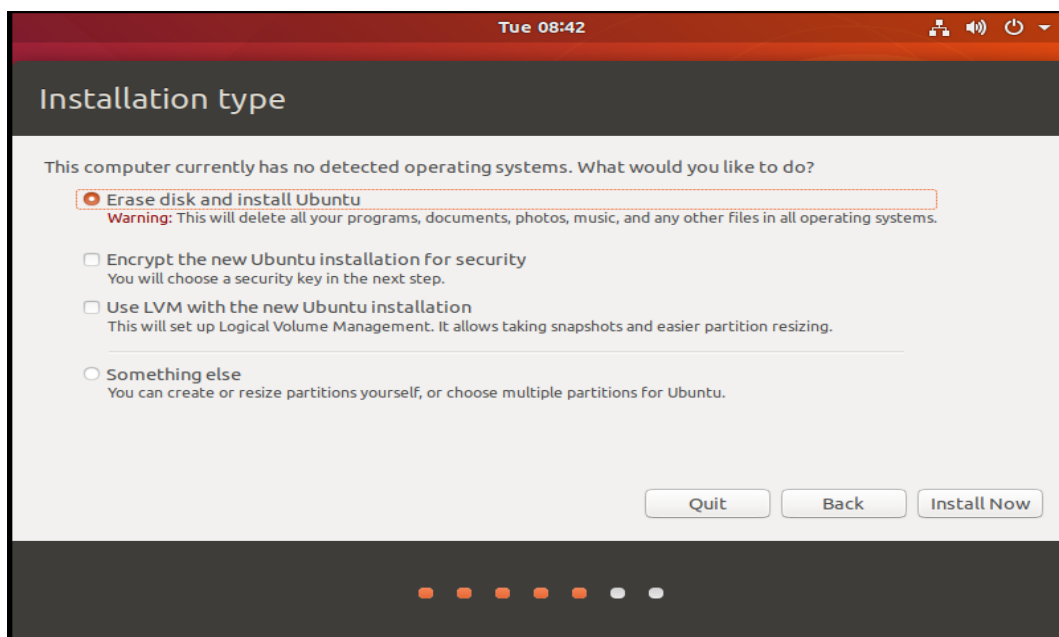
如图左侧选择语言后,点击“Install Ubuntu”后弹出选择语言界面。ubuntu 默认语言是英文的,当然,也可以选择中文,默认选择的语言在后期也是可以重新设置的,选择完成后 continue。



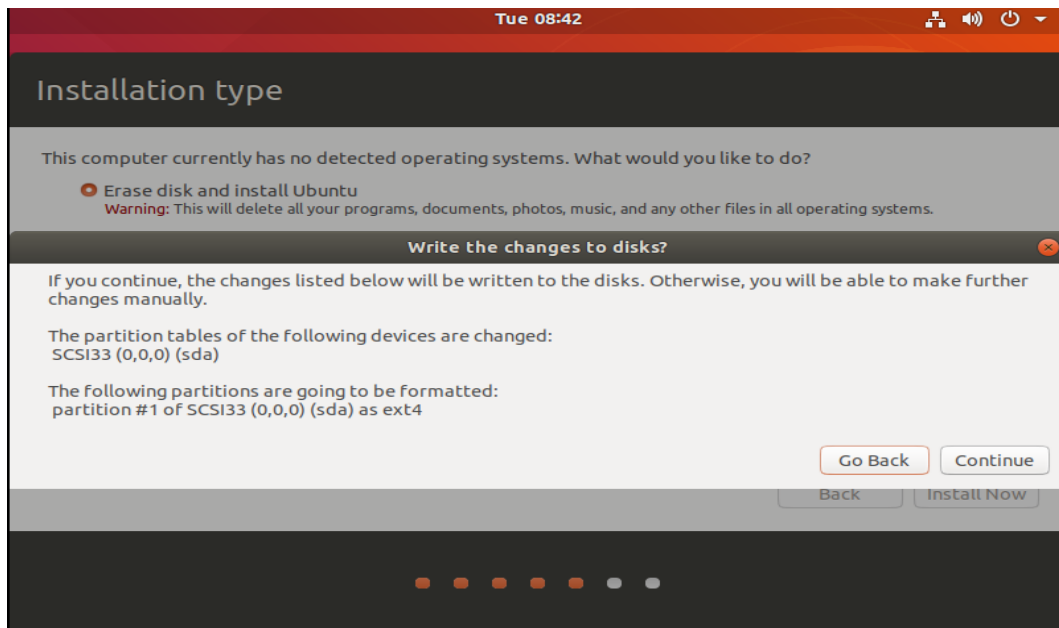
接下来,默认选择 continue 继续安装,安装过程会很慢,然后点击“continue”:



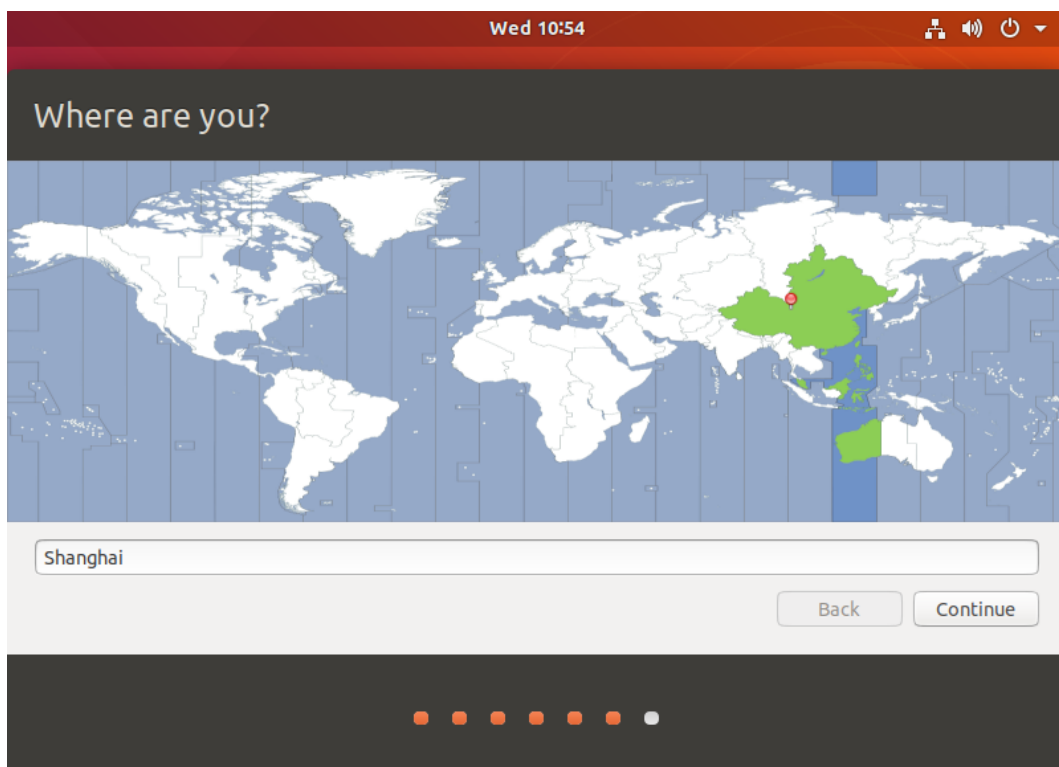
默认，点击 Install Now。



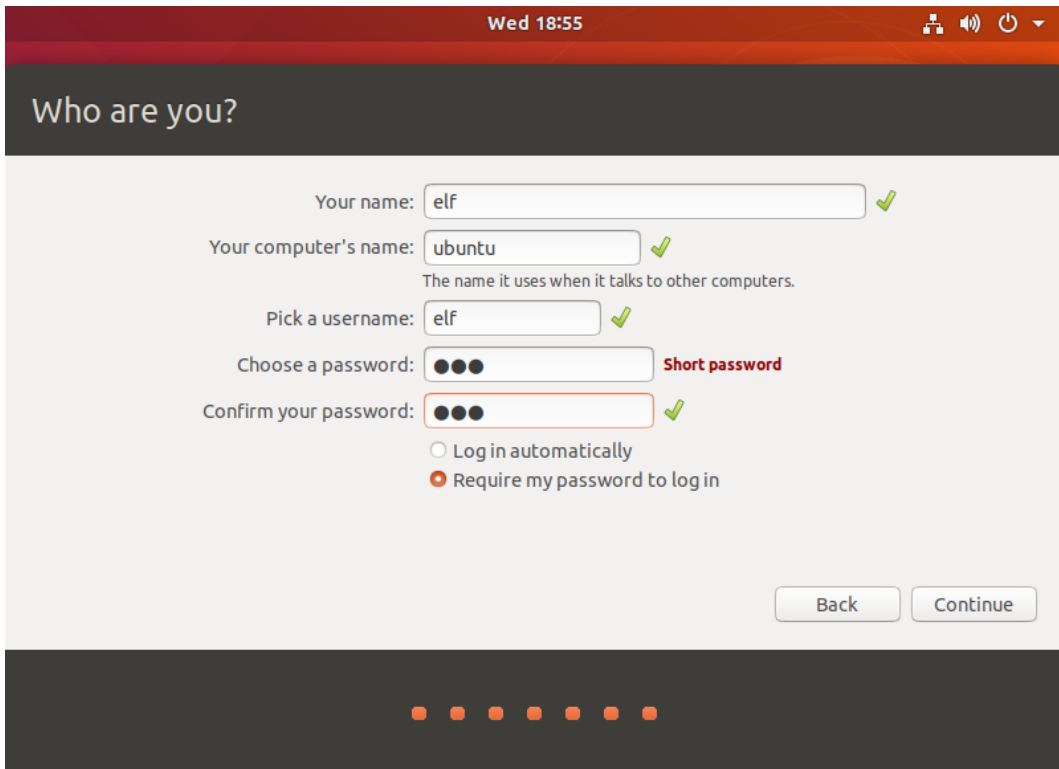
会弹出下图，点击“continue”即可。



接下来选择时区，这里点击上海时区或输入 Shanghai 即可（不同时区根据实际情况选择即可），点击“继续”。



最后设置用户名和密码，点击“continue”就会自动安装：



Wed 18:55

Who are you?

Your name: ✓

Your computer's name: ✓
The name it uses when it talks to other computers.

Pick a username: ✓

Choose a password: Short password

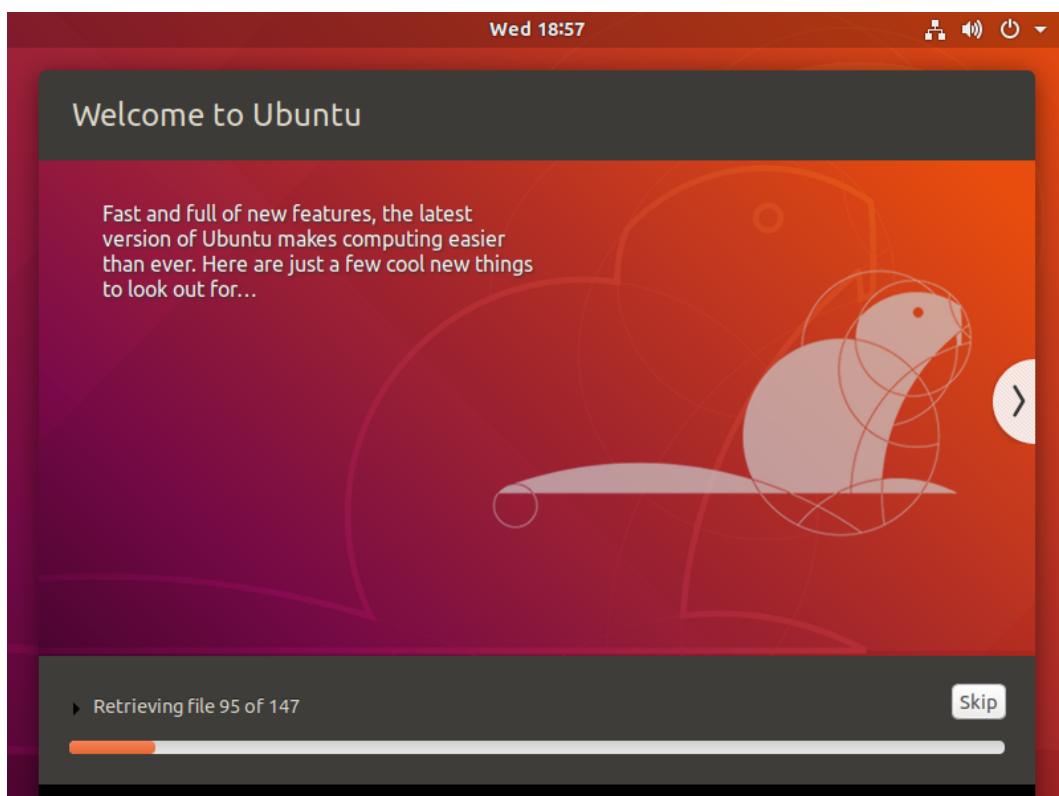
Confirm your password: ✓

☐ Log in automatically
☒ Require my password to log in

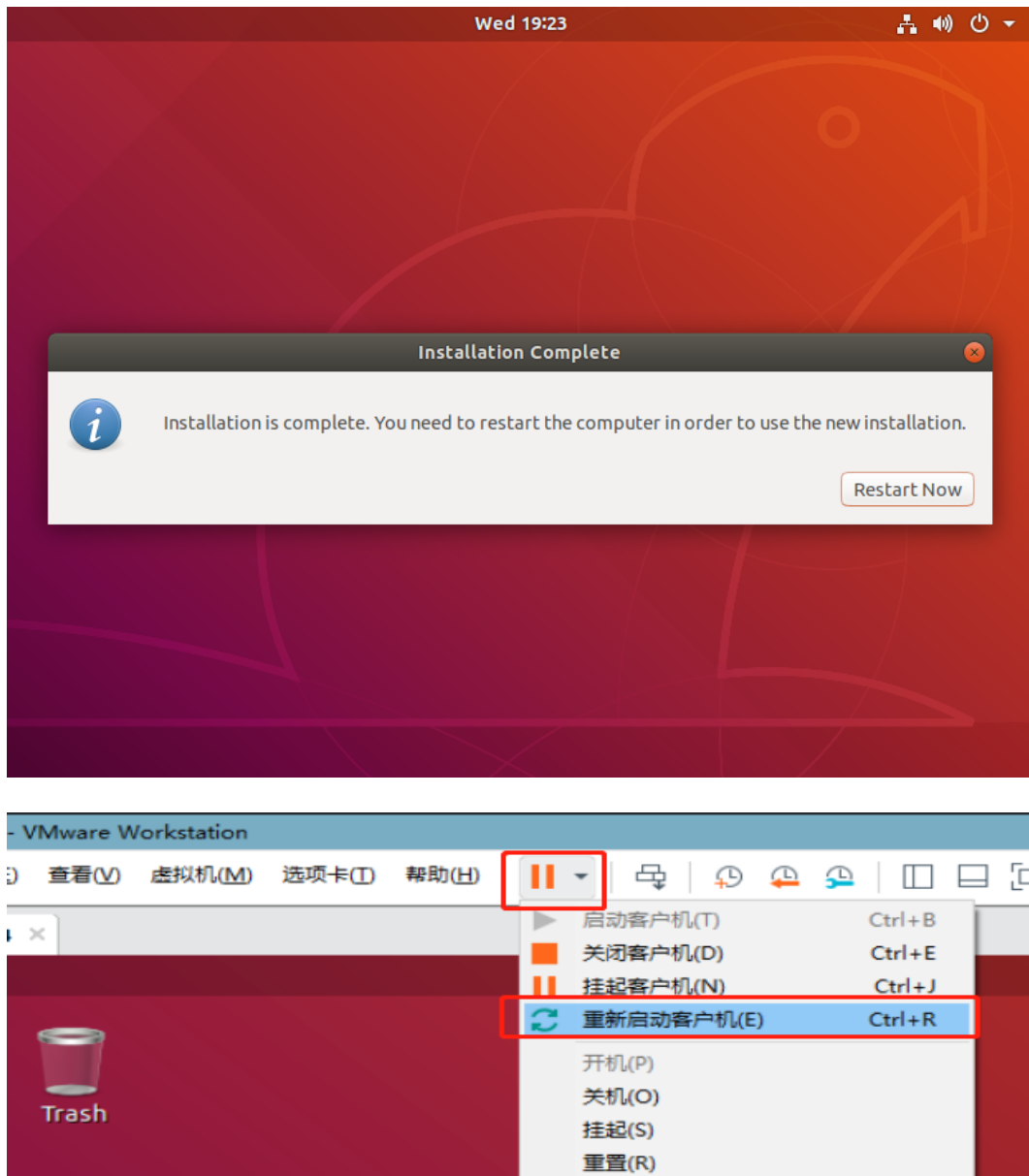
Back Continue

Progress indicator: 7 dots, 6th dot is highlighted.

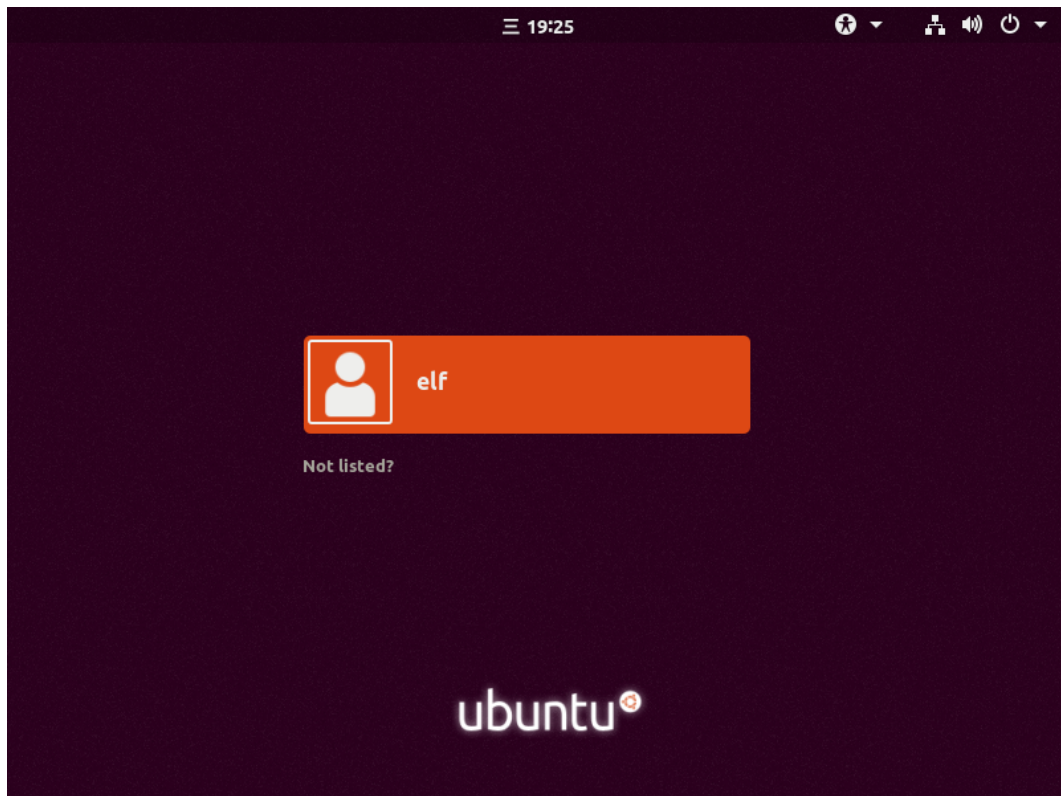
安装过程下图，网络不好可以 Skip 跳过，不影响安装。



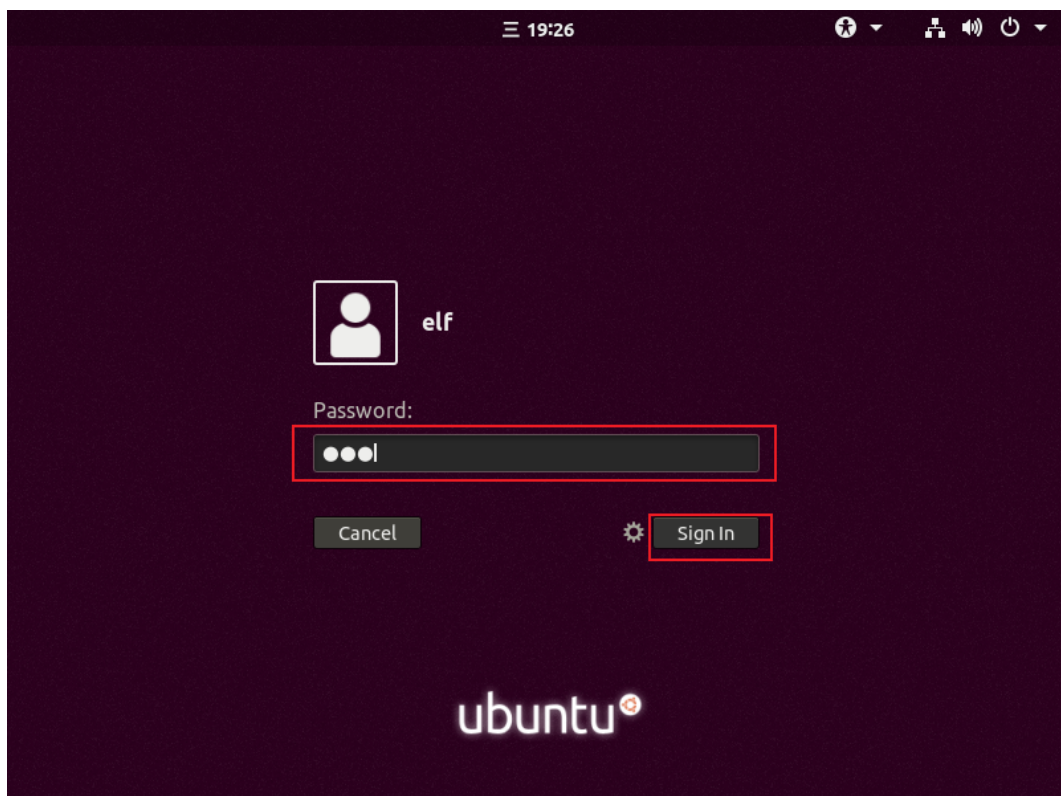
安装完成后显示如下图，点击“Restart Now”重启（或者点击“重新启动客户机”）：



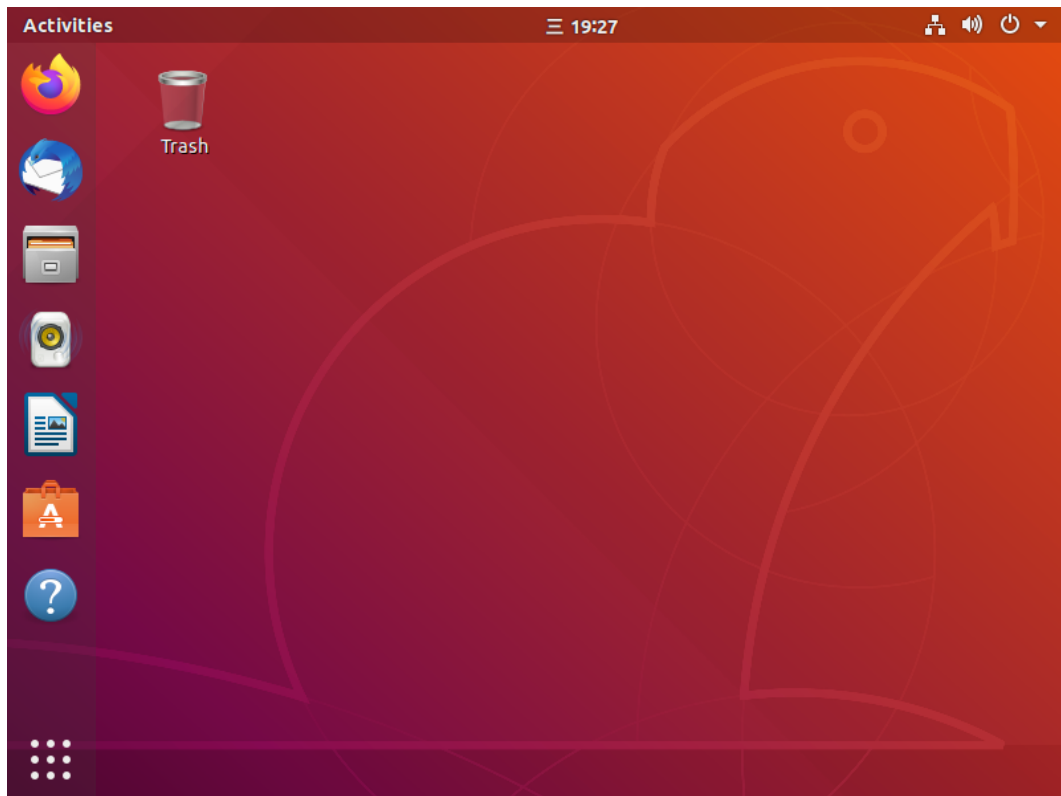
重启完成需要使用用户名和密码登录，登录后系统界面如下图：



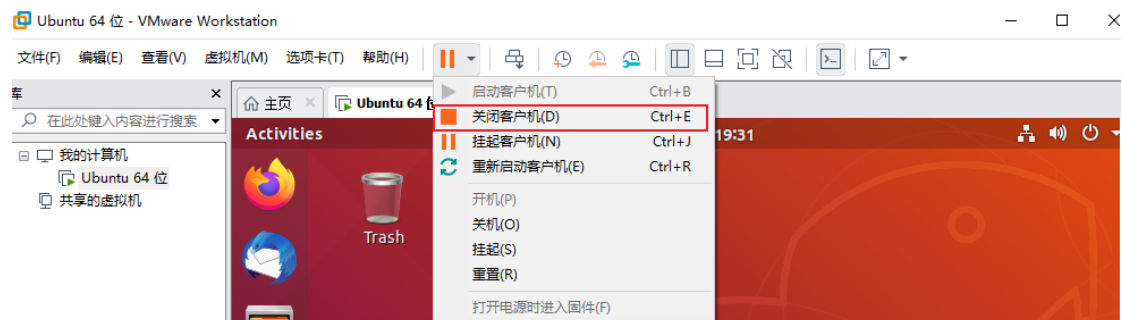
点击账户“elf”，输入密码，完成登录。



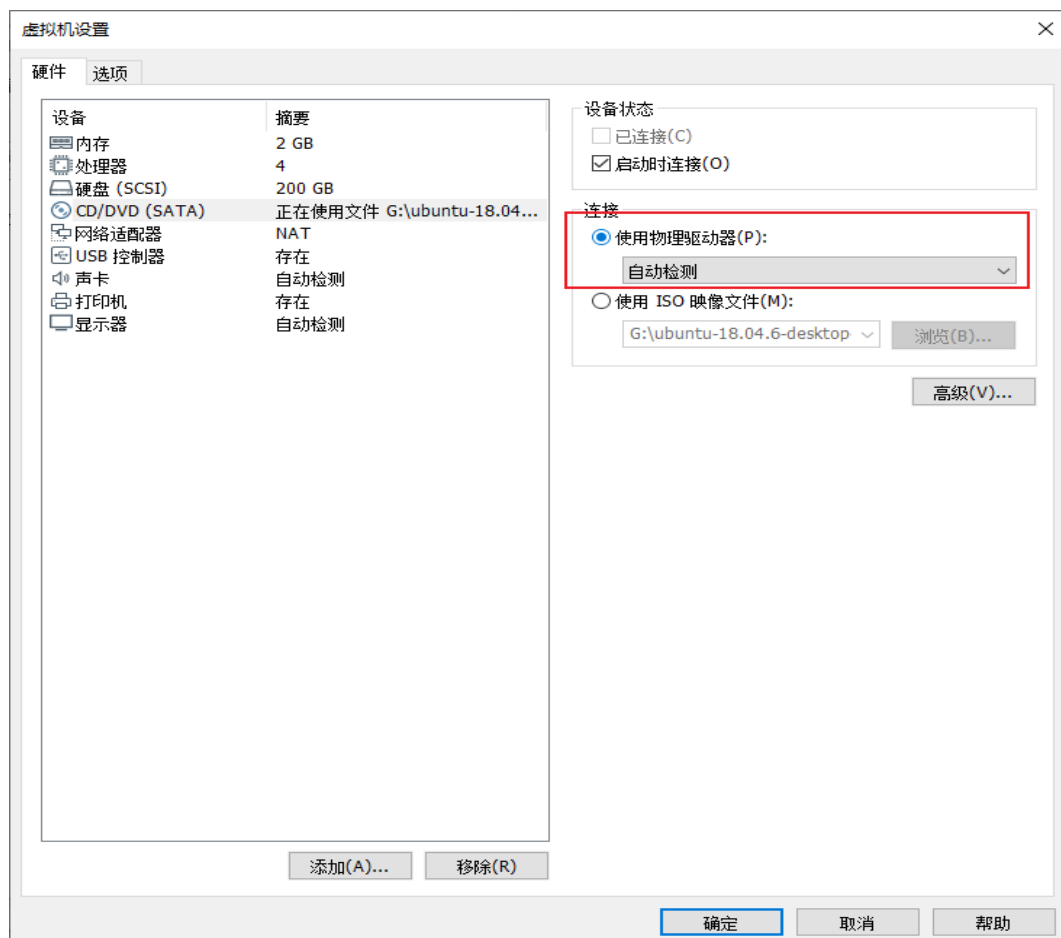
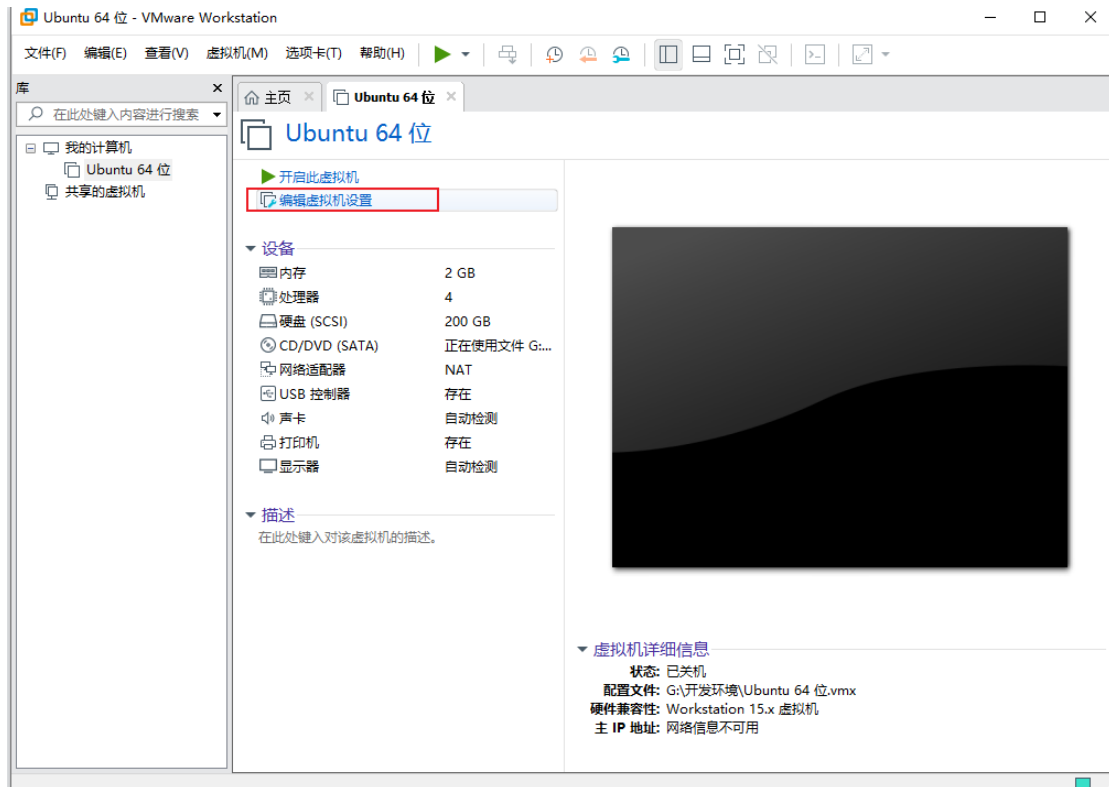
进入 ubuntu 系统桌面



以上，ubuntu 系统安装完成按照下图配置，点击确定，关闭客户机。



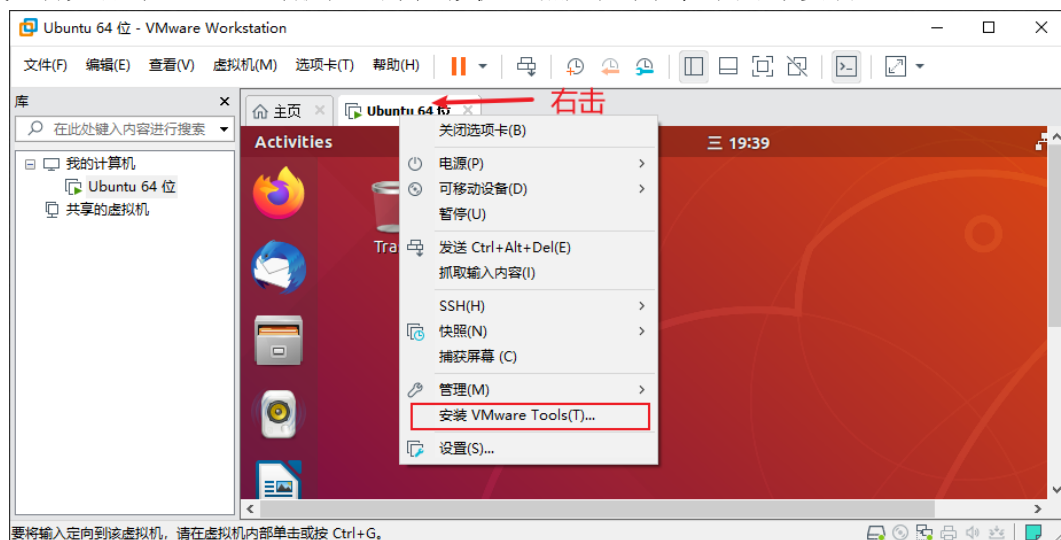
点击“编辑虚拟机设置”，如下图重新设置“使用物理驱动器”



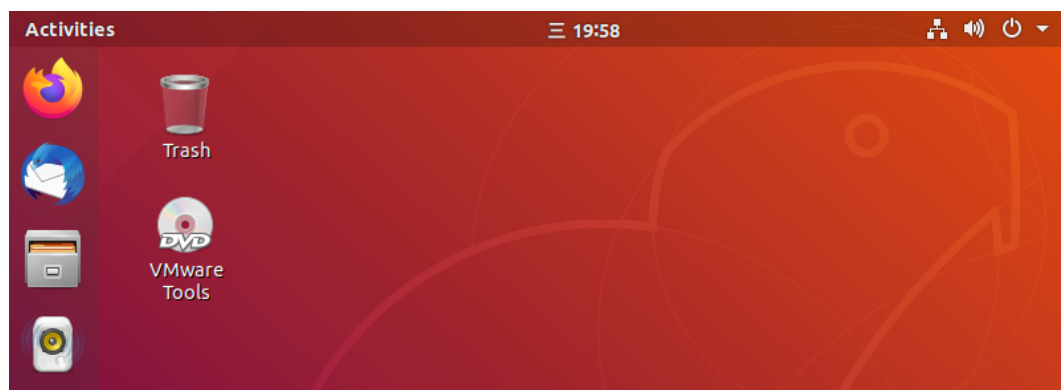
然后重新打开虚拟机，看能否正常启动 ubuntu。

2.4.3 VMware Tools 安装

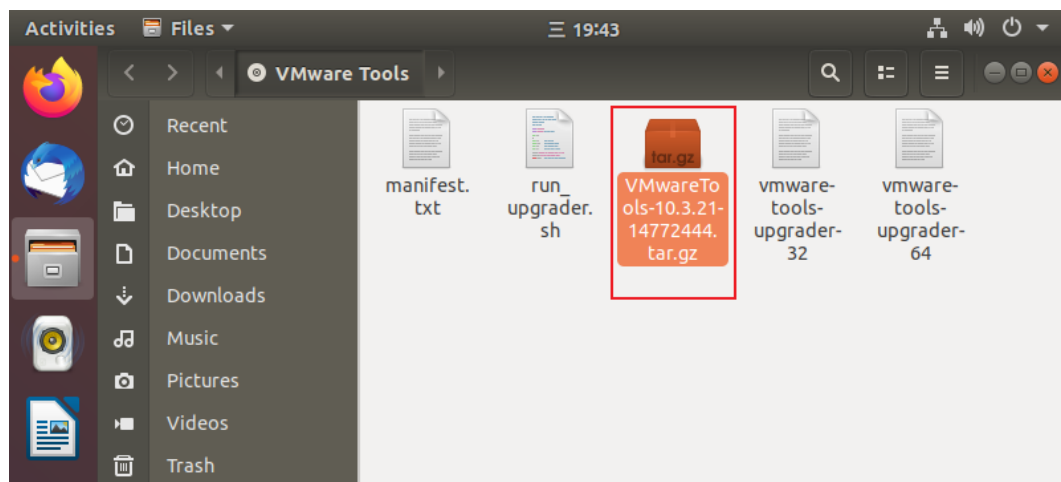
接下来安装 VMware Tools，如果不安装该工具，在 Windows 主机和虚拟机之间无法使用复制粘贴、文件拖拽。首先右击 VMware 导航栏上的“虚拟机”，然后在下拉框中点击“安装 VMware Tools”：



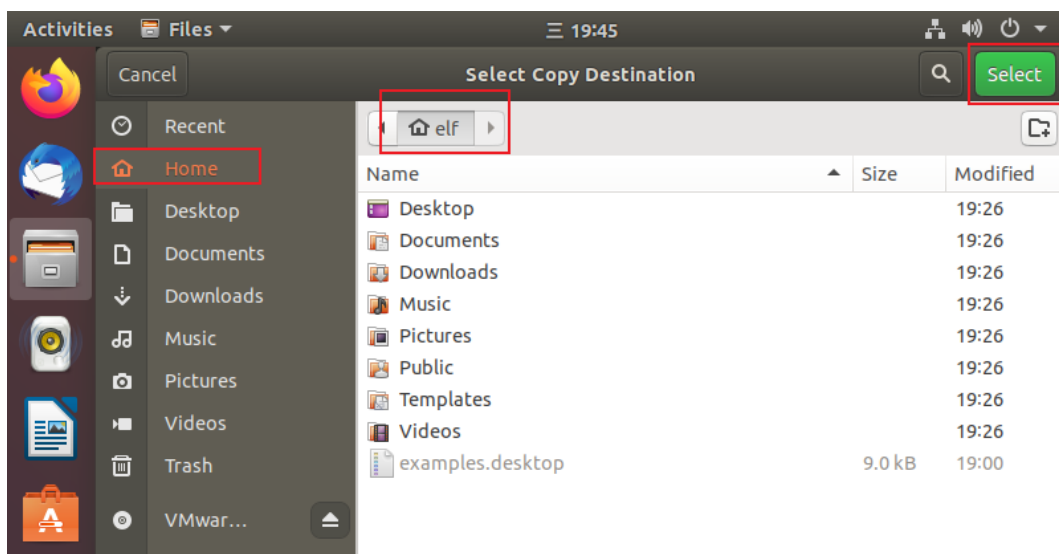
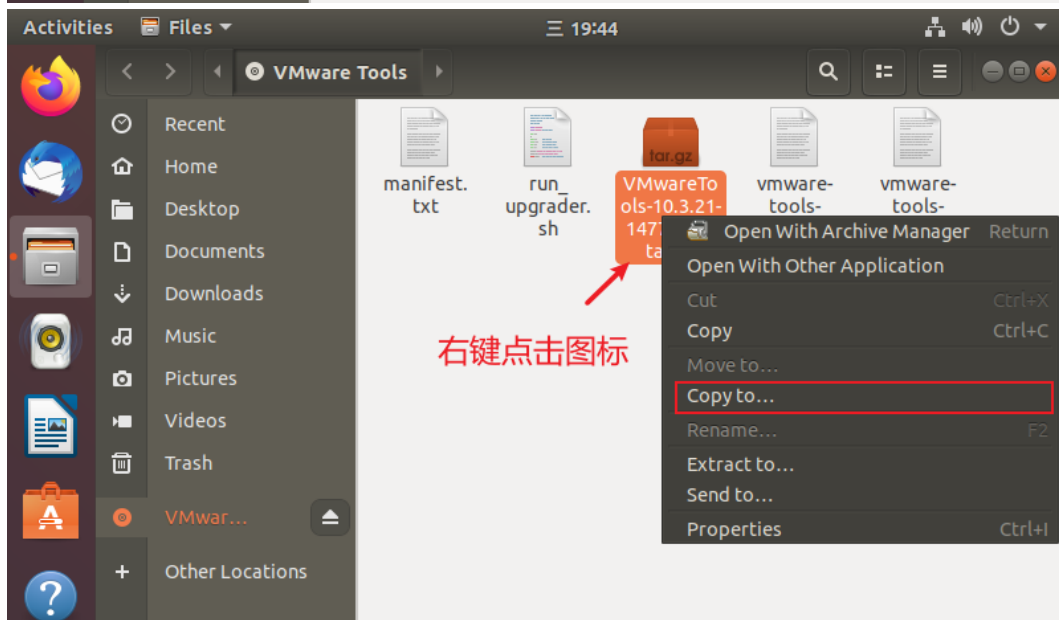
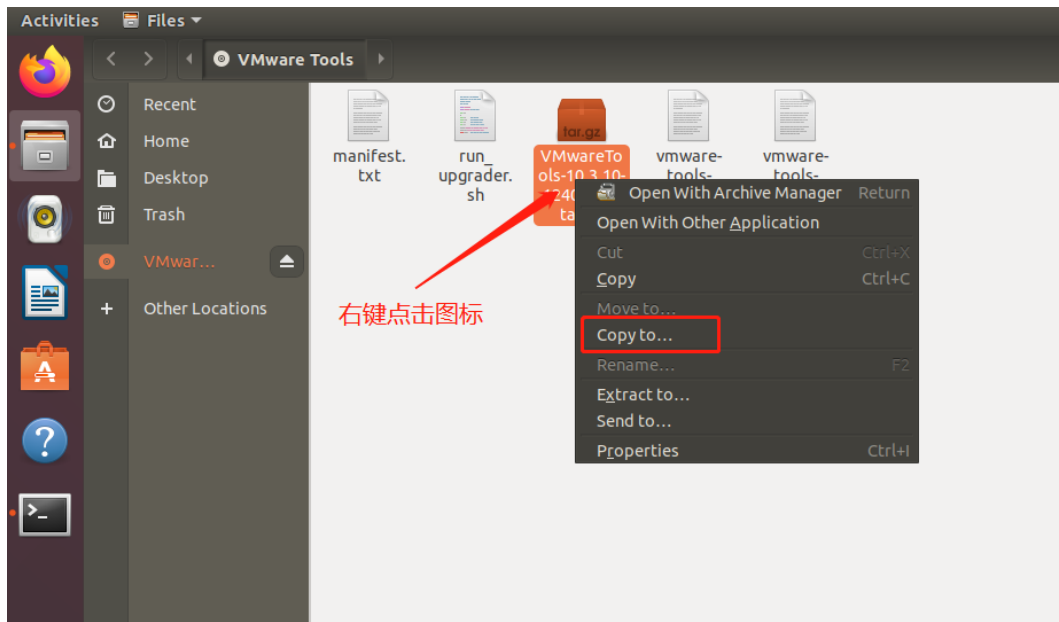
完成后进入 ubuntu，桌面会出现 VMware Tools 的光盘图标，点击进入其中：

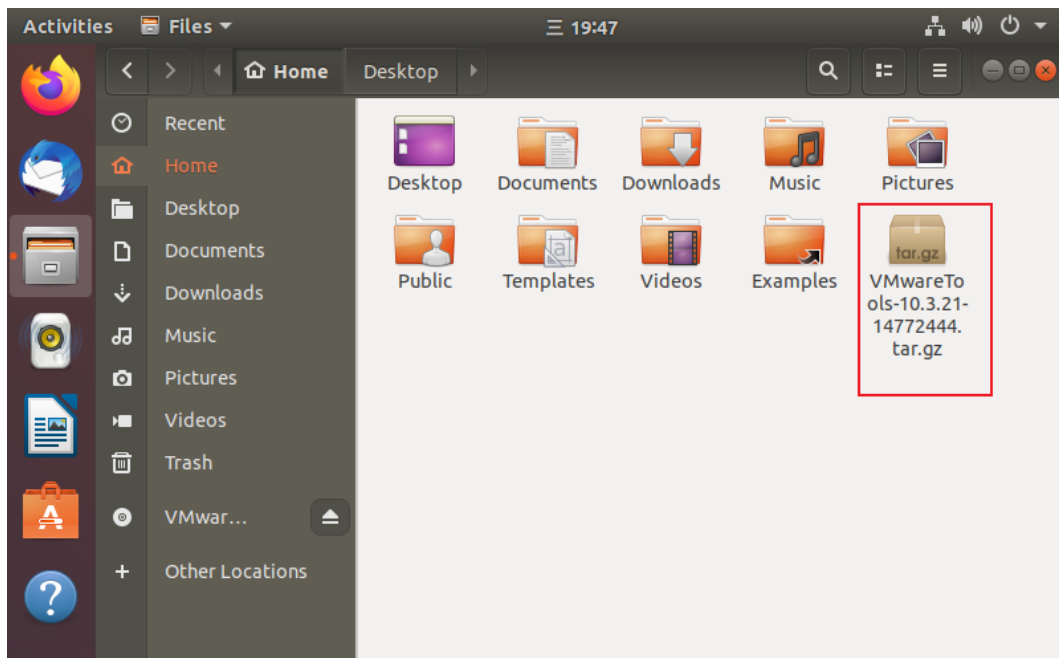


双击 VMwareTools 图标，进入后看到一个压缩文件 VMwareTools-10.3.21-14772444.tar.gz（不同的虚拟机版本可能会不同）。



复制文件到主目录下面（即 home 个人用户名的目录下）：

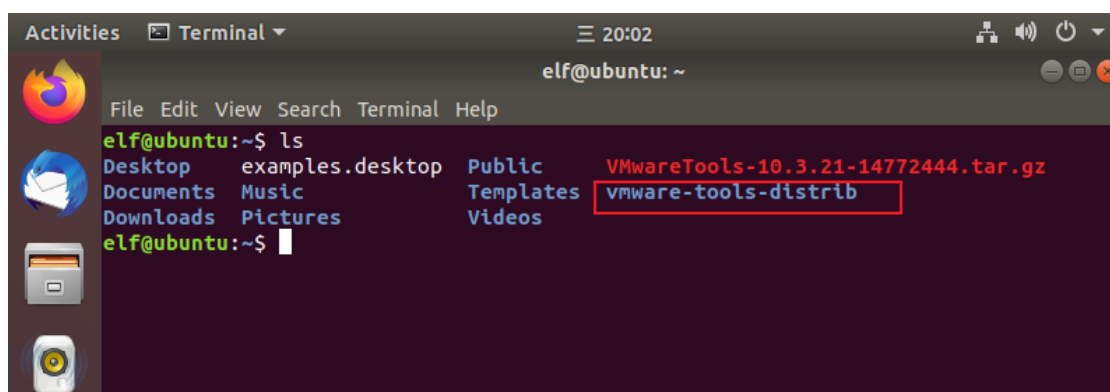




按键盘【Ctrl+Alt+T】调出终端命令界面，使用 tar 命令对 VMwareTools 安装包解压

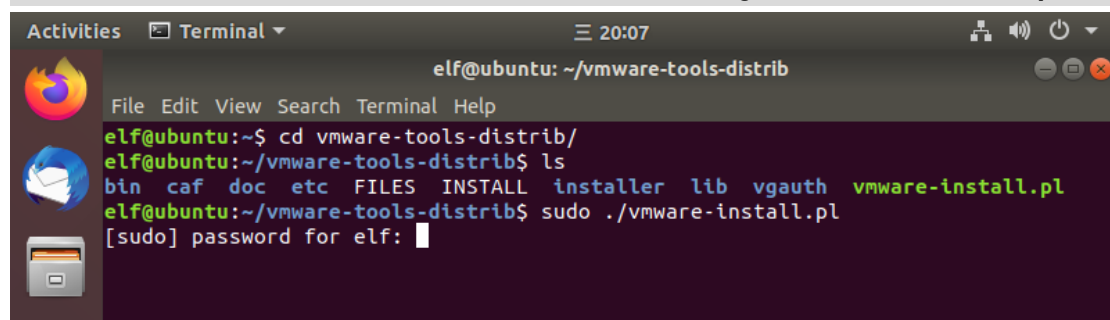
```
elf@ubuntu:~$ tar xvf VMwareTools-10.3.21-14772444.tar.gz
```

执行完解压命令后，使用 ls 查看，会出现一个 vmware-tools-distrib 的文件目录，进入到该目录。



进入到该目录，使用 sudo 运行 vmware-install.pl 进行安装（使用 sudo 命令会提示输入密码，根据提示直接输入密码回车即可，Linux 系统密码输入无显示）：

```
elf@ubuntu:~$ cd vmware-tools-distrib/           //使用 cd 命令进入该目录
elf@ubuntu:~/vmware-tools-distrib$ ls             //查看该目录下的文件
bin  caf  doc  etc  FILES  INSTALL  installer  lib  vgauth  vmware-install.pl
```



在当前目录下，输入 sudo ./vmware-install.pl，进行安装，回车后输入密码，然后就开始安装，遇到 yes 就输入 yes，其他一律回车默认安装就可以。

```
elf@ubuntu:~/vmware-tools-distrib$ sudo ./vmware-install.pl
[sudo] password for elf:
open-vm-tools packages are available from the OS vendor and VMware recommends
using open-vm-tools packages. See http://kb.vmware.com/kb/2073803 for more
information.
Do you still want to proceed with this installation? [no] yes          //输入 yes
... 此处省略...

Enjoy,

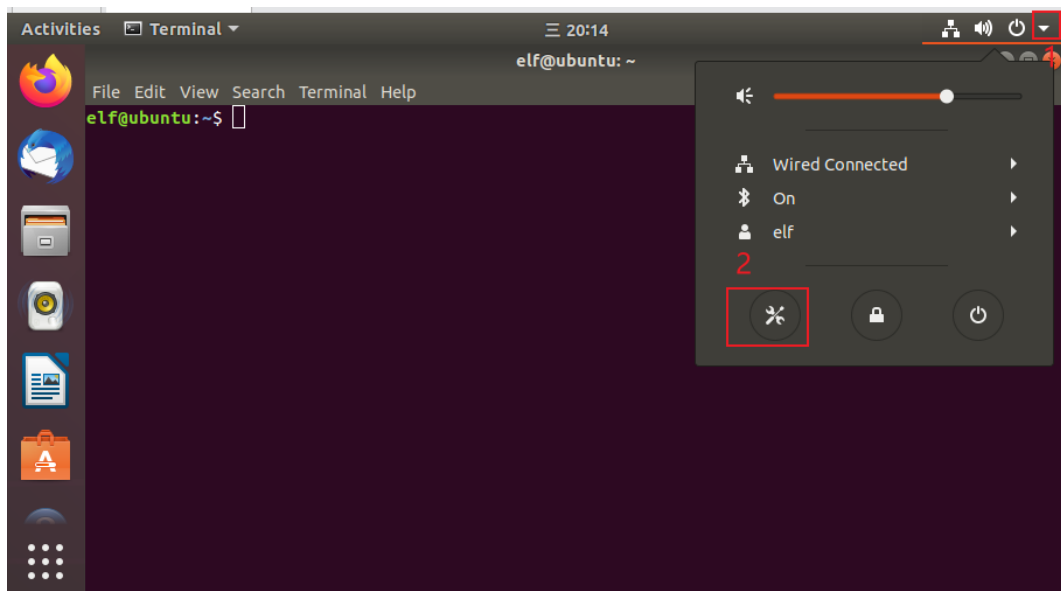
- - the VMware team
```

安装过程信息较长，此处省略。

VMware tools 工具完成后，可以实现 Windows 和 ubuntu 之间的文件复制粘贴，虚拟机自适应全显等功能。如果虚拟机不能够全屏显示，可以通过点击查看，选择自动调整大小，点击自动适应客户机，即可实现虚拟机的全屏显示，VMware tools 安装成功。



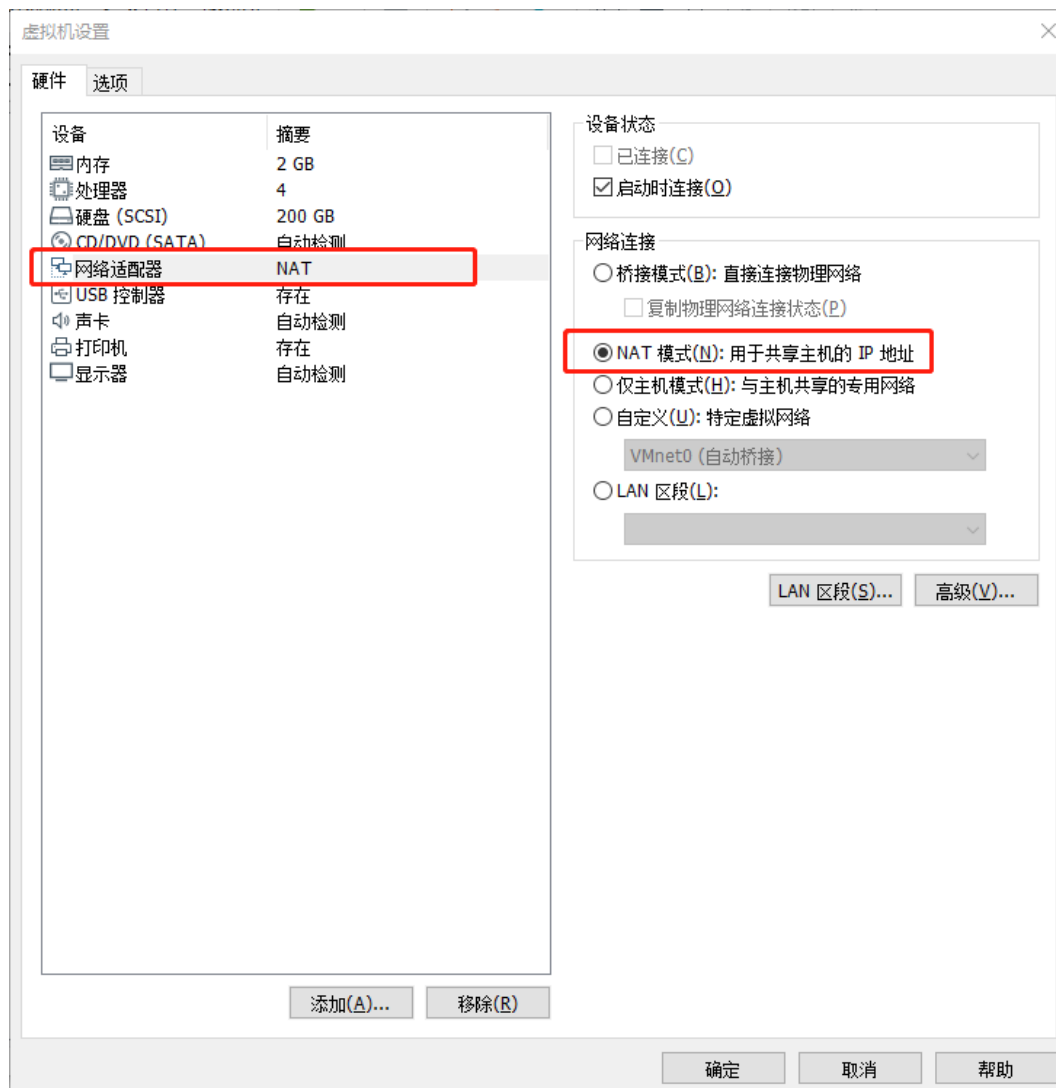
在下图位置进行大部分的系统设置。ubuntu 上很多设置的需求都可以在这里完成。



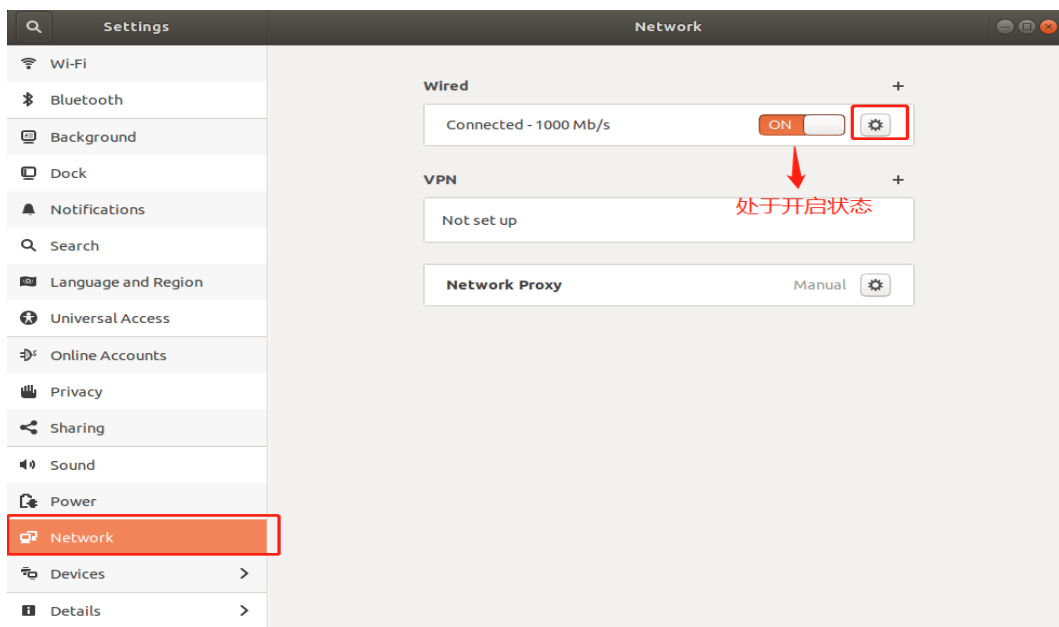
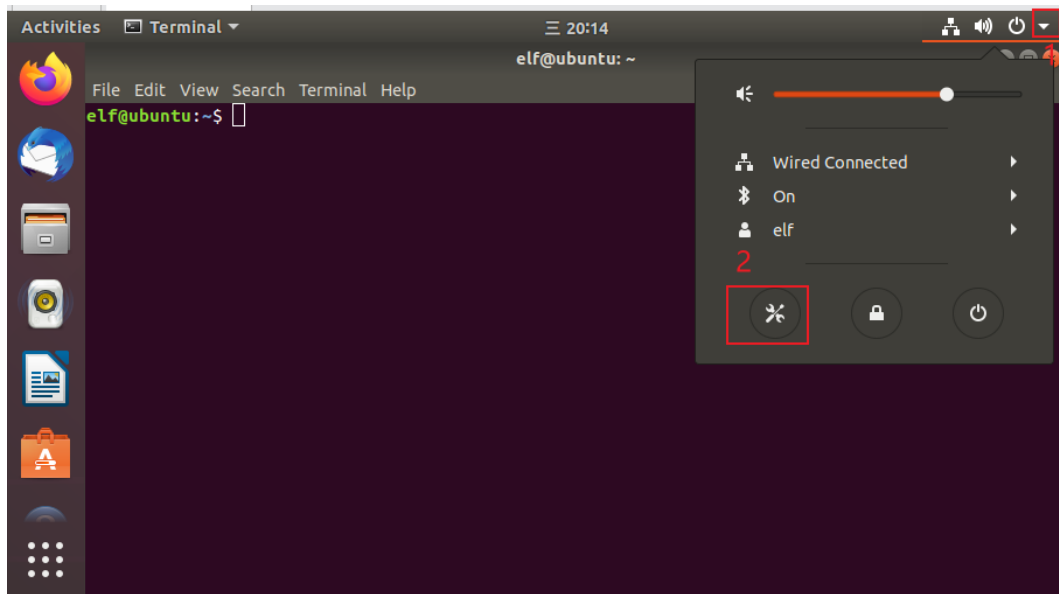
2.4.4 ubuntu 网络设置

1.NAT 模式

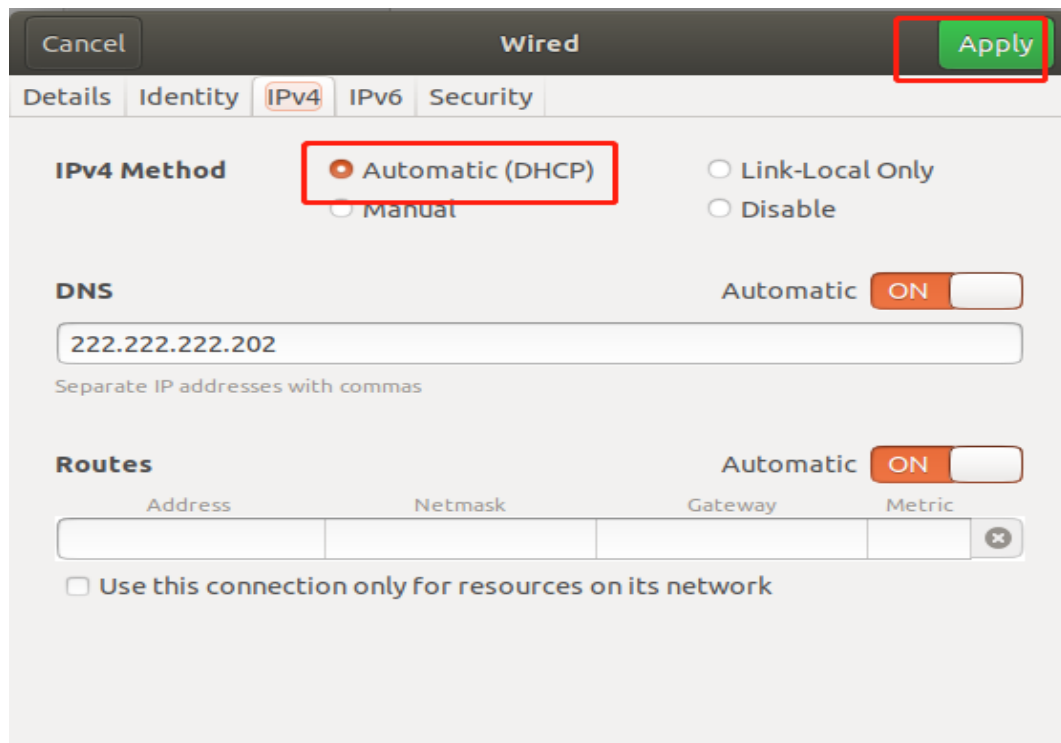
在使用网络前，先确保我们的虚拟机能连接互联网，打开虚拟机设置，网络适配器中的网络连接模式改为“NAT 模式”：



在虚拟机中，VMware 虚拟网卡设置为 NAT 模式时，ubuntu 环境中网络设置为动态 IP 即可。在这种模式下虚拟 NAT 设备和主机网卡相连通。这是我们虚拟机上外网最常用的方式。

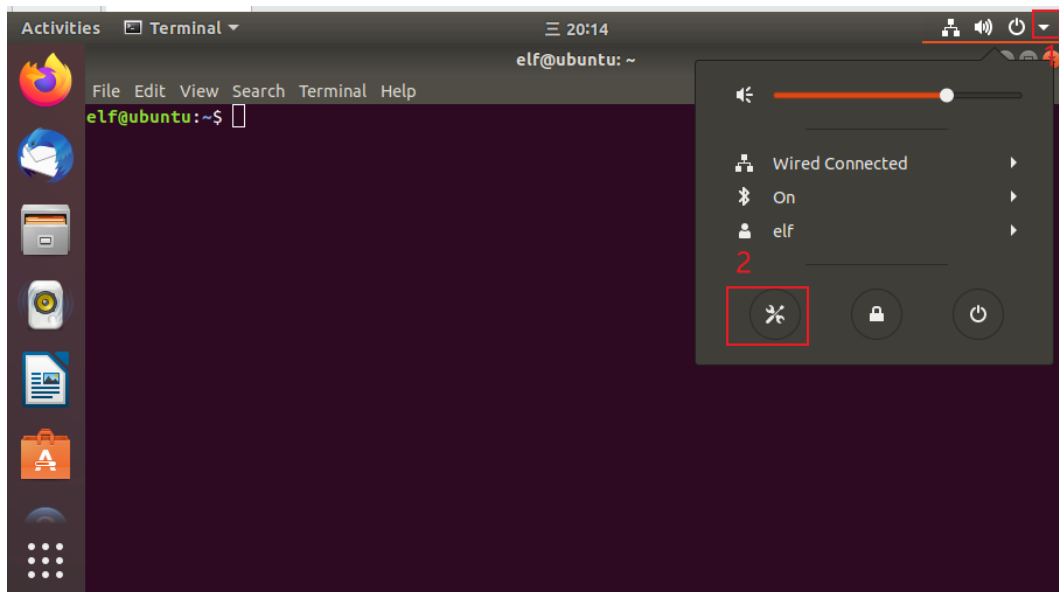
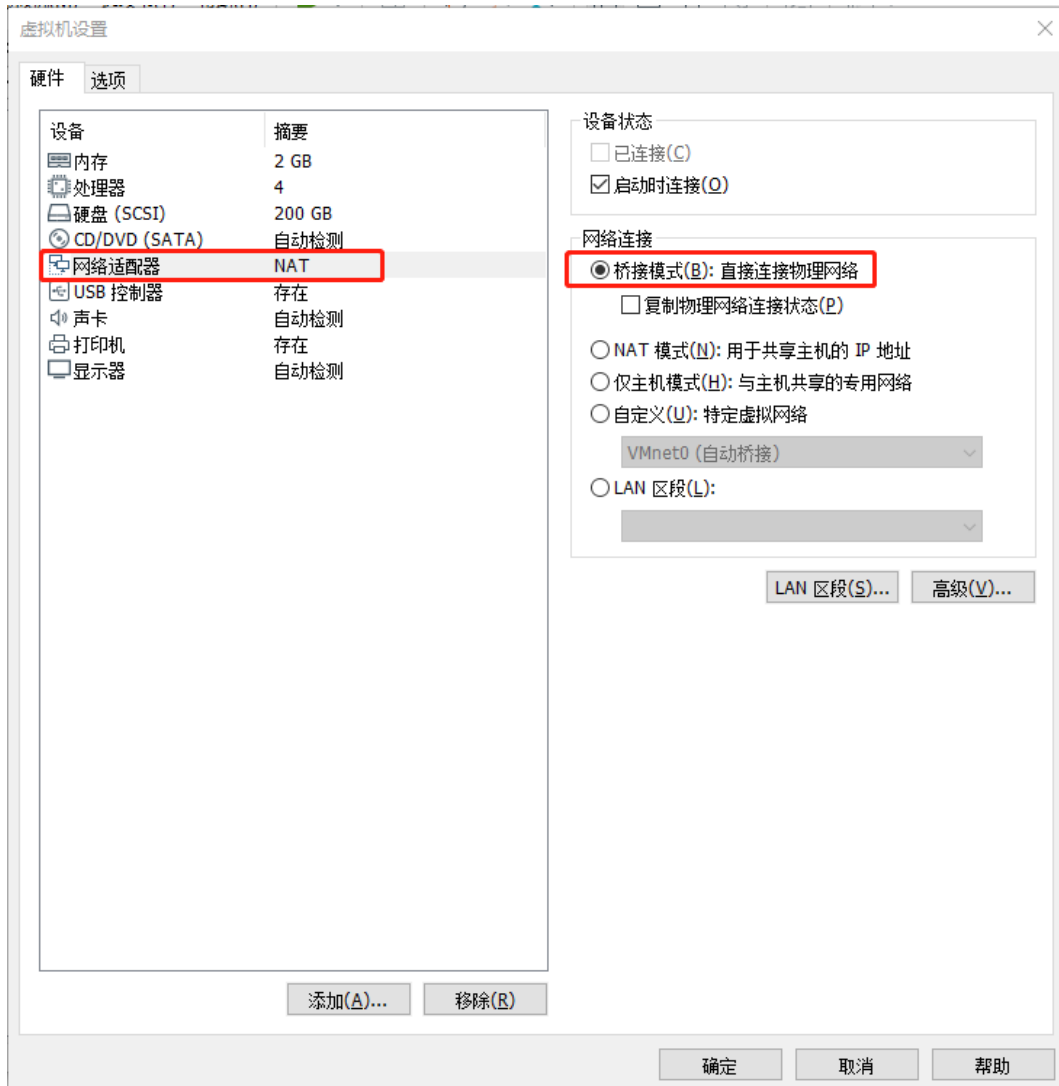


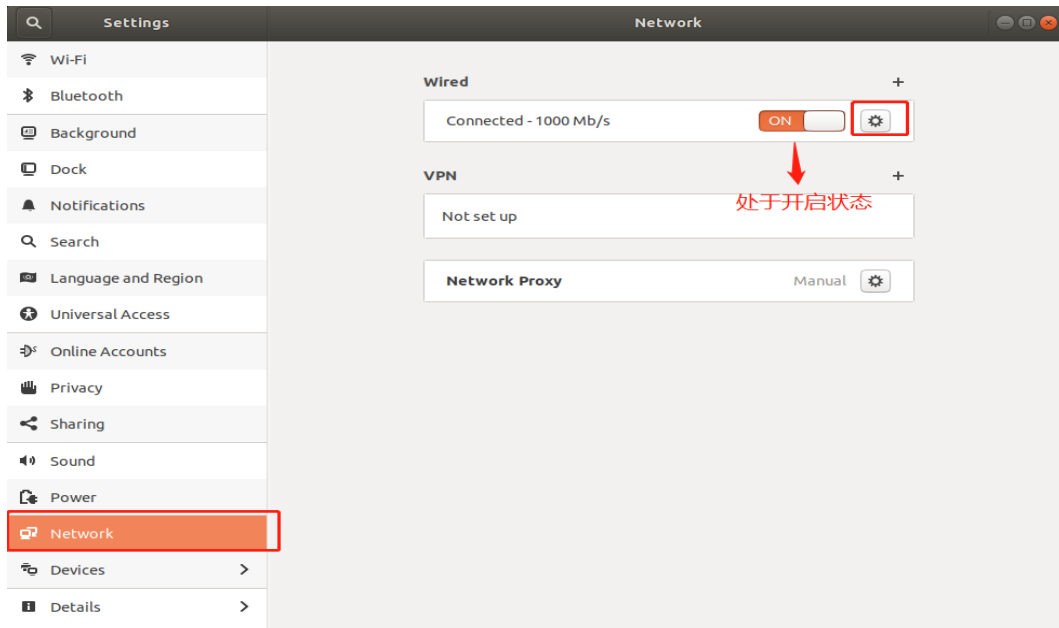
网络设置为动态 ip。



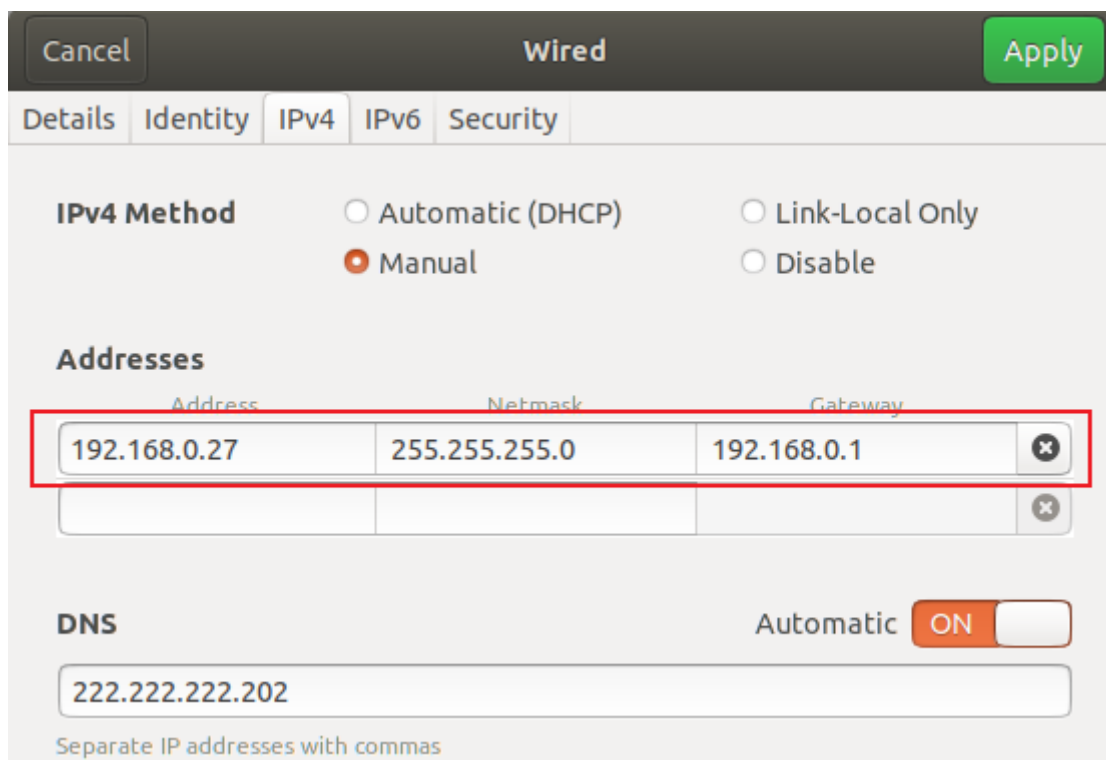
2.桥接模式

如果在使用 TFTP, SFTP 等服务器时则需要设置虚拟机的网络连接方式为桥接方式。VMware 虚拟网卡设置为桥接模式时，主机网卡和虚拟机网卡通过虚拟网桥进行通信，需要将 ubuntu 的 IP 与主机 IP 设置在同一个网段。





设置静态 ip，此时 ubuntu 的 IP 与主机 IP 需设置在同一个网段。



⚠ 注意：网络设置部分涉及到的 IP 以及 DNS 请按照用户自身的实际环境来设置，文档为举例说明。

2.4.5 必要库安装

在进行开发之前，还需要一些其他的必要库，我们使用以下命令逐一安装，安装前需保证网络可正常使用，能上外网：

```
elf@ubuntu:~$ sudo apt-get update //更新下载源信息
elf@ubuntu:~$ sudo apt-get install build-essential //提供编译程序必须软件包列表信息
```

```

息
elf@ubuntu:~$ sudo apt-get install libncurses* //用于生成基于文本的用户界面
elf@ubuntu:~$ sudo apt-get install lzop //基于 Lzo 库的压缩解压工具
elf@ubuntu:~$ sudo apt-get install net-tools //网络配置工具
  
```

2.4.6 ubuntu 系统的主要目录

目录	说明
/	根目录，所有的目录都是由根目录衍生出来
/bin	系统的基本指令，可以被 root 和一般用户使用
/home	用户主目录，默认用户的主文件夹都会放到这里
/dev	设备文件目录，设备与接口设备都是以文件形式存在在该目录
/etc	系统的主要配置文件目录
/lib	系统函数库目录
/lib/module	内核相关的模块驱动
/root	系统管理员 root 的主文件夹
/usr	Unix Software Resource 的缩写，系统的默认软件会放到该目录下
/usr/bin	一些用户使用命令，非系统必须的命令
/usr/lib	应用软件所需的库文件
/sbin	重要的系统执行文件
/mnt	用户临时挂载目录
/opt	第三方软件放置目录
/proc	该目录为虚拟文件系统，操作系统运行时，进程信息及内核信息（比如 cpu、硬盘分区、内存信息等）存放在这里，放置的数据都在内存中

2.5 软件包的管理方式

Ubuntu 系统提供了强大的软件包管理工具，用于安装、更新、卸载和管理软件包。这些工具主要通过 APT（Advanced Package Tool）和 dpkg 来实现。本节将详细介绍 Ubuntu 的软件包管理方式。

2.5.1 APT 工具

APT 是 Ubuntu 中最常用的软件包管理工具，它能够自动处理依赖关系并从软件源中下载和安装软件包。以下是 APT 的常用命令。

1.更新软件包列表

在安装或更新软件包之前，建议先更新软件包列表，以获取最新的软件信息：

```
elf@ubuntu:~$ sudo apt update
```

2.升级已安装的软件包

升级所有已安装的软件包到最新版本：

```
elf@ubuntu:~$ sudo apt upgrade
```

3.安装软件包

安装指定的软件包（以 vim 为例）：

```
elf@ubuntu:~$ sudo apt install vim
```

4.卸载软件包

卸载软件包但保留配置文件：

```
elf@ubuntu:~$ sudo apt remove vim
```

完全卸载软件包（包括配置文件）：


```
elf@ubuntu:~$ sudo apt purge vim
```

5. 搜索软件包

搜索包含关键字的软件包:

```
elf@ubuntu:~$ apt search keyword
```

6. 查看软件包信息

查看软件包的详细信息:

```
elf@ubuntu:~$ apt show package_name
```

7. 清理无用软件包

清理系统中已卸载软件包的残留文件:

```
elf@ubuntu:~$ sudo apt autoremove
```

2.5.2 dpkg 工具

dpkg 是 Ubuntu 底层的软件包管理工具,用于直接操作.deb 格式的软件包。以下是 dpkg 的常用命令。

1. 安装本地.deb 软件包

安装下载到本地的.deb 软件包:

```
elf@ubuntu:~$ sudo dpkg -i package_file.deb
```

如果安装过程中出现依赖问题,可以运行以下命令修复:

```
elf@ubuntu:~$ sudo apt --fix-broken install
```

2. 卸载软件包

卸载软件包但保留配置文件:

```
elf@ubuntu:~$ sudo dpkg -r package_name
```

完全卸载软件包(包括配置文件):

```
elf@ubuntu:~$ sudo dpkg -P package_name
```

3. 列出已安装的软件包

列出系统中所有已安装的软件包:

```
elf@ubuntu:~$ dpkg -l
```

4. 查看软件包内容

查看软件包安装的文件列表:

```
elf@ubuntu:~$ dpkg -L package_name
```

第三章 shell 基本介绍及常用命令

📖说明：本章内容为通用知识，关于 Shell 常用命令的讲解将在第二章搭建的环境中进行演示，适用于 ElfBoard 提供的所有环境。

3.1 Shell 是什么？

我们在刚开始接触 Linux 的时候，经常会听到工程师提到 Shell 这个词，刚开始不知道这是个干什么的，简单的说，它是一个应用，接收用户命令，调用相应的内核接口函数或应用程序，并输出执行结果呈现给用户；同时 Shell 也是一种解释性的编程语言或者脚本语言，开发容易可以在短时间内完成功能强大又好用的脚本，可以在不同的系统上移植非常方便。

3.2 Shell 的特性

不同的发行版本使用的 Shell 有可能不同，但是基本的特性还是一样的：

- 1.支持内置命令可直接使用；
- 2.支持命令组合成新的复合命令；
- 3.支持 TAB 键补齐；
- 4.支持后台运行；
- 5.支持脚本使用；
- 6.支持通配符（*、?、[]）；
- 7.Ctrl+C 命令终止程序；
- 8.命令及其参数区分大小写；

无论我们使用哪种 Shell，登陆系统后会生成一个 Shell 进程，根据用户不同显示不同的提示符，root 用户的通常提示符是“#”，普通用户的命令提示符是“\$”，在命令提示符后边输入命令即可和系统进行交互操作。

ubuntu 默认的 Shell 是 Bash（Bourne Again Shell）。

Linux 命令有很多，功能比较强大，下节我们简单介绍一些常用的命令。

3.3 常用的 shell 命令

Linux 命令一般由三部分组成：Command [option] [parameter]。

说明：

- 1.[]部分中的内容是可选项；
- 2.命令，选项，参数都以空格分隔；输入命令完毕后，按 enter 键执行；
- 3.option:是可选的，有些命令没有选项，主要是用来控制运行状态和行为的；
- 4.parameter:是命令要操作的文件，路径，数据等；

3.4 目录相关命令

3.4.1 pwd 查看当前目录

命令：pwd

英文全拼：print work directory

功能：打印当前所在的路径

参数：没有其他参数

示例：

```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ pwd  
/home/elf  
elf@ubuntu:~$
```

3.4.2 ls 查看当前目录的内容

命令：ls

英文全拼：list

功能：列出目录内容

参数：ls [选项] 目录

目录缺省时，默认为当前目录；选项如下，选项可以组合使用。

选项	说明
空	列出目录下所有非隐藏文件（包括目录）
-a	列出目录下所有文件，包括隐藏文件
-l	列出目录下所有文件，包括长度，拥有者，权限，时间等信息
-t	依照建立时间顺序列出文件
-F	按类型列出文件，在文件末尾用不同符号区分
	斜线 (/) 表示目录
	星号 (*) 表示可执行文件
	@符号 表示链接文件
-r	将文件以字母反序显示

示例：

```

elf@ubuntu: ~
File Edit View Search Terminal Help

elf@ubuntu:~$ ls
Desktop          nfs_rootfs      Videos
Documents        Pictures        VMwareTools-10.3.21-14772444.tar.gz
Downloads        Public          vmware-tools-distrib
examples.desktop Templates
Music            tftpboot

elf@ubuntu:~$ ls -l
total 55140
drwxr-xr-x 2 elf elf      4096 5月  10 19:46 Desktop
drwxr-xr-x 2 elf elf      4096 5月  10 19:26 Documents
drwxr-xr-x 2 elf elf      4096 5月  10 19:26 Downloads
-rw-r--r-- 1 elf elf     8980 5月  10 19:00 examples.desktop
drwxr-xr-x 2 elf elf      4096 5月  10 19:26 Music
drwxrwxr-x 2 elf elf      4096 5月  11 13:34 nfs_rootfs
drwxr-xr-x 2 elf elf      4096 5月  10 19:26 Pictures
drwxr-xr-x 2 elf elf      4096 5月  10 19:26 Public
drwxr-xr-x 2 elf elf      4096 5月  10 19:26 Templates
drwxrwxrwx 2 elf elf      4096 5月  11 11:22 tftpboot
drwxr-xr-x 2 elf elf      4096 5月  10 19:26 Videos
-rw----- 1 elf elf 56405224 10月  2  2019 VMwareTools-10.3.21-14772444.tar.gz
drwxr-xr-x 9 elf elf      4096 10月  2  2019 vmware-tools-distrib

elf@ubuntu:~$ ls -a
.          Downloads      Public
..         examples.desktop .sudo_as_admin_successful
.bash_history .gnupg          Templates
.bash_logout .ICEauthority   tftpboot
.bashrc      .local          Videos
.cache       Music           .viminfo
.config      nfs_rootfs      VMwareTools-10.3.21-14772444.tar.gz
Desktop      Pictures        vmware-tools-distrib
Documents    .profile

elf@ubuntu:~$ ls -r
vmware-tools-distrib  Templates  Music      Desktop
VMwareTools-10.3.21-14772444.tar.gz  Public    examples.desktop
Videos               Pictures   Downloads
tftpboot             nfs_rootfs Documents
elf@ubuntu:~$
  
```

3.4.3 cd 切换目录路径

命令: cd

英文全拼: change directory

功能: 改变路径, 切换路径

参数: cd 目标路径

表示方法	说明
/	根目录
.	当前目录
..	上一层目录
~	当前用户主目录
-	上一次工作目录

示例:

```
elf@ubuntu: /
File Edit View Search Terminal Help
elf@ubuntu:~$ pwd
/home/elf
elf@ubuntu:~$ cd .. → 切换到上级目录
elf@ubuntu:/home$ pwd
/home
elf@ubuntu:/home$ cd ~ → 切换到用户主目录
elf@ubuntu:~$ pwd
/home/elf
elf@ubuntu:~$ cd - → 切换到上一次工作目录
/home
elf@ubuntu:/home$ pwd
/home
elf@ubuntu:/home$ cd / → 切换到根目录
elf@ubuntu:/$ pwd
/
elf@ubuntu:/$
```

3.4.4 mkdir 创建目录

命令: mkdir

英文全拼: make directory

功能: 创建目录

参数: mkdir [选项] 目录

选项	说明
-m	创建目录并指定访问权限（类似 chmod）
-p	递归创建目录即使上级目录不存在，会按目录层级创建目录

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ mkdir -m 775 dir_test1
elf@ubuntu:~/work$ mkdir -p dir_test2/dir_test3
elf@ubuntu:~/work$
```

3.4.5 rmdir 删除空目录

命令: rmdir

英文全拼: remove directory

功能: 删除空目录或多级空目录，无法删除非空目录

参数:

选项	说明
-p	递归删除目录，当子目录删除后其父目录为空时，也一同被删除

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 10:28 dir_test1
drwxrwxrwx 3 elf elf 4096 5月 12 10:28 dir_test2
elf@ubuntu:~/work$ ls -l dir_test1/ →空目录
total 0
elf@ubuntu:~/work$ ls -l dir_test2/ →非空目录，含有子目录，子目录为空
total 4
drwxrwxr-x 2 elf elf 4096 5月 12 10:28 dir_test3
elf@ubuntu:~/work$ rmdir dir_test1/
elf@ubuntu:~/work$ rmdir dir_test2/
rmdir: failed to remove 'dir_test2/': Directory not empty
elf@ubuntu:~/work$ rmdir -p dir_test2/dir_test3/
elf@ubuntu:~/work$ ls -l
total 0
```

3.4.6 rm 删除命令

命令：rm

英文全拼：remove

功能：删除文件目录，不管目录是否非空。

参数：rm [选项] 文件/目录

选项	说明
-f	强制删除文件或目录
-i	删除文件或目录之前需要用户确认
-r	递归删除目录及子目录下的文件
-v	显示删除过程

 **注意：**文件或目录删除后不可恢复。

示例：

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 10:33 dir_test1
drwxrwxr-x 2 elf elf 4096 5月 12 10:34 dir_test2
-rw-rw-r-- 1 elf elf 0 5月 12 10:33 file_test1 → 文件
elf@ubuntu:~/work$ ls -l dir_test1/ → 空目录
total 0
elf@ubuntu:~/work$ ls -l dir_test2/ → 非空目录
total 0
-rw-rw-r-- 1 elf elf 0 5月 12 10:34 file_test2
elf@ubuntu:~/work$ rm file_test1 → 文件删除
elf@ubuntu:~/work$ rm dir_test1/
rm: cannot remove 'dir_test1/': Is a directory
elf@ubuntu:~/work$ rm -rf dir_test1
elf@ubuntu:~/work$ rm dir_test2/
rm: cannot remove 'dir_test2/': Is a directory
elf@ubuntu:~/work$ rm -rf dir_test2/
elf@ubuntu:~/work$ ls -l
total 0
```

3.5 文件操作相关命令

3.5.1 touch 创建新文件

命令: touch

英文全拼:

功能: 创建大小为 0 的一个空文件

参数: touch 文件名

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ touch file_test
elf@ubuntu:~/work$ ls -l
total 0
-rw-rw-r-- 1 elf elf 0 5月 12 11:57 file_test
elf@ubuntu:~/work$
```

3.5.2 echo 创建有内容的文件

命令: echo

功能: 将内容回显到输出设备, echo 命令加上重定向可以创建带内容的文件。

参数: echo 内容或者“内容” > 文件

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ echo "Hello" > file_test1
elf@ubuntu:~/work$ cat file_test1
Hello
elf@ubuntu:~/work$ echo "world" > file_test2
elf@ubuntu:~/work$ cat file_test2
world
elf@ubuntu:~/work$
```

3.5.3 file 查看文件类型

命令: file

功能: 读取文件头并识别文件

参数: file 选项 文件

选项	说明
-b	列出辨识结果时, 不显示文件名称
-c	详细显示指令执行过程, 便于分析执行程序的情形
-f	指定名称文件
-L	直接显示符号连接所指向的文件的类型
-v	显示版本信息

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ file file_test
file_test: empty
elf@ubuntu:~/work$ file file_test1
file_test1: ASCII text
elf@ubuntu:~/work$
```

3.5.4 cp 文件复制命令

命令: cp

英文全拼: copy

功能: 实现文件的复制操作

参数: cp [选项] 源文件/目录 目的文件/目录

选项	说明
-a	复制目录时,保留链接,文件属性并递归复制目录下的内容
-d	复制时保留链接
-f	强制复制,如果目标文件已存在则删除不提示
-i	若存在目标文件需要用户确认操作
-p	将访问权限以及修改时间也复制到新文件中
-r	递归复制目录下的文件和目录
-v	显示文件复制过程

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls
file_test
elf@ubuntu:~/work$ mkdir dir_cp_test1
elf@ubuntu:~/work$ mkdir dir_cp_test2
elf@ubuntu:~/work$ ls
dir_cp_test1  dir_cp_test2  file_test
elf@ubuntu:~/work$ cp file_test dir_cp_test1
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 14:38 dir_cp_test1
drwxrwxr-x 2 elf elf 4096 5月 12 14:38 dir_cp_test2
-rw-rw-r-- 1 elf elf 0 5月 12 14:03 file_test
elf@ubuntu:~/work$ ls -l dir_cp_test1/file_test
-rw-rw-r-- 1 elf elf 0 5月 12 14:38 dir_cp_test1/file_test
elf@ubuntu:~/work$
```




```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 14:38 dir_cp_test1
drwxrwxr-x 2 elf elf 4096 5月 12 14:38 dir_cp_test2
-rw-rw-r-- 1 elf elf 0 5月 12 14:03 file_test
elf@ubuntu:~/work$ ls -l dir_cp_test1/file_test
-rw-rw-r-- 1 elf elf 0 5月 12 14:40 dir_cp_test1/file_test
elf@ubuntu:~/work$ cp -i file_test dir_cp_test1/
cp: overwrite 'dir_cp_test1/file_test'?
elf@ubuntu:~/work$ ls -l dir_cp_test1/file_test
-rw-rw-r-- 1 elf elf 0 5月 12 14:40 dir_cp_test1/file_test
elf@ubuntu:~/work$
elf@ubuntu:~/work$ cp -i file_test dir_cp_test1/
cp: overwrite 'dir_cp_test1/file_test'? y
elf@ubuntu:~/work$ ls -l dir_cp_test1/file_test
-rw-rw-r-- 1 elf elf 0 5月 12 14:42 dir_cp_test1/file_test
elf@ubuntu:~/work$
```

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 14:38 dir_cp_test1
drwxrwxr-x 2 elf elf 4096 5月 12 14:38 dir_cp_test2
-rw-rw-r-- 1 elf elf 0 5月 12 14:03 file_test
elf@ubuntu:~/work$ ls -l dir_cp_test1/
total 0
-rw-rw-r-- 1 elf elf 0 5月 12 14:42 file_test
elf@ubuntu:~/work$ ls -l dir_cp_test2/
total 0
elf@ubuntu:~/work$ cp -r dir_cp_test1/ dir_cp_test2/
elf@ubuntu:~/work$ ls -l dir_cp_test2/
total 4
drwxrwxr-x 2 elf elf 4096 5月 12 15:04 dir_cp_test1
elf@ubuntu:~/work$ ls -l dir_cp_test2/dir_cp_test1/file_test
-rw-rw-r-- 1 elf elf 0 5月 12 15:04 dir_cp_test2/dir_cp_test1/file_test
elf@ubuntu:~/work$
```

3.5.5 mv 文件改名移动

命令: mv

英文全拼: move

功能: 可以将一个目录移到另一个目录, 或者对文件进行改名

参数: mv 源文件/目录 目的文件/目录

示例:

```

elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls
dir_cp_test1 dir_cp_test2 file_test
elf@ubuntu:~/work$ ls -l dir_cp_test1/
total 0
-rw-rw-r-- 1 elf elf 0 5月 12 14:42 file_test
elf@ubuntu:~/work$ mv file_test file_test1
elf@ubuntu:~/work$ ls
dir_cp_test1 dir_cp_test2 file_test1
elf@ubuntu:~/work$ mv file_test1 dir_cp_test1/
elf@ubuntu:~/work$ ls dir_cp_test1/
file_test file_test1
elf@ubuntu:~/work$
  
```

3.5.6 chmod 权限设置命令

命令：chmod

英文全拼：change mode

功能：设置用户对于文件的权限

参数：chmod [参数] 目录或文件

示例：

```

elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 15:44 dir_cp_test1
drwxrwxr-x 3 elf elf 4096 5月 12 15:04 dir_cp_test2
-rwxrwxrwx 1 elf elf 0 5月 12 15:51 file_test
-rw-rw-r-- 1 elf elf 0 5月 12 16:08 file_test1
-rw-rw-r-- 1 elf elf 0 5月 12 16:08 file_test2
  
```

其中-rw-r--就是权限信息。第一个字符是文件类型，-表示普通文件，b表示块设备，c表示字符设备，d表示目录，l表示链接设备，p表示命名管道，s表示Socket文件。

剩下的9个字符-rw-r--分成三组，-表示无权限，其他字符含义如下：

项目	Owner			Group			Others		
权限	读	写	执行	读	写	执行	读	写	执行
字符	r	w	x	r	w	x	r	w	x
数字	4	2	1	4	2	1	4	2	1

文件权限设备时可以通过字符或数字来实现。

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 15:44 dir_cp_test1
drwxrwxr-x 3 elf elf 4096 5月 12 15:04 dir_cp_test2
-rwxrwxrwx 1 elf elf 0 5月 12 15:51 file_test
-rw-rw-r-- 1 elf elf 0 5月 12 16:08 file_test1
-rw-rw-r-- 1 elf elf 0 5月 12 16:08 file_test2
elf@ubuntu:~/work$ chmod 777 file_test1
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 15:44 dir_cp_test1
drwxrwxr-x 3 elf elf 4096 5月 12 15:04 dir_cp_test2
-rwxrwxrwx 1 elf elf 0 5月 12 15:51 file_test
-rwxrwxrwx 1 elf elf 0 5月 12 16:08 file_test1
-rw-rw-r-- 1 elf elf 0 5月 12 16:08 file_test2
elf@ubuntu:~/work$ chmod a+x file_test2
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 15:44 dir_cp_test1
drwxrwxr-x 3 elf elf 4096 5月 12 15:04 dir_cp_test2
-rwxrwxrwx 1 elf elf 0 5月 12 15:51 file_test
-rwxrwxrwx 1 elf elf 0 5月 12 16:08 file_test1
-rwxrwxr-x 1 elf elf 0 5月 12 16:08 file_test2
elf@ubuntu:~/work$
```

3.5.7 chown 文件属主设置命令

命令: chown

英文全拼: change ownerp

功能: 设置文件所有者和文件关联组的命令, 只有超级用户能够使用该命令

参数: chown 用户: 组 文件或目录

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 15:44 dir_cp_test1
drwxrwxr-x 3 elf elf 4096 5月 12 15:04 dir_cp_test2
-rwxrwxrwx 1 elf elf 0 5月 12 15:51 file_test
-rwxrwxrwx 1 elf elf 0 5月 12 16:08 file_test1
-rwxrwxr-x 1 elf elf 0 5月 12 16:08 file_test2
elf@ubuntu:~/work$ sudo chown root:root file_test
[sudo] password for elf:
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 15:44 dir_cp_test1
drwxrwxr-x 3 elf elf 4096 5月 12 15:04 dir_cp_test2
-rwxrwxrwx 1 root root 0 5月 12 15:51 file_test
-rwxrwxrwx 1 elf elf 0 5月 12 16:08 file_test1
-rwxrwxr-x 1 elf elf 0 5月 12 16:08 file_test2
elf@ubuntu:~/work$
```

3.5.8 cat 文件查看合并命令

命令: cat

英文全拼: concatenate

功能: 命令用于连接文件并打印到标准输出设备上

参数: cat 文件名

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls
file_test1 file_test2
elf@ubuntu:~/work$ cat file_test1
Hello
elf@ubuntu:~/work$ cat file_test2
world
elf@ubuntu:~/work$ cat file_test1 file_test2 > file_test
elf@ubuntu:~/work$ cat file_test
Hello
world
elf@ubuntu:~/work$
```

3.5.9 tar 文件压缩解压命令

命令: tar

英文全拼: tape archive

功能: 文件打包工具, 可实现文件的压缩和解压

参数: tar [选项] 文件

选项	说明
-c	创建压缩文件和-x 相斥
-t	列出文件的内容
-x	解压存档文件和-c 相斥
-A	合并存档文件
-d	比较压缩文件和源文件
-r	追加文件到存档文件的末尾
-u	更新存档文件
-f	指定存档文件, 与其他选共享同时使用, 必须放在最后
-v	显示详细处理信息
-C	转到指定目录, 一般用于解压文档
-j	调用 bzip2 程序
-z	调用 gzip 程序
-Z	调用 compress 程序
--exclude=PATH	排除指定目录, 用于打包文件
H	

示例:

将文件打包成 tar.bz2 的压缩包

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls
file_test file_test1 file_test2
elf@ubuntu:~/work$ tar cvjf tar_test.tar.bz2 file_test*
file_test
file_test1
file_test2
elf@ubuntu:~/work$ ls
file_test file_test1 file_test2 tar_test.tar.bz2
elf@ubuntu:~/work$
```

将 tar.bz2 的压缩包解压到指定目录

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ mkdir dir_tar_test
elf@ubuntu:~/work$ ls -l
total 20
drwxrwxr-x 2 elf elf 4096 5月 12 13:58 dir_tar_test
-rw-rw-r-- 1 elf elf 12 5月 12 13:35 file_test
-rw-rw-r-- 1 elf elf 6 5月 12 13:31 file_test1
-rw-rw-r-- 1 elf elf 6 5月 12 13:31 file_test2
-rw-rw-r-- 1 elf elf 189 5月 12 13:56 tar_test.tar.bz2
elf@ubuntu:~/work$ tar xvf tar_test.tar.bz2 -C dir_tar_test/
file_test
file_test1
file_test2
elf@ubuntu:~/work$ ls
dir_tar_test file_test file_test1 file_test2 tar_test.tar.bz2
elf@ubuntu:~/work$ ls -l dir_tar_test/
total 12
-rw-rw-r-- 1 elf elf 12 5月 12 13:35 file_test
-rw-rw-r-- 1 elf elf 6 5月 12 13:31 file_test1
-rw-rw-r-- 1 elf elf 6 5月 12 13:31 file_test2
elf@ubuntu:~/work$
```

3.5.10 ln 创建链接命令

命令: ln

英文全拼: link files

功能: 为某一个文件在另外一个位置建立一个同步的链接

参数: ln 选项 源文件 目标文件

选项	说明
-b	删除, 覆盖以前建立的链接
-d	允许超级用户制作目录的硬链接
-f	强制执行
-i	交互模式, 文件存在则提示用户是否覆盖
-n	把符号链接视为一般目录
-s	软连接 (符号链接)
-v	显示详细的处理过程

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls
dir_ln_test  file_test
elf@ubuntu:~/work$ cat file_test
Hello
elf@ubuntu:~/work$ ln -s file_test file_test1
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 16:32 dir_ln_test
-rw-rw-r-- 1 elf elf 6 5月 12 16:19 file_test
lrwxrwxrwx 1 elf elf 9 5月 12 16:32 file_test1 -> file_test
elf@ubuntu:~/work$ cat file_test1
Hello
elf@ubuntu:~/work$
```

跨目录创建软连接时，需要指定源文件的绝对路径

```
elf@ubuntu: ~/work/dir_ln_test
File Edit View Search Terminal Help
elf@ubuntu:~/work$ ls -l
total 8
drwxrwxr-x 2 elf elf 4096 5月 12 16:32 dir_ln_test
-rw-rw-r-- 1 elf elf 6 5月 12 16:19 file_test
lrwxrwxrwx 1 elf elf 9 5月 12 16:32 file_test1 -> file_test
elf@ubuntu:~/work$ ls -l dir_ln_test/
total 0
elf@ubuntu:~/work$ pwd
/home/elf/work
elf@ubuntu:~/work$ ln -s /home/elf/work/file_test dir_ln_test/
elf@ubuntu:~/work$ cd dir_ln_test/
elf@ubuntu:~/work/dir_ln_test$ ls -l
total 0
lrwxrwxrwx 1 elf elf 24 5月 12 16:34 file_test -> /home/elf/work/file_test
elf@ubuntu:~/work/dir_ln_test$ cat file_test
Hello
elf@ubuntu:~/work/dir_ln_test$
```

3.5.11 find 文件查找命令

命令：find

功能：在指定目录下查找文件

参数：find 目录 [选项] 查找条件

选项	说明
-name	文件名称符合
-iname	文件名称符合，忽略大小写

示例：

```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ pwd
/home/elf
elf@ubuntu:~$ ls
Desktop          nfs_rootfs      Videos
Documents        Pictures        VMwareTools-10.3.21-14772444.tar.gz
Downloads        Public          vmware-tools-distrib
examples.desktop Templates        work
Music            elftpboot
elf@ubuntu:~$ find -name "file_test"
./work/dir_ln_test/file_test
./work/file_test
elf@ubuntu:~$
```

3.5.12 whereis 查找文件

命令: whereis

功能: 用于定位可执行文件、源代码文件、帮助文件在文件系统中的位置

参数: whereis [选项] 文件名

选项	说明
-b	只查找二进制文件
-m	只查找帮助文件
-s	只查找原始代码文件
-u	查找不包含指定类型的文件
-f	不显示文件名前的路径名称
-B	只在设定的目录下查找二进制文件
-M	只在设置的目录下查找说明文件

示例:

```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ whereis pwd
pwd: /bin/pwd /usr/include/pwd.h /usr/share/man/man1/pwd.1.gz
elf@ubuntu:~$
```

3.5.13 grep 文本查找命令

命令: grep

功能: 用于查找文件里符合条件的字符串

参数: grep [选项]

选项	说明
-r	递归方式查找符合条件的文件

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ grep "Hello" ./ -r
./file_test:Hello
elf@ubuntu:~/work$ grep "Hello" ./ -nr
./file_test:1:Hello
elf@ubuntu:~/work$
```

3.5.14 uname 列出内核信息

命令: `uname`

英文全拼: `unix name`

功能: 用于显示系统信息

参数: `uname` 选项

选项	说明
-a	显示全部的信息
-m	显示电脑的类型
-n	显示网络上的主机名
-r	显示操作系统的发行编号
-s	显示操作系统名称
-v	显示操作系统的版本

示例:

```
elf@ubuntu: ~/work
File Edit View Search Terminal Help
elf@ubuntu:~/work$ uname
Linux
elf@ubuntu:~/work$ uname -a
Linux ubuntu 5.4.0-84-generic #94~18.04.1-Ubuntu SMP Thu Aug 26 23:17:46 UTC 2021 x86_64 x86_64 x86_64 GNU/Linux
elf@ubuntu:~/work$ uname -m
x86_64
elf@ubuntu:~/work$ uname -r
5.4.0-84-generic
elf@ubuntu:~/work$ uname -s
Linux
elf@ubuntu:~/work$ uname -v
#94~18.04.1-Ubuntu SMP Thu Aug 26 23:17:46 UTC 2021
elf@ubuntu:~/work$
```

3.6 网络相关命令

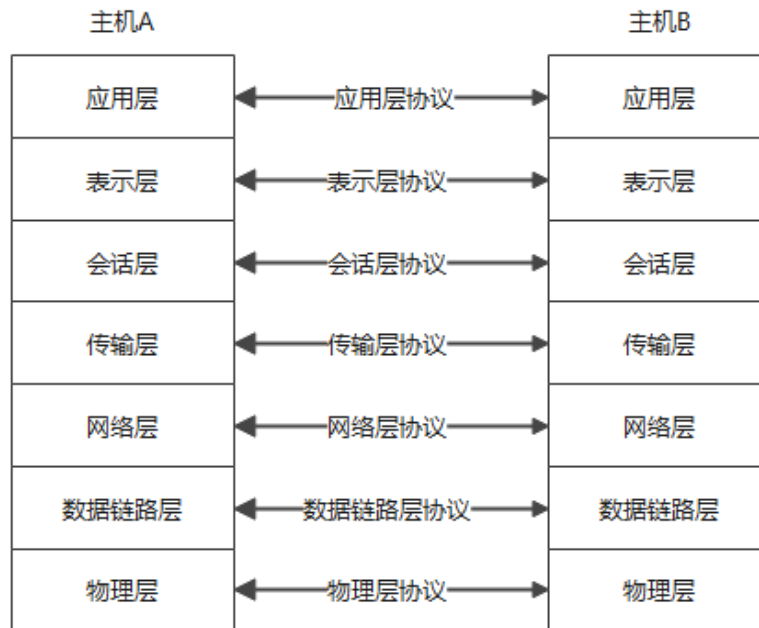
以下测试的网络环境为: 虚拟机 NAT 模式, ubuntu 设置为自动获取 ip。

3.6.1 网络基本知识

具备基本的网络知识对于 linux 的学习非常重要, 只有了解了这些基础知识, 才能更好的学习网络相关指令、网络设置、网络服务等内容。

1.OSI 七层模型

OSI 模型即网络结构的标准模型, 是由国际互联网标准化组织定义的网络分层模型, 虽然由于模型的实现太复杂, 并没有实际的七层网络协议栈, 但对于理解网络协议内部的架构很有帮, 提供了很好的参考模型。



在 OSI 七层结构中，每一层规定了不同的特性，负责不同的功能：

应用层（Application Layer）：用户接口，为应用程序提供访问网络服务的接口，为用户提供常用的应用。应用层使用报文进行数据交互，本层定义的规范有：Telnet、FTP 协议、HTTP 协议、域名 DNS 等。

表示层（Presentation Layer）：对传输的数据进行解释交换，使主机之间通信的信息能互相理解。该层功能主要包括数据的压缩、加密、格式转换等，定义的标准有：ASCII、JPEG 等。

会话层（Session Layer）：对应用会话的建立、中止、会话过程的管理。

传输层（Transport Layer）：该层为主机进程之间的通信提供服务，以数据段为单位，提供可靠或在不可靠的传输，为上层掩盖下层细节，保证会话层的数据信息能够传送到另一方的会话层，是计算机通信体系结构中关键的一层。主要的传输层协议是 TCP 和 UDP。

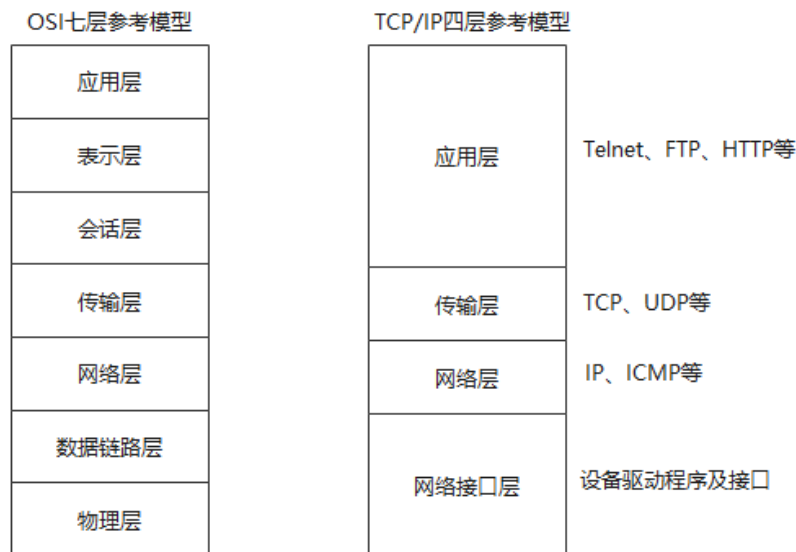
网络层（NetWork Layer）：负责将各个子网之间的数据进行路由选择，将数据从发送端主机发送到接收端主机，数据单位为数据包，主要用到的协议就是 IP 协议。

数据链路层（Data Link Layer）：数据链路层将网络层交下来的 IP 数据包组装成帧，在这一层利用通信信道实现无差错传输，在物理介质基础之上提供了可靠的数据传输。

物理层（Physical Layer）：为上层提供物理介质，通信端点之间的硬件物理接口，设备之间以比特流进行传输。该层主要与硬件相关，确定了与物理接口的一些特性，如电气特性、功能特性等。

2.TCP/IP 四层模型

TCP/IP 传输协议，即传输控制/网络协议，也叫做网络通讯协议。是 linux 世界的网络基础，是在网络的使用中的最基本的通信协议。TCP/IP 协议不是一个独立的协议，而是一组不同层次上的多个协议的组合，不同层次负责不同的功能。



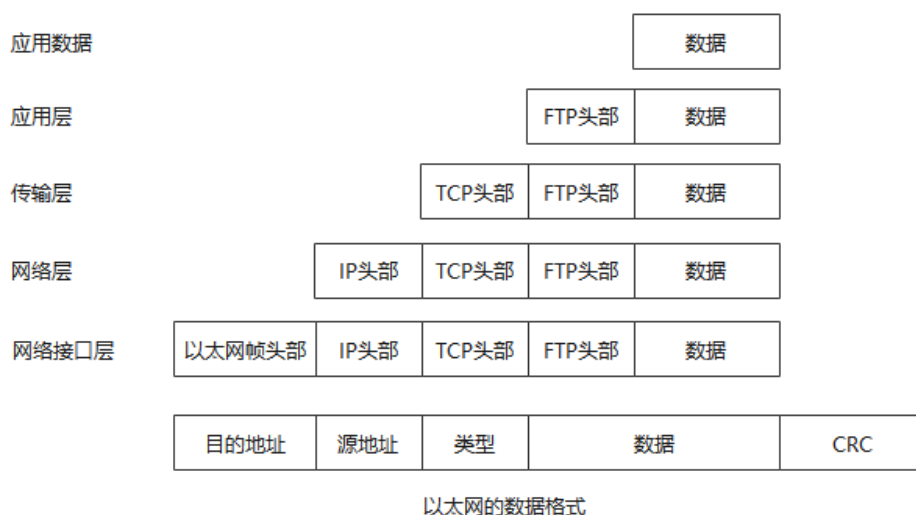
网络接口层：对应的 OSI 参考模型中的物理层和数据链路层。TCP/IP 本身并未定义该层的协议，只是规定了为上一层网络层提供访问接口，可以传输 IP 数据包，发送 ARP 请求和接收 ARP 应答。

网络层：对应 OSI 参考模型中的网络层，是 TCP/IP 协议栈的核心，将数据包进行分组并发往目的主机。主要解决主机到主机的通信问题，它所包含的协议涉及数据包在整个网络上的逻辑传输。该层有三个主要协议：IP（国际协议）、IGMP（互联网组管理协议）、ICMP（互联网控制报文协议）

传输层：对应 OSI 参考模型中的传输层，提供主机与主机之间对等层之间可以进行会话的机制，保证了数据包的顺序传送及数据的完整性。该层定义了两个主要的协议：TCP 传输控制协议、UDP 用户数据报协议

应用层：将 OSI 参考模型中的会话层和表示层取消，将功能合并到应用层，为用户提供所需要的各种服务。例如，基于 TCP 协议的 FTP（文件传输协议）、Telnet 协议、HTTP（超文本链接协议）等，基于 UDP 协议的 TFTP、域名服务 DNS、NFS 等。

以 FTP 文件传输为例，在各层的数据传输过程如下：



3.IP

互联网上的每个接口必须有一个唯一的 Internet 地址（也称作 IP 地址）主机或网络设备如果想要连接

到 Internet，就必须配置 IP 地址。应用范围最广泛的因特网地址使用 IPv4 的 IP 地址，长为 32 位，由 4 组十进制数组成，每组数值范围为 0~255，中间用点号隔开。例如 IP 地址 192.168.0.232。

IP 地址的一般格式为：类别+网络标号+主机标识。其中类别用来区分 IP 地址的类型，网络号表示主机所在的网络，主机号表示主机在网络中的标识。IP 地址通常分为五类：A 类、B 类、C 类、D 类、E 类：



类型	IP 地址范围
A	0.0.0.0 到 127.255.255.255
B	128.0.0.0 到 191.255.255.255
C	192.0.0.0 到 223.255.255.255
D	224.0.0.0 到 239.255.255.255
E	240.0.0.0 到 247.255.255.255

其中 D、E 类为特殊地址不对民用开发。对于 ABC 类地址划分了公有地址和私有地址，其中公有地址由 Inter INC 负责，这些 IP 需要进行注册申请。私有地址属于非注册地址，专门为组织机构内部使用，不能直接访问公网，是不同局域网内的计算机可以使用相同的 IP 地址

类型	私有 IP 地址范围
A	10.0.0.0 到 10.255.255.255
B	172.16.0.0 到 172.31.255.255
C	192.168.0.0 到 192.168.255.255

一些特殊的 IP 地址：

- (一) 主机号全为 0 的 IP 地址，不分配给任何主机，用于表示某个网络的网络地址
- (二) 主机号全为 1 的 IP 地址，不分配给任何主机，仅用作广播地址
- (三) IP 地址 127.0.0.1 为回环结构，用于本地进行软件测试

4.子网掩码

子网掩码是一个 32 位的二进制地址，用来屏蔽原来网络地址的划分情况，从而获得一个范围小的，可以实际使用的网络。子网掩码使用与 IP 地址相同的格式，由 4 组十进制数组成，其中值为 0 的部分对应 IP 地址的主机 ID 部分。

使用子网掩码，网络设备可以分析得出一个 IP 地址的网络地址和子网地址，以及主机地址。子网掩码和 IP 地址进行“与”运算后，所得到的值为网络地址和子网地址。

标准的子网掩码：

类型	子网掩码地址
----	--------

A	255.0.0.0
B	255.255.0.0
C	255.255.255.0

子网掩码便于网络设备的尽快寻址，区分本网段地址和非本网段的地址，划分子网，进一步缩小子网的地址空间，充分利用目前紧缺的 IP 地址。

举例：

比较两台主机的 IP 地址与子网掩码进行“与”运算后的值，如果结果相同，则说明两台计算机处于同一个子网络上。在以太网结构的网络中，同一子网内的两台计算机可以直接通信，而不用路由器对 IP 分组进行转发

主机（IP/Netmask）	A(192.168.0.232/255.255.255.0)	B(192.168.0.51/255.255.255.0)
IP 地址(二进制)	11000000.10101000.00000000.1101000	11000000.10101000.00000000.00110011
子网掩码(二进制)	11111111.11111111.11111111.00000000	11111111.11111111.11111111.00000000
网络地址(二进制)	11000000.10101000.00000000.00000000	11000000.10101000.00000000.00000000

两个主机的 IP 地址和子网掩码进行按位“与”运算，得到两个主机的网络地址均为 11000000.10101000.00000000.00000000，即 192.168.0.0，可知两个主机在同一个网络上。

5.端口号

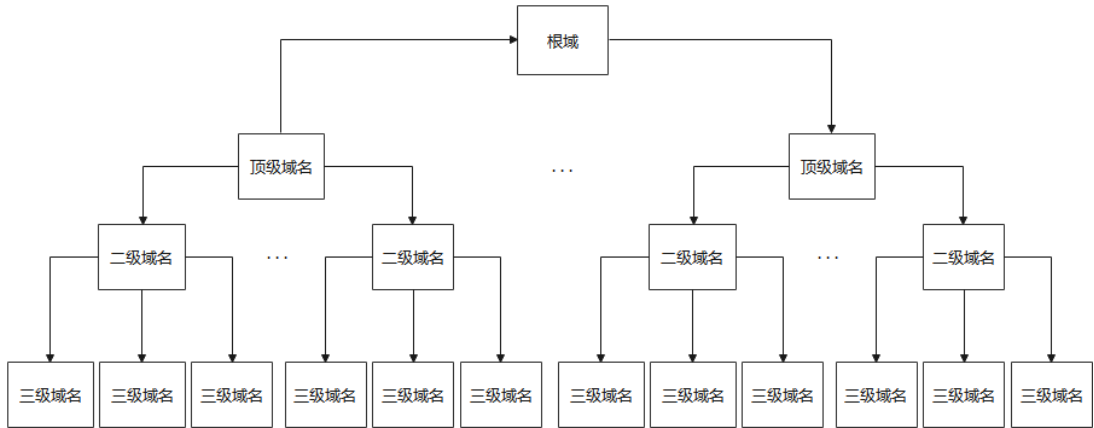
端口是一个 16 位的整数，为了方便多种应用程序，区分不同应用程序的数据和状态，引入了端口。服务程序需要对某个端口进行绑定，客户端访问主机的此端口号与应用程序进行通信。

端口号可以由系统分配，也由用户自定义，一些常用的服务程序使用固定的静态端口号。

6.DNS（域名系统）

DNS 是 Domain Name System 的缩写，即域名系统，是一个用于 TCP/IP 应用程序的分布式数据库，用来提供 IP 地址和主机之间的映射信息。

DNS 的名字空间具有树形的层次结构，结构如下



根域是特殊结点，没有任何结点，树形结构中的域名从最高层域名串联起来，中间用“.”分隔，顶级域名是域名的最后一部分。

顶级域名分为组织域（3 个字符长的普通域）和国家/地区域（2 个字符的域）。

组织域：

顶级域	描述
-----	----

com	商业组织
edu	教育机构
gov	政务机构
int	国际组织
mil	军事机构
net	网络服务机构
org	其他组织

国家/地区域:

顶级域	描述
cn	中国
hk	中国香港
uk	英国
au	澳大利亚
jp	日本

3.6.2 ifconfig 网络配置命令

命令: ifconfig

功能: 显示或设置网络设备。

参数: ifconfig [网络接口] [选项] 地址/参数

选项	说明
ifconfig -a	查看全部网络接口
ifconfig ens33 192.168.0.51	设置网络接口的 IP 地址
ifconfig ens33 broadcast 192.168.2.255	设置网口广播地址
ifconfig ens33 netmask 255.255.255.0	设置网口的子网掩码
ifconfig ens33 hw ether 22:22:22:22:22:2	设置网卡物理地址 (MAC 地址)
ifconfig ens33 up	激活网口
ifconfig ens33 down	关闭网口



注意:

1. 以上命令以 ens33 网口为例, 可以更换为其他网络接口;
2. 操作需要使用 root 用户权限;
3. 该命令设置后不能够复位保存;

```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ ifconfig  
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.0.52 netmask 255.255.255.0 broadcast 192.168.0.255  
    inet6 fe80::9e2d:6f3a:ff4e:b372 prefixlen 64 scopeid 0x20<link>  
    ether 00:0c:29:4f:b3:31 txqueuelen 1000 (Ethernet)  
    RX packets 188176 bytes 106466089 (106.4 MB)  
    RX errors 0 dropped 170 overruns 0 frame 0  
    TX packets 15385 bytes 1203310 (1.2 MB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 1995 bytes 195796 (195.7 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 1995 bytes 195796 (195.7 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
elf@ubuntu:~$
```

```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ ifconfig  
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500  
    inet 192.168.102.129 netmask 255.255.255.0 broadcast 192.168.102.255  
    inet6 fe80::4441:a826:a481:ff26 prefixlen 64 scopeid 0x20<link>  
    ether 00:0c:29:4f:b3:31 txqueuelen 1000 (Ethernet)  
    RX packets 81750 bytes 98149643 (98.1 MB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 14414 bytes 1070287 (1.0 MB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 1026 bytes 118333 (118.3 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 1026 bytes 118333 (118.3 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0
```

关闭 ens33 网口，需要用 `ifconfig -a` 指令看网卡信息。

```
elf@ubuntu:~$ sudo ifconfig ens33 down  
elf@ubuntu:~$ ifconfig  
elf@ubuntu:~$ ifconfig -a
```

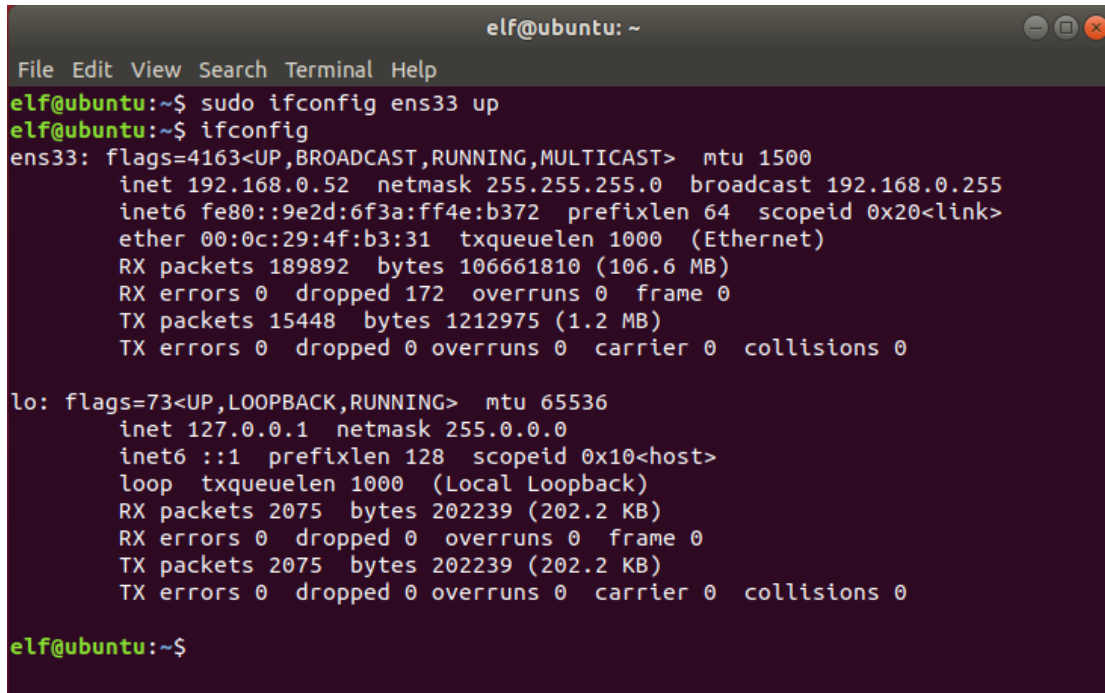


```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ sudo ifconfig ens33 down  
elf@ubuntu:~$ ifconfig  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 2054 bytes 200692 (200.6 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 2054 bytes 200692 (200.6 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
elf@ubuntu:~$ ifconfig -a  
ens33: flags=4098<BROADCAST,MULTICAST> mtu 1500  
    ether 00:0c:29:4f:b3:31 txqueuelen 1000 (Ethernet)  
    RX packets 189754 bytes 106650228 (106.6 MB)  
    RX errors 0 dropped 172 overruns 0 frame 0  
    TX packets 15423 bytes 1206888 (1.2 MB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 2054 bytes 200692 (200.6 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 2054 bytes 200692 (200.6 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
elf@ubuntu:~$
```

```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ sudo ifconfig ens33 down  
[sudo] password for elf:  
elf@ubuntu:~$ ifconfig  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 1034 bytes 118965 (118.9 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 1034 bytes 118965 (118.9 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
elf@ubuntu:~$ ifconfig -a  
ens33: flags=4098<BROADCAST,MULTICAST> mtu 1500  
    ether 00:0c:29:4f:b3:31 txqueuelen 1000 (Ethernet)  
    RX packets 81781 bytes 98151503 (98.1 MB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 14414 bytes 1070287 (1.0 MB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536  
    inet 127.0.0.1 netmask 255.0.0.0  
    inet6 ::1 prefixlen 128 scopeid 0x10<host>  
    loop txqueuelen 1000 (Local Loopback)  
    RX packets 1034 bytes 118965 (118.9 KB)  
    RX errors 0 dropped 0 overruns 0 frame 0  
    TX packets 1034 bytes 118965 (118.9 KB)  
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0  
  
elf@ubuntu:~$
```

打开 ens33 网口。

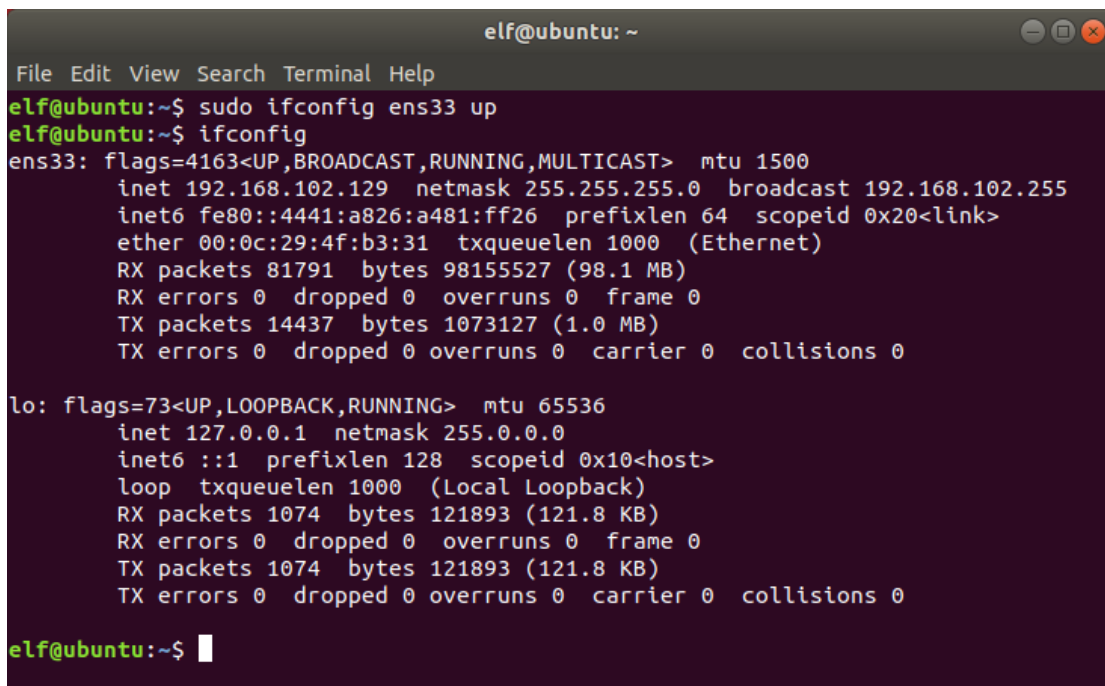
```
elf@ubuntu:~$ sudo ifconfig ens33 up
elf@ubuntu:~$ ifconfig
```



```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ sudo ifconfig ens33 up
elf@ubuntu:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.52 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::9e2d:6f3a:ff4e:b372 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:4f:b3:31 txqueuelen 1000 (Ethernet)
    RX packets 189892 bytes 106661810 (106.6 MB)
    RX errors 0 dropped 172 overruns 0 frame 0
    TX packets 15448 bytes 1212975 (1.2 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2075 bytes 202239 (202.2 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2075 bytes 202239 (202.2 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

elf@ubuntu:~$
```



```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ sudo ifconfig ens33 up
elf@ubuntu:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.102.129 netmask 255.255.255.0 broadcast 192.168.102.255
    inet6 fe80::4441:a826:a481:ff26 prefixlen 64 scopeid 0x20<link>
    ether 00:0c:29:4f:b3:31 txqueuelen 1000 (Ethernet)
    RX packets 81791 bytes 98155527 (98.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14437 bytes 1073127 (1.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1074 bytes 121893 (121.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1074 bytes 121893 (121.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

elf@ubuntu:~$
```

设置网卡信息。

```
elf@ubuntu:~$ sudo ifconfig ens33 192.168.0.51
elf@ubuntu:~$ sudo ifconfig ens33 broadcast 192.168.0.255
elf@ubuntu:~$ sudo ifconfig ens33 netmask 255.255.255.0
elf@ubuntu:~$ sudo ifconfig ens33 hw ether 22:22:22:22:22:22
```



```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ sudo ifconfig ens33 192.168.0.51
elf@ubuntu:~$ sudo ifconfig ens33 broadcast 192.168.0.255
elf@ubuntu:~$ sudo ifconfig ens33 netmask 255.255.255.0
elf@ubuntu:~$ sudo ifconfig ens33 hw ether 22:22:22:22:22:22
elf@ubuntu:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.51 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::4441:a826:a481:ff26 prefixlen 64 scopeid 0x20<link>
    ether 22:22:22:22:22:22 txqueuelen 1000 (Ethernet)
    RX packets 81844 bytes 98176465 (98.1 MB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 14471 bytes 1077562 (1.0 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 1086 bytes 122849 (122.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 1086 bytes 122849 (122.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

elf@ubuntu:~$
```

3.6.3 ping 网络通断检查命令

命令: ping

功能: 检查网络的连接情况, 通常情况下只能 ping 网段的主机, 跨网段的 ping 操作需要进行特殊的路由设置

参数: ping [选项] IP 地址/域名

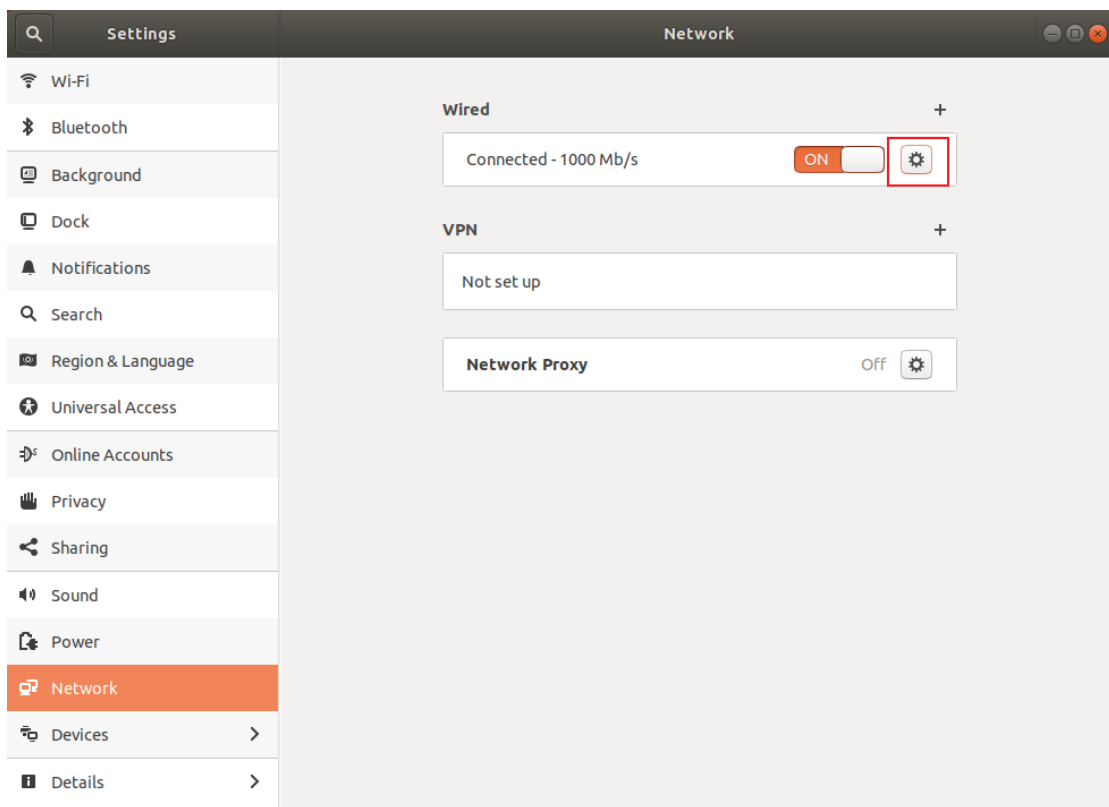
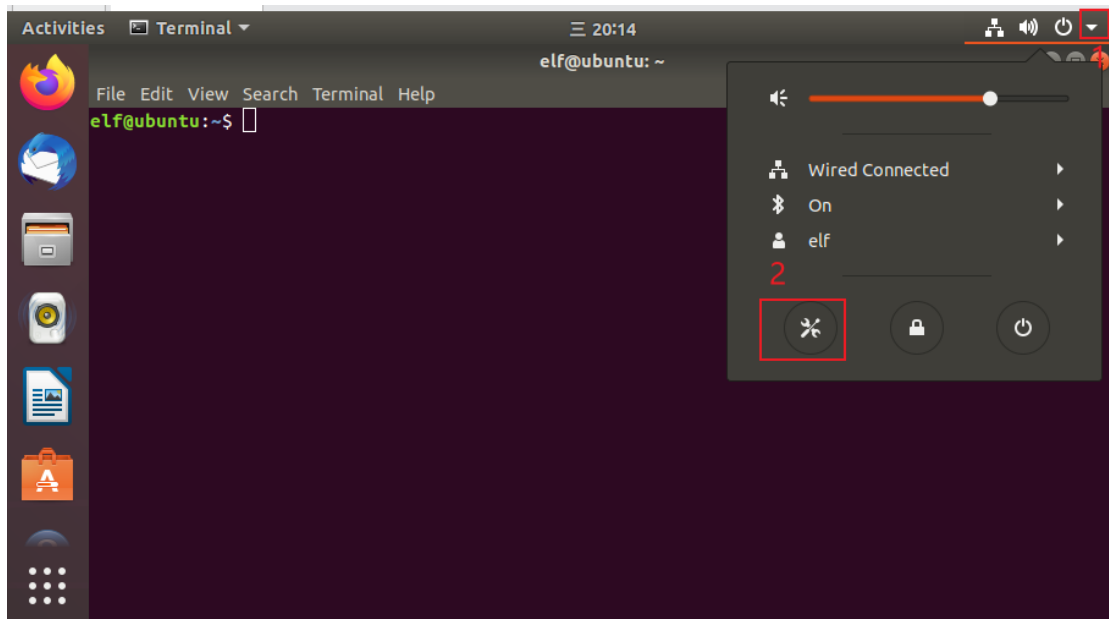
选项	说明
-c	设置完成要求回应的次数
-i	指定收发信息的间隔时间
-n	只输出数值
-r	忽略普通的 Routing Table, 直接将数据包送到远端主机上
-s	设置数据包的大小
-t	设置存活值 TTL 的大小
-v	详细显示指令的执行过程

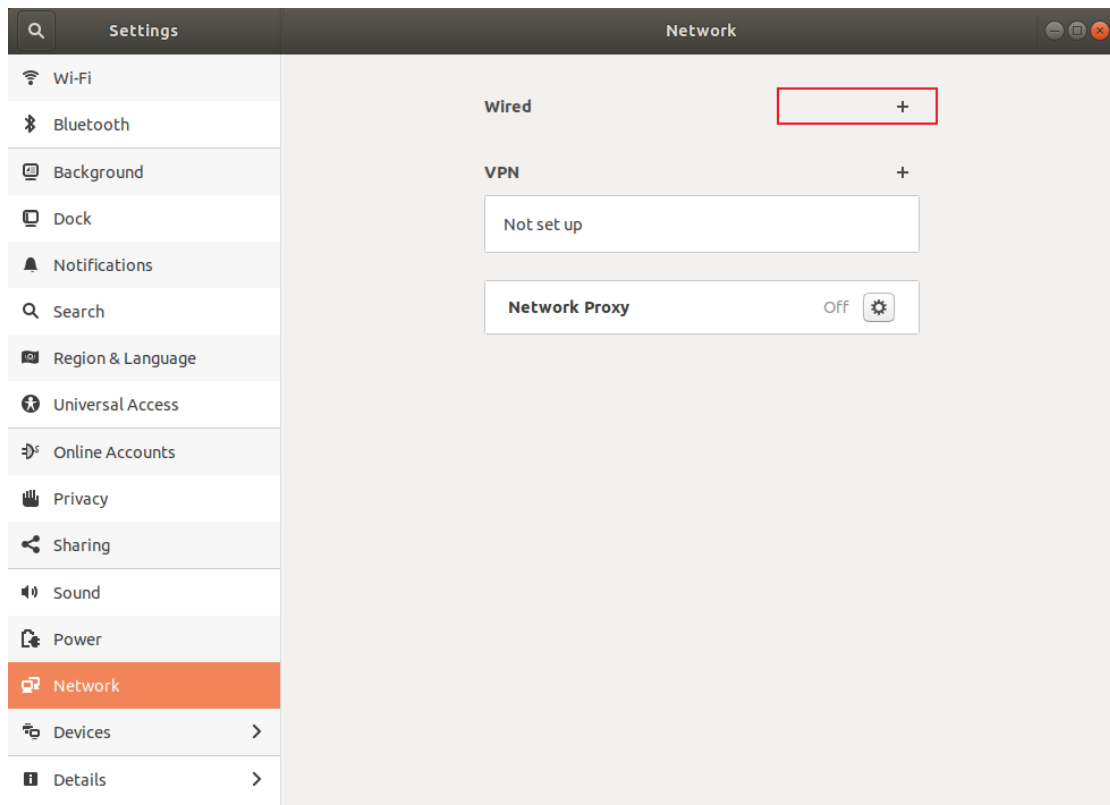
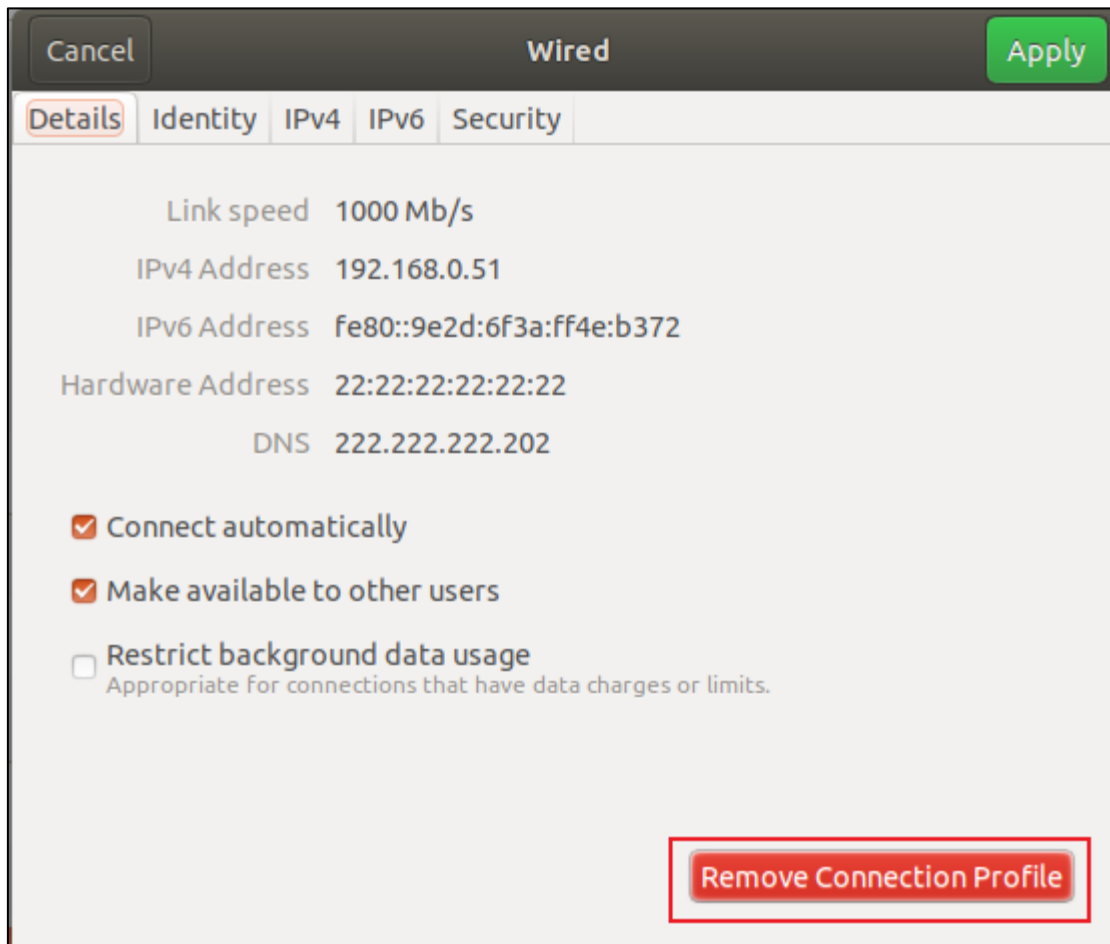
注意:

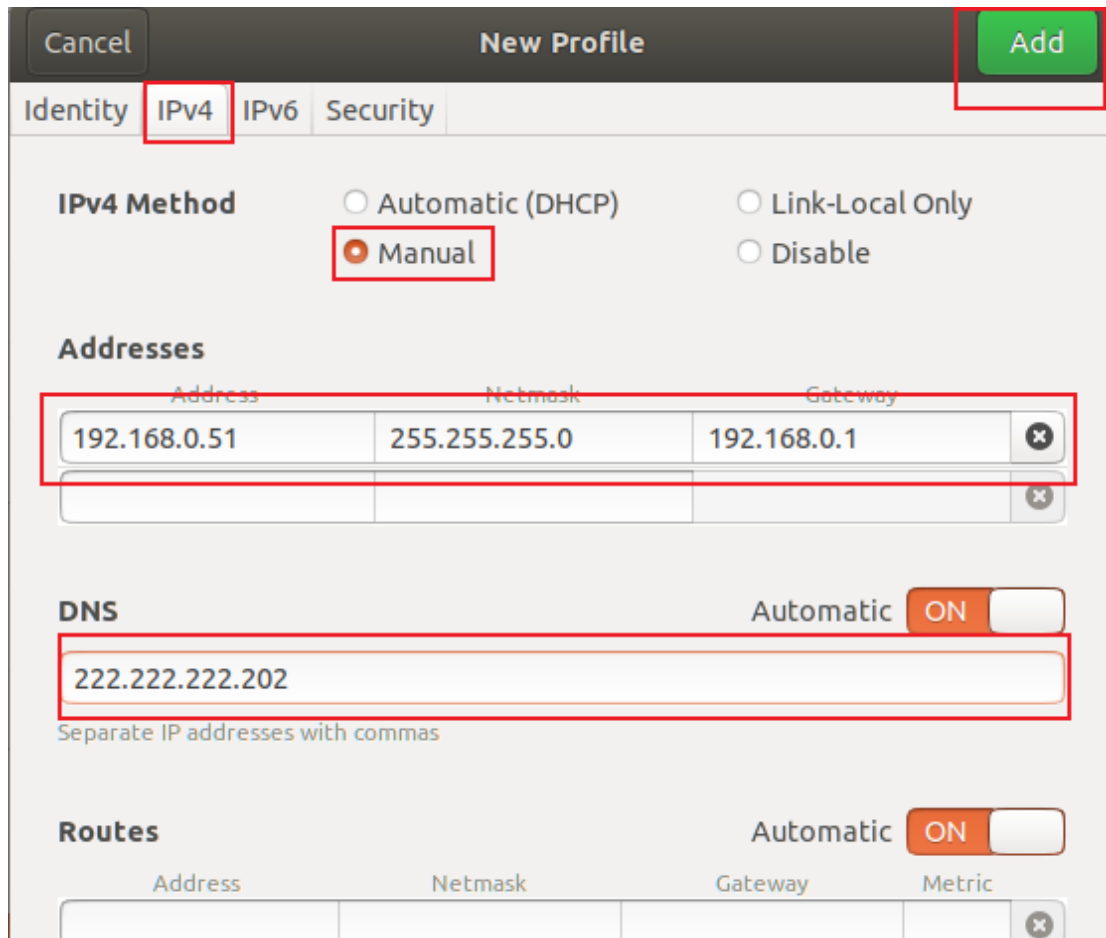
如果是操作完上面小节的网络测试命令, 需要先执行以下命令使网络配置生效。

```
elf@ubuntu:~$ systemctl restart network-manager.service
```

在进行 ping 测试前需要移除现有的网络配置, 重新设置 ip。







The image shows a 'New Profile' configuration window with several sections. The 'IPv4' tab is selected. In the 'IPv4 Method' section, the 'Manual' radio button is selected. The 'Addresses' section contains a table with the following data:

Address	Netmask	Gateway	
192.168.0.51	255.255.255.0	192.168.0.1	✕
			✕

The 'DNS' section has an 'Automatic' toggle set to 'ON' and a text field containing '222.222.222.202'. Below this is the text 'Separate IP addresses with commas'. The 'Routes' section also has an 'Automatic' toggle set to 'ON' and a table with the following headers:

Address	Netmask	Gateway	Metric	
				✕

示例：

与电脑主机 ip（192.168.0.64）进行 ping 测试。

```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ ifconfig
ens33: flags=4163<UP,BROADCAST,RUNNING,MULTICAST> mtu 1500
    inet 192.168.0.51 netmask 255.255.255.0 broadcast 192.168.0.255
    inet6 fe80::1863:84f9:cfde:57d prefixlen 64 scopeid 0x20<link>
    ether 22:22:22:22:22:22 txqueuelen 1000 (Ethernet)
    RX packets 209760 bytes 108237084 (108.2 MB)
    RX errors 0 dropped 209 overruns 0 frame 0
    TX packets 15616 bytes 1242640 (1.2 MB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

lo: flags=73<UP,LOOPBACK,RUNNING> mtu 65536
    inet 127.0.0.1 netmask 255.0.0.0
    inet6 ::1 prefixlen 128 scopeid 0x10<host>
    loop txqueuelen 1000 (Local Loopback)
    RX packets 2201 bytes 212875 (212.8 KB)
    RX errors 0 dropped 0 overruns 0 frame 0
    TX packets 2201 bytes 212875 (212.8 KB)
    TX errors 0 dropped 0 overruns 0 carrier 0 collisions 0

elf@ubuntu:~$ ping 192.168.0.64
PING 192.168.0.64 (192.168.0.64) 56(84) bytes of data.
64 bytes from 192.168.0.64: icmp_seq=1 ttl=128 time=0.309 ms
64 bytes from 192.168.0.64: icmp_seq=2 ttl=128 time=0.174 ms
64 bytes from 192.168.0.64: icmp_seq=3 ttl=128 time=0.175 ms
^C
--- 192.168.0.64 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2014ms
rtt min/avg/max/mdev = 0.174/0.219/0.309/0.064 ms
elf@ubuntu:~$
```

3.6.4 route 路由配置命令

命令: route

功能: 添加, 删除或查看网关参数

参数: route

Destination	目标网段或者主机
Gateway	网关地址, "*" 表示目标是本主机所属的网络, 不需要路由
Genmask	网络掩码
Flags	标记。一些可能的标记如下:
	U - 路由是活动的
	H - 目标是一个主机
	G - 路由指向网关
	R - 恢复动态路由产生的表项
	D - 由路由的后台程序动态地安装
	M - 由路由的后台程序修改
	! - 拒绝路由
Metric	路由距离, 到达指定网络所需的中转数
Ref	路由项引用次数
Use	此路由项被路由软件查找的次数
Iface	该路由表项对应的输出接口

查看路由:

```
elf@ubuntu:~$ route
```

添加路由:

```
elf@ubuntu:~$ sudo route add default gw 192.168.0.1
```

删除路由:

```
elf@ubuntu:~$ sudo route del default gw 192.168.0.1
```

```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ route
Kernel IP routing table
Destination    Gateway         Genmask         Flags Metric Ref    Use Iface
link-local     0.0.0.0         255.255.0.0     U        1000   0      0 ens33
192.168.0.0    0.0.0.0         255.255.255.0   U        100    0      0 ens33
elf@ubuntu:~$ sudo route add default gw 192.168.0.1
elf@ubuntu:~$ route
Kernel IP routing table
Destination    Gateway         Genmask         Flags Metric Ref    Use Iface
default        _gateway        0.0.0.0         UG        0      0      0 ens33
link-local     0.0.0.0         255.255.0.0     U        1000   0      0 ens33
192.168.0.0    0.0.0.0         255.255.255.0   U        100    0      0 ens33
elf@ubuntu:~$ sudo route del default gw 192.168.0.1
elf@ubuntu:~$ route
Kernel IP routing table
Destination    Gateway         Genmask         Flags Metric Ref    Use Iface
link-local     0.0.0.0         255.255.0.0     U        1000   0      0 ens33
192.168.0.0    0.0.0.0         255.255.255.0   U        100    0      0 ens33
elf@ubuntu:~$
```

3.6.5 nameserver 设置 DNS

命令: `nameserver`

功能: 使用域名访问 DNS, 需要设置 DNS

参数: `nameserver xx.xx.xx.xx`

该设置存放的位置在 `/etc/resolv.conf` 文件中:

```
elf@ubuntu: ~
File Edit View Search Terminal Help
# This file is managed by man:systemd-resolved(8). Do not edit.
#
# This is a dynamic resolv.conf file for connecting local clients to the
# internal DNS stub resolver of systemd-resolved. This file lists all
# configured search domains.
#
# Run "systemd-resolve --status" to see details about the uplink DNS servers
# currently in use.
#
# Third party programs must not access this file directly, but only through the
# symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a different way,
# replace this symlink by a static file or a different symlink.
#
# See man:systemd-resolved.service(8) for details about the supported modes of
# operation for /etc/resolv.conf.

nameserver 127.0.0.53
options edns0
~
~
~
```

3.7 其他命令

3.7.1 clear 清屏命令

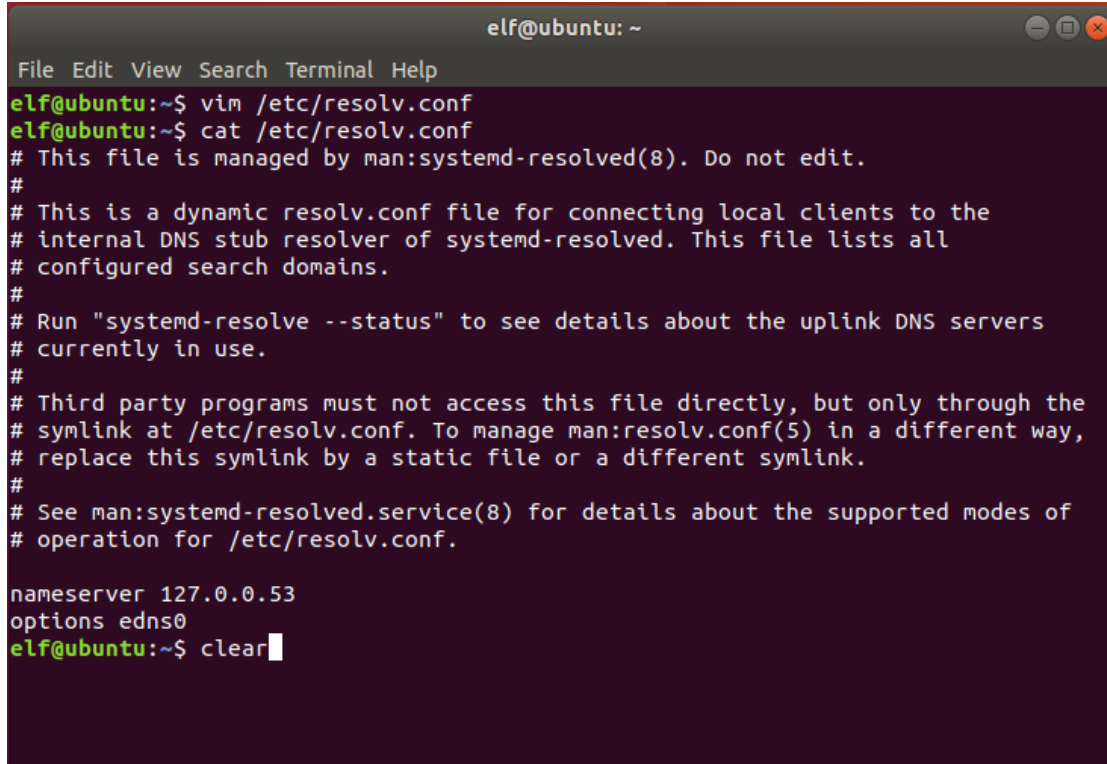
命令：clear

功能：清除屏幕

参数：无

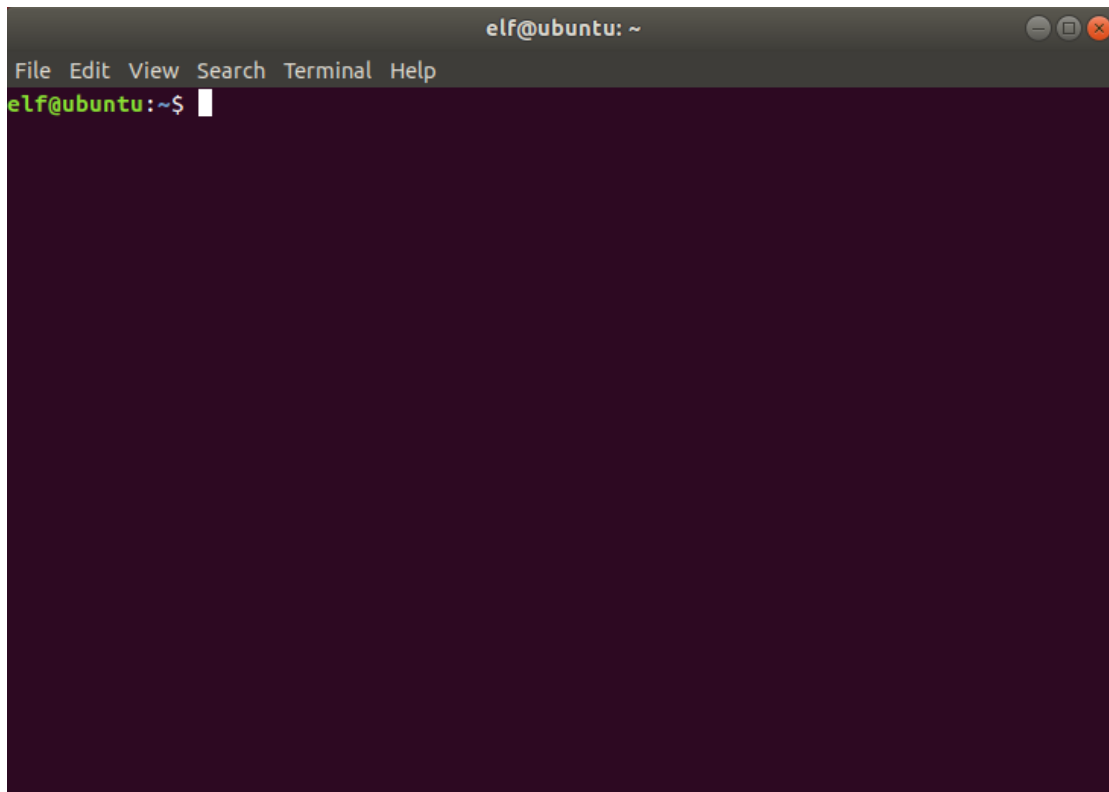
示例：

清屏前

A terminal window titled 'elf@ubuntu: ~' with a menu bar (File, Edit, View, Search, Terminal, Help). The terminal shows the output of 'cat /etc/resolv.conf'. The content includes several comments about the file being managed by systemd-resolved, followed by 'nameserver 127.0.0.53' and 'options edns0'. The prompt 'elf@ubuntu:~\$ clear' is visible at the bottom, with the cursor at the end of the command.

```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ vim /etc/resolv.conf  
elf@ubuntu:~$ cat /etc/resolv.conf  
# This file is managed by man:systemd-resolved(8). Do not edit.  
#  
# This is a dynamic resolv.conf file for connecting local clients to the  
# internal DNS stub resolver of systemd-resolved. This file lists all  
# configured search domains.  
#  
# Run "systemd-resolve --status" to see details about the uplink DNS servers  
# currently in use.  
#  
# Third party programs must not access this file directly, but only through the  
# symlink at /etc/resolv.conf. To manage man:resolv.conf(5) in a different way,  
# replace this symlink by a static file or a different symlink.  
#  
# See man:systemd-resolved.service(8) for details about the supported modes of  
# operation for /etc/resolv.conf.  
  
nameserver 127.0.0.53  
options edns0  
elf@ubuntu:~$ clear
```

执行清屏命令后，终端界面显示信息被清除：



3.7.2 man 帮助命令

命令：man

功能：显示命令的帮助信息

用法：man 具体命令

示例：

```
elf@ubuntu:~$ man ls
```



```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
LS(1) User Commands LS(1)  
  
NAME  
ls - list directory contents  
  
SYNOPSIS  
ls [OPTION]... [FILE]...  
  
DESCRIPTION  
List information about the FILES (the current directory by default).  
Sort entries alphabetically if none of -cftuvSUX nor --sort is speci-  
fied.  
  
Mandatory arguments to long options are mandatory for short options  
too.  
  
-a, --all  
do not ignore entries starting with .  
  
-A, --almost-all  
do not list implied . and ..  
  
--author  
with -l, print the author of each file  
  
-b, --escape  
print C-style escapes for nongraphic characters  
  
--block-size=SIZE  
scale sizes by SIZE before printing them; e.g., '--block-size=M'  
Manual page ls(1) line 1 (press h for help or q to quit)
```

3.7.3 su/sudo 用户切换命令

sudo: 暂时切换到超级用户以执行超级用户权限

su: 切换到某某用户

```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ sudo su  
[sudo] password for elf:  
root@ubuntu:/home/elf# su elf  
elf@ubuntu:~$
```

3.7.4 who

命令: who

功能: 显示关于当前在本地系统上的所有用户的信息

用法: who

示例:

```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ who
elf      :0                2023-05-12 09:29 (:0)
elf@ubuntu:~$
```

3.7.5 PS 查询进程命令

命令: ps

功能: 显示当前操作系统中由该用户运行的进程列表

常见参数:

-u 查看进程所有者及其详细信息

-x 显示没有控制终端的进程

-e 显示所有进程

-l PPID.PID 等详细格式显示: PID (进程 ID) PPID (父进程 ID)

用法:

示例 1: ps

```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ ps
  PID TTY          TIME CMD
 110756 pts/0    00:00:00 bash
 110764 pts/0    00:00:00 ps
elf@ubuntu:~$
```

示例 2: ps -el 查看所有进程的详细信息

```
elf@ubuntu: ~
File Edit View Search Terminal Help
elf@ubuntu:~$ ps -el
 F S      UID        PID  PPID  C PRI  NI ADDR SZ WCHAN  TTY          TIME CMD
 4 S      0           1       0  0  80   0 - 56449 -          ?           00:00:08 systemd
 1 S      0           2       0  0  80   0 -      0 -          ?           00:00:00 kthreadd
 1 I      0           3       2  0  60 -20 -      0 -          ?           00:00:00 rcu_gp
 1 I      0           4       2  0  60 -20 -      0 -          ?           00:00:00 rcu_par_gp
 1 I      0           6       2  0  60 -20 -      0 -          ?           00:00:00 kworker/0:0H
 1 I      0           9       2  0  60 -20 -      0 -          ?           00:00:00 mm_percpu_wq
 1 S      0          10       2  0  80   0 -      0 -          ?           00:00:00 ksoftirqd/0
 1 I      0          11       2  0  80   0 -      0 -          ?           00:00:10 rcu_sched
 1 S      0          12       2  0 -40 -      0 -          ?           00:00:00 migration/0
 5 S      0          13       2  0   9 -      0 -          ?           00:00:00 idle_inject/
 1 S      0          14       2  0  80   0 -      0 -          ?           00:00:00 cpuhp/0
 1 S      0          15       2  0  80   0 -      0 -          ?           00:00:00 cpuhp/1
 5 S      0          16       2  0   9 -      0 -          ?           00:00:00 idle_inject/
 1 S      0          17       2  0 -40 -      0 -          ?           00:00:00 migration/1
 1 S      0          18       2  0  80   0 -      0 -          ?           00:00:00 ksoftirqd/1
 1 I      0          20       2  0  60 -20 -      0 -          ?           00:00:00 kworker/1:0H
 1 S      0          21       2  0  80   0 -      0 -          ?           00:00:00 cpuhp/2
 5 S      0          22       2  0   9 -      0 -          ?           00:00:00 idle_inject/
 1 S      0          23       2  0 -40 -      0 -          ?           00:00:00 migration/2
 1 S      0          24       2  0  80   0 -      0 -          ?           00:00:00 ksoftirqd/2
 1 I      0          26       2  0  60 -20 -      0 -          ?           00:00:00 kworker/2:0H
 1 S      0          27       2  0  80   0 -      0 -          ?           00:00:00 cpuhp/3
```

示例 3：使用 `ps aux` 查看详细进程，区别 `ps -el` 可以更直观看到进程的账户信息等

```
elf@ubuntu: ~
File Edit View Search Terminal Help
root      86735  0.0  0.0    0    0 ?        S    11:15   0:00 [jfsCommit]
root      86736  0.0  0.0    0    0 ?        S    11:15   0:00 [jfsCommit]
root      86737  0.0  0.0    0    0 ?        S    11:15   0:00 [jfsSync]
systemd+  88200  0.0  0.2  70624  4976 ?        Ss   11:15   0:00 /lib/systemd/s
systemd+  88204  0.0  0.1 145964  2488 ?        Ssl  11:15   0:00 /lib/systemd/s
root      88207  0.0  1.0 103092 20256 ?        S<s  11:15   0:00 /lib/systemd/s
root      90715  0.0  0.5 298020 10520 ?        Ssl  11:15   0:01 /usr/lib/polic
root      102400  0.0  0.1  24188  2500 ?        Ss   11:16   0:00 /usr/sbin/xine
root      102687  0.0  0.4 102348  8192 ?        Ss   11:16   0:00 /usr/sbin/cups
root      102688  0.0  0.5 305732 11340 ?        Rsl  11:16   0:00 /usr/sbin/cups
elf       104657  0.0  0.3 199144  6220 ?        Ssl  11:45   0:00 /usr/lib/gvfs/
elf       108487  0.0  0.4 651896  8564 ?        Sl   20:21   0:00 /usr/lib/gvfs/
elf       108580  0.0  0.4 916876  8300 ?        Sl   20:21   0:00 /usr/lib/gvfs/
root      108624  0.0  0.0    0    0 ?        I    20:22   0:00 [kworker/3:1-c
root      108920  0.0  0.0    0    0 ?        I    20:29   0:00 [kworker/0:2-c
root      110088  0.0  0.0    0    0 ?        I    21:13   0:00 [kworker/2:3-r
root      110244  0.0  0.0    0    0 ?        I    21:16   0:00 [kworker/u256:
root      110283  0.0  0.0    0    0 ?        I    21:17   0:00 [kworker/1:2-m
root      110469  0.0  0.0    0    0 ?        I    21:20   0:00 [kworker/3:3-e
root      110551  0.0  0.0    0    0 ?        I    21:22   0:00 [kworker/0:1-e
root      110647  0.0  0.0    0    0 ?        I    21:35   0:00 [kworker/2:1-r
root      110668  0.0  0.0    0    0 ?        R    21:36   0:00 [kworker/u256:
root      110670  0.0  0.0    0    0 ?        I    21:36   0:00 [kworker/1:1-c
elf       110746  0.3  1.8 794632 37308 ?        Rsl  21:37   0:02 /usr/lib/gnome
elf       110756  0.0  0.2  24336  5204 pts/0    Ss   21:37   0:00 bash
root      110809  0.0  0.0    0    0 ?        I    21:47   0:00 [kworker/2:0-e
root      110810  0.0  0.8 482524 16600 ?        Ssl  21:47   0:00 /usr/sbin/Netw
root      110815  0.0  0.0    0    0 ?        I    21:47   0:00 [kworker/3:0-e
root      110855  0.0  0.0    0    0 ?        I    21:47   0:00 [kworker/1:0-c
root      110878  0.0  0.0    0    0 ?        I    21:48   0:00 [kworker/3:2-c
elf       110882  0.0  0.1  41416  3788 pts/0    R+   21:48   0:00 ps aux
elf@ubuntu:~$
```

3.7.6 Kill 强制终止进程命令

命令：kill

功能：强制终止进程

参数：-signal pid

终止进程之前

elf@ubuntu: ~										
File	Edit	View	Search	Terminal	Help					
root	86735	0.0	0.0	0	0 ?	S	11:15	0:00	[jfsCommit]	
root	86736	0.0	0.0	0	0 ?	S	11:15	0:00	[jfsCommit]	
root	86737	0.0	0.0	0	0 ?	S	11:15	0:00	[jfsSync]	
systemd+	88200	0.0	0.2	70624	4976 ?	Ss	11:15	0:00	/lib/systemd/s	
systemd+	88204	0.0	0.1	145964	2488 ?	Ssl	11:15	0:00	/lib/systemd/s	
root	88207	0.0	1.0	103092	20256 ?	S<s	11:15	0:00	/lib/systemd/s	
root	90715	0.0	0.5	298020	10520 ?	Ssl	11:15	0:01	/usr/lib/polic	
root	102400	0.0	0.1	24188	2500 ?	Ss	11:16	0:00	/usr/sbin/xine	
root	102687	0.0	0.4	102348	8192 ?	Ss	11:16	0:00	/usr/sbin/cups	
root	102688	0.0	0.5	305732	11340 ?	Rsl	11:16	0:00	/usr/sbin/cups	
elf	104657	0.0	0.3	199144	6220 ?	Ssl	11:45	0:00	/usr/lib/gvfs/	
elf	108487	0.0	0.4	651896	8564 ?	Sl	20:21	0:00	/usr/lib/gvfs/	
elf	108580	0.0	0.4	916876	8300 ?	Sl	20:21	0:00	/usr/lib/gvfs/	
root	108624	0.0	0.0	0	0 ?	I	20:22	0:00	[kworker/3:1-c	
root	108920	0.0	0.0	0	0 ?	I	20:29	0:00	[kworker/0:2-c	
root	110088	0.0	0.0	0	0 ?	I	21:13	0:00	[kworker/2:3-r	
root	110244	0.0	0.0	0	0 ?	I	21:16	0:00	[kworker/u256:	
root	110283	0.0	0.0	0	0 ?	I	21:17	0:00	[kworker/1:2-m	
root	110469	0.0	0.0	0	0 ?	I	21:20	0:00	[kworker/3:3-e	
root	110551	0.0	0.0	0	0 ?	I	21:22	0:00	[kworker/0:1-e	
root	110647	0.0	0.0	0	0 ?	I	21:35	0:00	[kworker/2:1-r	
root	110668	0.0	0.0	0	0 ?	R	21:36	0:00	[kworker/u256:	
root	110670	0.0	0.0	0	0 ?	I	21:36	0:00	[kworker/1:1-c	
elf	110746	0.3	1.8	794632	37308 ?	Rsl	21:37	0:02	/usr/lib/gnome	
elf	110756	0.0	0.2	24336	5204 pts/0	Ss	21:37	0:00	bash	
root	110809	0.0	0.0	0	0 ?	I	21:47	0:00	[kworker/2:0-e	
root	110810	0.0	0.8	482524	16600 ?	Ssl	21:47	0:00	/usr/sbin/Netw	
root	110815	0.0	0.0	0	0 ?	I	21:47	0:00	[kworker/3:0-e	
root	110855	0.0	0.0	0	0 ?	I	21:47	0:00	[kworker/1:0-c	
root	110878	0.0	0.0	0	0 ?	I	21:48	0:00	[kworker/3:2-c	
elf	110882	0.0	0.1	41416	3788 pts/0	R+	21:48	0:00	ps aux	

终止进 110746 程之后，kill -9 表示无条件杀死该进程，杀死进程后终端窗口会关闭：

```
elf@ubuntu:~$ kill -9 110746
```

3.7.7 top 动态显示命令

命令：top

功能：动态显示当前系统中由该用户运行的进程列表

参数：无

```
elf@ubuntu:~$ top
```



```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
top - 21:40:14 up 10:42, 1 user, load average: 0.01, 0.00, 0.00  
Tasks: 365 total, 1 running, 279 sleeping, 0 stopped, 0 zombie  
%Cpu(s): 0.0 us, 0.1 sy, 0.0 ni, 99.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st  
KiB Mem : 2006384 total, 183584 free, 1220260 used, 602540 buff/cache  
KiB Swap: 2097148 total, 1820964 free, 276184 used. 605704 avail Mem
```

PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
3291	elf	20	0	453504	51616	15388	S	0.5	2.6	0:57.14	Xorg
1	root	20	0	225796	8784	5888	S	0.0	0.4	0:08.76	systemd
2	root	20	0	0	0	0	S	0.0	0.0	0:00.02	kthreadd
3	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_gp
4	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	rcu_par_gp
6	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/0+
9	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	mm_percpu+
10	root	20	0	0	0	0	S	0.0	0.0	0:00.22	ksoftirqd+
11	root	20	0	0	0	0	I	0.0	0.0	0:10.89	rcu_sched
12	root	rt	0	0	0	0	S	0.0	0.0	0:00.26	migration+
13	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inje+
14	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/0
15	root	20	0	0	0	0	S	0.0	0.0	0:00.00	cpuhp/1
16	root	-51	0	0	0	0	S	0.0	0.0	0:00.00	idle_inje+
17	root	rt	0	0	0	0	S	0.0	0.0	0:00.34	migration+
18	root	20	0	0	0	0	S	0.0	0.0	0:00.15	ksoftirqd+
20	root	0	-20	0	0	0	I	0.0	0.0	0:00.00	kworker/1+

第四章 常用编辑器

在 Linux 下我们对文本进行编辑，经常用到的编辑器就是 Vi/Vim，Vi/Vim 命令众多，功能强大，是大多数 Linux 程序员选择使用的编辑器。但是 Vi/Vim 没有图形界面，对于经常在 windows 环境做开发，有 windows 界面软件使用习惯的程序员不怎么友好，所以，也可以使用 gedit 文本编辑器，gedit 类似 windows 的写字板程序，有友好的图形界面。对于不经常在 Linux 做开发的程序员，要进行 Linux 环境下的文本查看和修改，gedit 是一个不错的选择。下面介绍一下，这两种编辑器的安装和使用。

说明：本章内容为通用知识，关于编辑器的讲解将在第二章搭建的环境中进行演示，适用于 ElfBoard 提供的所有环境。

4.1 Vim 编辑器

在 Linux 环境下默认会集成 Vi 编辑器，Vim 是 Vi 编辑器的高阶版本，更容易使用；Vi 的命令几乎全部可以在 Vim 上使用。

在进行安装编辑器之前，环境中的指令集默认没有安装，先执行以下命令更新软件源中的软件列表：

```
elf@ubuntu:~$ sudo apt-get update
```

接下来我们讨论 Vi/Vim 的命令以及选项，不对 Vi/Vim 进行区分。

4.1.1 Vim 的安装

ubuntu 中可以使用 apt-get 命令安装需要的工具：下面是安装 vim 的命令

```
elf@ubuntu:~$ sudo apt-get install vim
```

//安装 vim 编辑器

4.1.2 Vim 的工作模式

Vim 编辑器一般有三种模式：一般模式、命令模式、编辑模式。输入 vim 或者是 vim 文件名，即可启动 Vim 编辑器，默认进入命令模式。

1.一般模式

该模式下可以进行光标的移动、复制、粘贴、删除。处于其他模式时，按下 Escape (Esc) 键返回一般模式

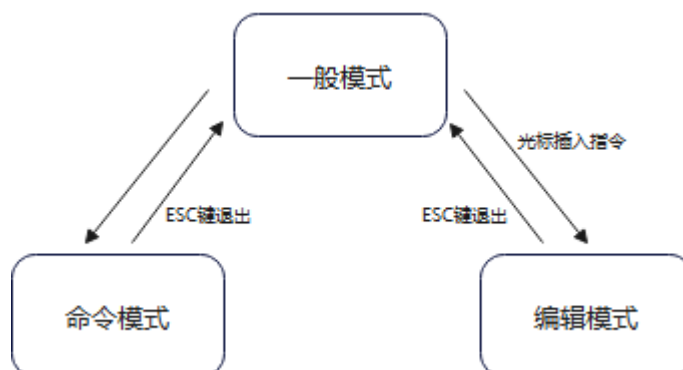
2.命令模式

该模式下以执行保存文件，以及查找、替换等任务。

3.编辑模式

在“一般模式”下按下字母 i 进入编辑模式，该模式下进行文本编辑。

三模式之间可以任意切换。



4.2 Vim 常用操作命令

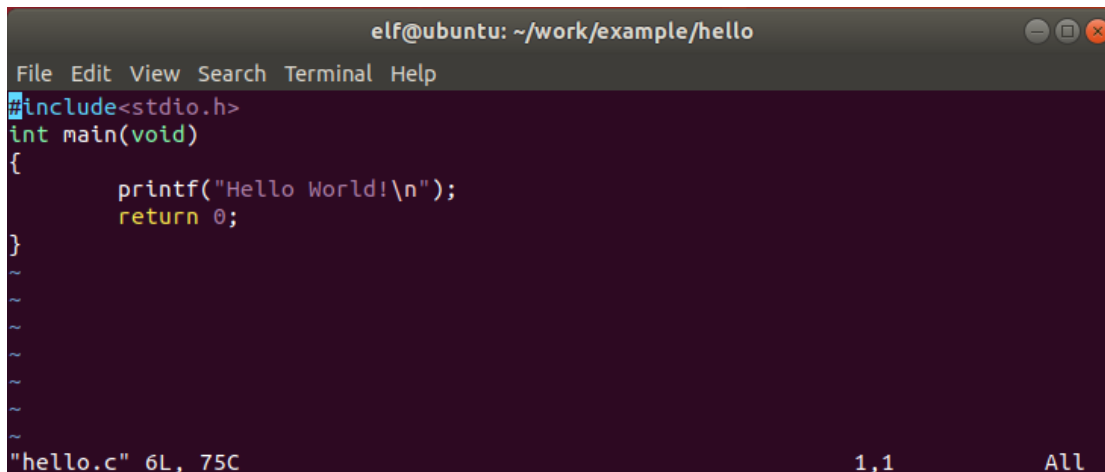
4.2.1 系统终端常用的相关命令

1. 创建打开文件

命令：vim filename

我们可以通过以上命令创建一个新文件或者是打开已有文件进行编辑。已打开 hello.c 文件为例：

```
elf@ubuntu:~/work/example/hello$ vim hello.c
```



The screenshot shows the Vim editor interface with the file 'hello.c' open. The menu bar at the top includes 'File', 'Edit', 'View', 'Search', 'Terminal', and 'Help'. The code content is as follows:

```
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
~
~
~
~
~
~
~
"hello.c" 6L, 75C                               1,1      All
```

如果需要打开文件，并指定到某一行，可以用“vim filename +行号”直接进入文件的改行：

```
elf@ubuntu:~/work/example/hello$ vim hello.c +4
```



The screenshot shows the Vim editor interface with the file 'hello.c' open. The cursor is positioned at the start of the fourth line, which contains the printf statement. A red box highlights the cursor position, and a red text annotation '光标直接定位到第4行' (Cursor directly positioned to the 4th line) is present. The status bar at the bottom right shows '4,2-9'.

```
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
~
~
~
~
~
~
~
"hello.c" 6L, 75C                               4,2-9    All
```

使用 vim 打开文件后，默认为一般模式。

2. 文件对比

Vim 中的 vimdiff 对比工具可以很容易实现文件对比的功能，用法举例如下：我们修改一个 hello.c 文件中的一个字母命名为另一个 hello1.c 文件，我们可以通过以下指令对比出两个文件差异的地方。

```
elf@ubuntu:~/work/example/hello$ vimdiff hello.c hello1.c
```


[illegible]

1.移动光标

Vim 编辑器的编辑过程使用键盘来完成的。在命令模式可以通过 k、j、h、l（小写的 L）来完成光标的移动。

指令	作用
k 或↑	将光标上移一行
j 或↓	将光标下移一行
h 或←	将光标左移一个字母
l 或→	将光标右移一个字母

如果你想通过一个命令上移或下移多行，或者左移、右移多个字母，可以使用数字和字母结合使用。例如向下移动 3 行，可以执行 3j。当前光标位置如下：

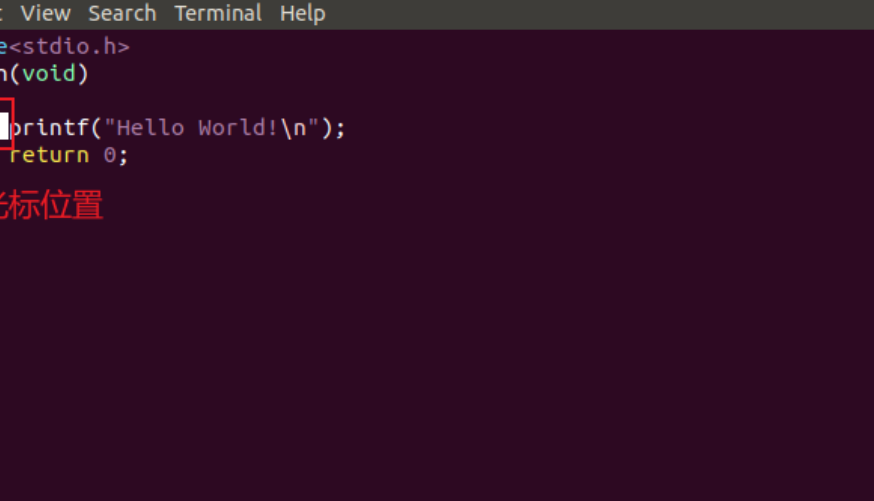


The screenshot shows a terminal window titled "elf@ubuntu: ~/work/example/hello". The menu bar includes File, Edit, View, Search, Terminal, and Help. The code editor displays the following C code:

```
#include<stdio.h>  
int main(void)  
{  
    printf("Hello World!\n");  
    return 0;  
}  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~
```

A red box highlights the first character of the first line, and a red arrow points to it from the right. The status bar at the bottom indicates the file path "hello.c", line 6, column 75, and the search scope "All".

执行 3j 后光标位置:



The screenshot shows a terminal window with the title bar "elf@ubuntu: ~/work/example/hello". The menu bar includes "File", "Edit", "View", "Search", "Terminal", and "Help". The code being edited is:

```
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

A red rectangle highlights the cursor position at the start of the `printf` statement. Below the code, the text "光标位置" (Cursor position) is written in red. The status bar at the bottom shows the file path and line/column information: "hello.c" 6L, 75C on the left, "4,1-8" in the center, and "All" on the right.

2. 屏幕翻页

指令	作用
Ctrl+f 或 Page Down	向下翻页
Ctrl+b 或 Page Up	向上翻页

3.删除文本

指令	作用
dd	删除光标所在的整行内容； dd 前增加数字，比如 2dd 可以删除从光标所在行开始的两行
d\$	删除本行光标所在位置及以后的所有字符
d^	删除本行光标之前的所有字符

dw	删除从光标所在位置直到下一个字符开始的所有内容
----	-------------------------

4.复制与粘贴

Vim 提供了很多简单快捷的复制方法，将光标移动到将要粘贴的地方，按 **p** 就可以完成文本粘贴；在命令行模式下连接 **yy** 即可复制光标所在行的内容，连接 **yny** 即可复制从光标所在行开始的 **n** 行。

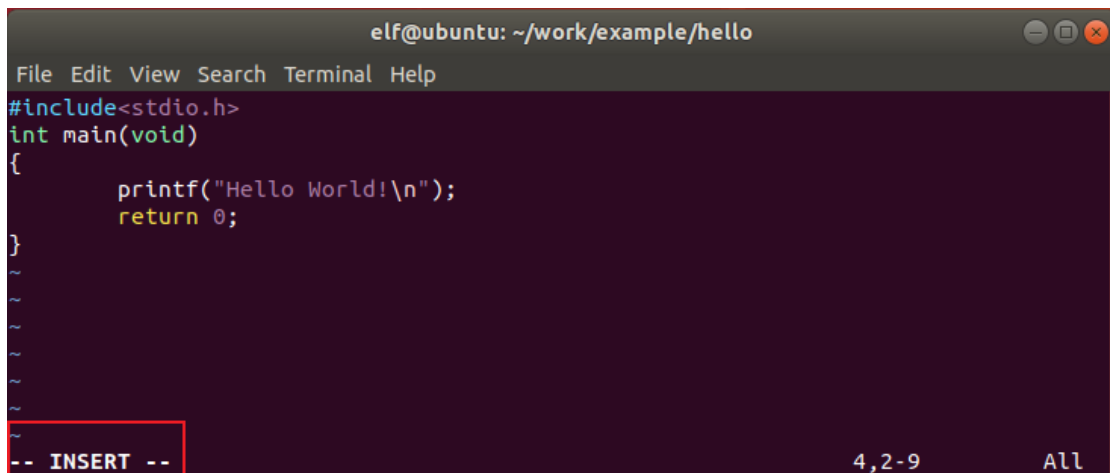
指令	作用
yy	复制当前行，在 yy 前添加数字可以复制多行
p	在光标之后粘贴复制行
P	在光标之前粘贴复制行

5.编辑文件

从“一般模式”进入“编辑模式”，可以使用以下指令：

指令	作用
i	在光标所在行的位置插入内容，进入编辑模式
I	在光标所在行的行首插入内容，进入编辑模式
a	在当前光标之后插入内容，进入编辑模式
A	在光标所在行的行尾插入内容，进入编辑模式
o	在当前光标所在行下方添加一行
O	在当前光标所在行上方添加一行

进入“编辑模式”后，如下图：



```

elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
~
~
~
~
~
-- INSERT -- 4,2-9 All

```

从“编辑模式”按 **ESC** 会再次进入“一般模式”。

6.撤销操作

在“一般模式”下可以使用以下指令：

指令	作用
u	撤销最近的一次编辑操作

4.2.3 Vim 命令模式常用命令

我们首先需要按 **ESC** 键进入一般模式，然后再进行操作。

1.退出 vim

根据不同的需要使用以下命令退出 vim。



指令	作用
:q	未修改退出
:q!	不保存退出
:wq!	保存退出

示例：

未修改退出：

The screenshot shows a terminal window titled "elf@ubuntu: ~/work/example/hello". The menu bar includes File, Edit, View, Search, Terminal, and Help. The main area contains C source code:

```
#include<stdio.h>\nint main(void)\n{\n    printf("Hello World!\\n");\n    return 0;\n}\n~\n~\n~\n~\n~\n~\n~\n~\n~\n~\n~
```

A red box highlights the prompt character ":" at the bottom left.

不保存退出：

```
elf@ubuntu: ~/work/example/hello
```

```
File Edit View Search Terminal Help
```

```
#include<stdio.h>  
int main(void)  
{  
    printf("Hello World!\n");  
    return 0;  
}  
  
~  
~  
~  
~  
~  
~  
~  
~  
~  
~  
:  
:q!
```

保存退出：



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
~
~
~
~
~
~
~
~
~
:wq!
```

2.行号的显示和隐藏

指令	作用
:set nu	显示行号
:set nonu	隐藏行号

显示行号:

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("Hello World!\n");
5     return 0;
6 }
~
~
~
~
~
~
~
~
~
:set nu                                     5,2-9      All
```

隐藏行号:

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
~
~
~
~
~
~
~
~
~
:set nonu                                  5,2-9      All
```

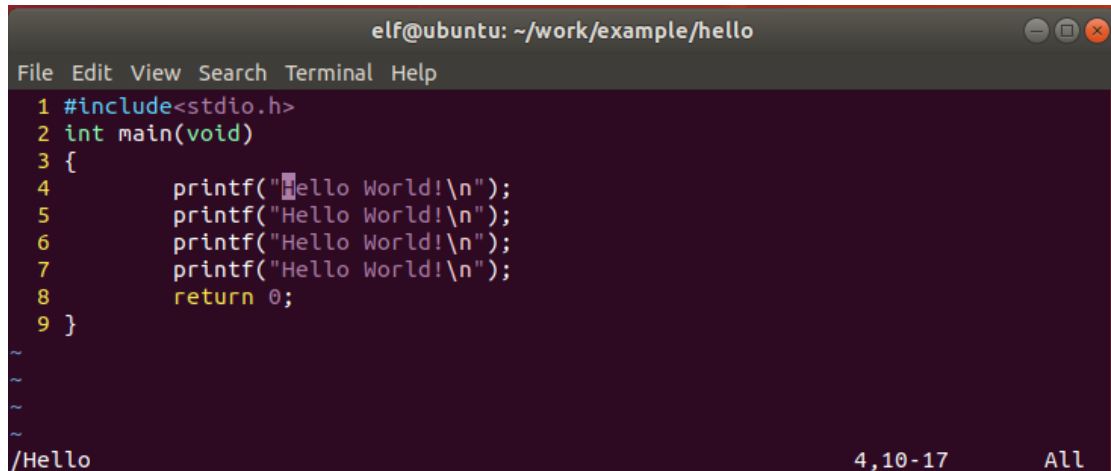
3.查找和替换

指令	作用
/p1	查找 p1
:%s/p1/p2/g	文件中的 p2 替换 p1
:%s/p1/p2/gc	文件中的 p2 替换 p1，并需要确认

在一般模式下，使用“/”字符”可以用于在文本中查找指定字符，搜索过程中，使用 n 向下查找，N 向上查找。

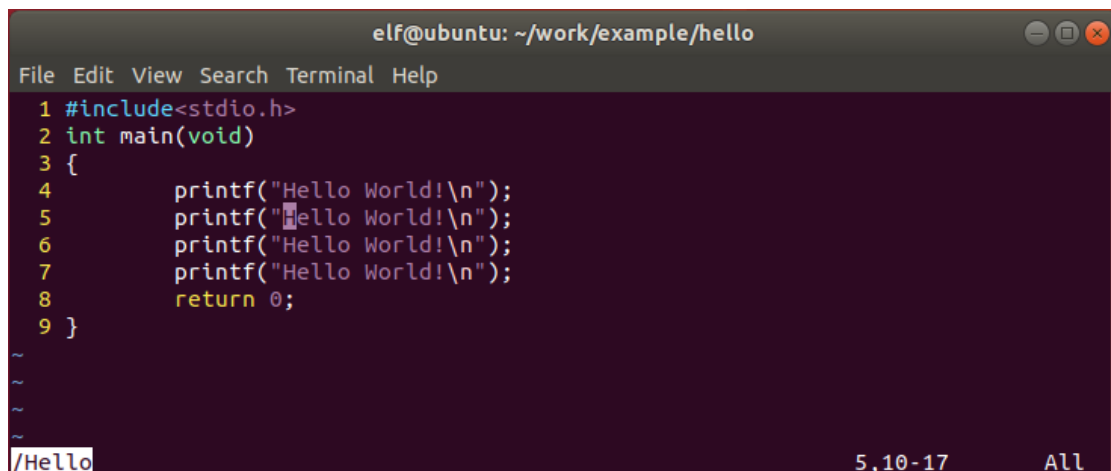
示例：

打开文件，光标在第 1 行，查找 Hello 时，输入/Hello 后按回车，从光标所在位置向下查找到第 4 行：



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("Hello World!\n");
5     printf("Hello World!\n");
6     printf("Hello World!\n");
7     printf("Hello World!\n");
8     return 0;
9 }
~
~
~
~
/Hello 4,10-17 All
```

此时按下 n，会继续向下查找到第 5 行，继续按 n，会继续向下查找。



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("Hello World!\n");
5     printf("Hello World!\n");
6     printf("Hello World!\n");
7     printf("Hello World!\n");
8     return 0;
9 }
~
~
~
~
/Hello 5,10-17 All
```

如上图时按下 N，会向上查找到第 4 行。



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("Hello World!\n");
5     printf("Hello World!\n");
6     printf("Hello World!\n");
7     printf("Hello World!\n");
8     return 0;
9 }
~
~
~
~
?Hello 4,10-17 All
```

使用“hello”替换“Hello”，且不需要确认，执行:%s/Hello/hello/g。

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("Hello World!\n");
5     printf("Hello World!\n");
6     printf("Hello World!\n");
7     printf("Hello World!\n");
8     return 0;
9 }
~
~
~
~
~
:%s/Hello/hello/g
```

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("hello World!\n");
5     printf("hello World!\n");
6     printf("hello World!\n");
7     printf("hello World!\n");
8     return 0;
9 }
~
~
~
~
4 substitutions on 4 lines 7,2-9 All
```

使用“world”替换“World”，且需要确认，执行:%s/World/world/gc。



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("hello World!\n");
5     printf("hello World!\n");
6     printf("hello World!\n");
7     printf("hello World!\n");
8     return 0;
9 }
~
~
~
~
:%s/World/world/gc
```

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("hello World!\n");
5     printf("hello World!\n");
6     printf("hello World!\n");
7     printf("hello World!\n");
8     return 0;
9 }
~
~
~
~
replace with world (y/n/a/q/l/^E/^Y)? 4,16-23 All
```

此时输入 y 确认替换，输入 n 取消替换，输入时不会显示。以下为输入 y 的结果。

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 #include<stdio.h>
2 int main(void)
3 {
4     printf("hello world!\n");
5     printf("hello World!\n");
6     printf("hello World!\n");
7     printf("hello World!\n");
8     return 0;
9 }
~
~
~
~
replace with world (y/n/a/q/l/^E/^Y)? 5,16-23 All
```

4.3 Gedit 编辑器

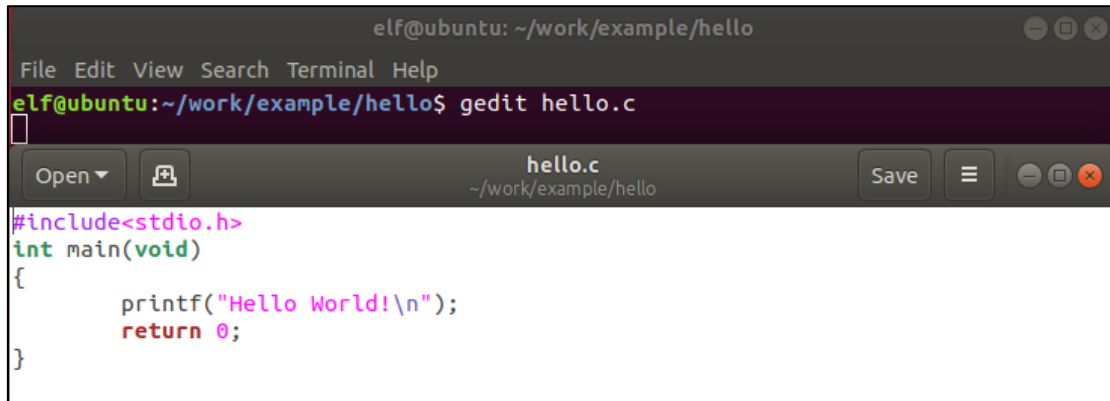
1.Gedit 的安装

我们可以直接使用 apt 命令安装 gedit:

```
elf@ubuntu:~$ sudo apt-get install gedit
```

2.Gedit 的使用

通过 `gedit filename` 对文件进行编辑，对于习惯使用界面编码修改的人来说，也是一个非常不错的工具。



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/hello$ gedit hello.c
hello.c
~/work/example/hello
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```


第五章 GCC 编译器

GCC (GNU Compiler Collection) 是由 GNU 开发的编程语言编译器，最初是作为 GNU 操作系统的编译器编写的。GCC 是一个编译器套件，包含很多软件包，支持多种语言编译。GCC 的官网是 <http://gcc.gnu.org>

说明：本章内容为通用知识，关于 GCC 编译器的讲解将在第二章搭建的环境中进行演示，适用于 ElfBoard 提供的所有环境。

5.1 GCC 的安装

ubuntu 默认安装了 GCC，但软件包可能不全，为了确保有一个基本完善的本地编译环境，可安装 build-essential 软件包。在确保 Linux 能联网的情况下，在终端输入如下安装命令：

```
elf@ubuntu:~$ sudo apt-get install build-essential
```

也可以通过简单的命令安装 gcc：

```
elf@ubuntu:~$ sudo apt install gcc
```

5.2 GCC 的编译选项

GCC 的基本用法是：gcc [选项] 文件名

GCC 有很多编译控制选项，使得 GCC 可以根据不同的参数进行不同的编译处理，我们简单介绍一些常用的基本控制选项。

选项	说明
-c	只编译不链接。只将.c 文件生成.o 为后缀的目标文件，通常用于编译不包含主程序的子程序文件
-S	只对文件进行编译，不汇编和链接
-E	只对文件进行预处理，不编译汇编和链接
-o outname	确定输出文件的名称为 outname,不能和源文件同名。如果不给出这个选项，gcc 给出预设的可执行文件 a.out
-g	产生符号调试工具（GNU 的 gdb）所必要的符号信息，要对源码进行调试必须加入这个选项。G 也分等级，默认是-g2,-g1 是最基本的，-g3 包含宏信息
-O	对程序进行优化编译链接。采用这个选项，整个源代码会在编译链接过程中进行优化处理，这样产生的可执行文件的效率可以提高，但是编译链接的速度相对要慢一些。
-I dirname	将 dirname 目录加入到程序头文件搜索目录列表中，是在预编译过程中使用的参数。注意：此处的 I 为 i 的大写
-L dirname	将 dirname 目录加入到库文件的搜索目录列表中
-l FOO	链接名为 libFOO 的函数库。注意：此处的 l 为 L 的小写
-static	链接静态库
-ansi	支持 ANSI/ISO C 的标准语法，取消 GNU 的语法中与该标准相冲突的部分
-w	关闭所有警告，不建议使用
-W	开启所有 gcc 提供的警告
-werror	将所有警告转换为错误，开启该选项，遇到警告都会终止编译
-v	显示执行的详细过程，gcc 以及相关程序的版本号

5.3 GCC 的基本使用

我们现在以我们学习语言的经典的 `hello.c` 的编译为例，对 `gcc` 的基本用法做一个了解。已编译 `hello.c` 文件为例。创建测试例程路径 `/home/elf/work/example/hello`，在该路径下进行 `hello.c` 文件的编写、编译和运行。

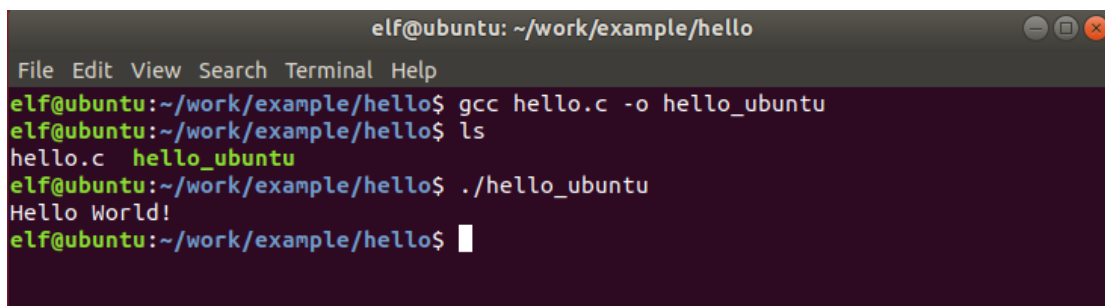
```
elf@ubuntu:~/work/example/hello$ vim hello.c
```

测试例程内容如下：

```
#include<stdio.h>
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

用 `cd` 命令切换到 `hello` 文件夹下，通过 `gcc` 命令编译 `hello.c` 文件：

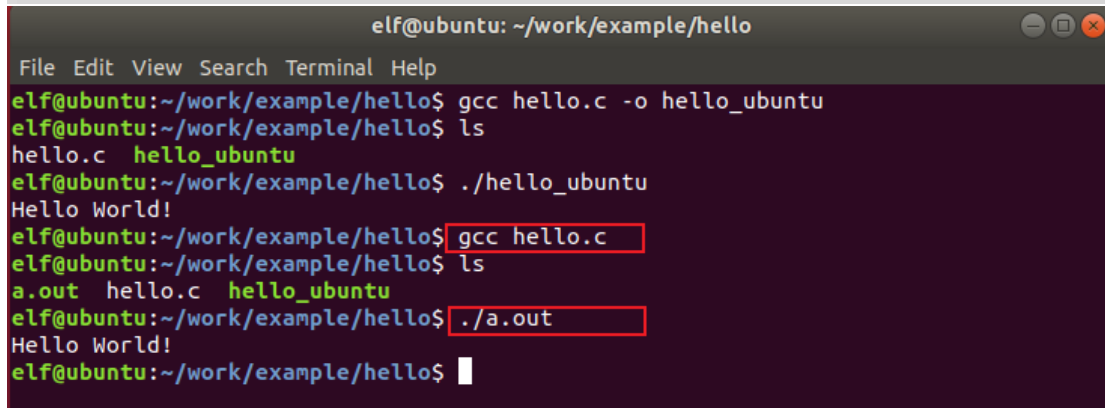
```
elf@ubuntu:~/work/example/hello$ gcc hello.c -o hello_ubuntu
```



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/hello$ gcc hello.c -o hello_ubuntu
elf@ubuntu:~/work/example/hello$ ls
hello.c  hello_ubuntu
elf@ubuntu:~/work/example/hello$ ./hello_ubuntu
Hello World!
elf@ubuntu:~/work/example/hello$
```

可以看到，`hello.c` 文件被编译成功，生成可执行文件 `hello_ubuntu`，我们在终端运行 `./hello_ubuntu`，可以看到输出结果和我们程序的要求一致。如果编译命令不添加 `-o hello_ubuntu`，则编译完成生成的是 `a.out` 可执行文件，输出结果是一样的。

```
elf@ubuntu:~/work/example/hello$ gcc hello.c
```



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/hello$ gcc hello.c -o hello_ubuntu
elf@ubuntu:~/work/example/hello$ ls
hello.c  hello_ubuntu
elf@ubuntu:~/work/example/hello$ ./hello_ubuntu
Hello World!
elf@ubuntu:~/work/example/hello$ gcc hello.c
elf@ubuntu:~/work/example/hello$ ls
a.out  hello.c  hello_ubuntu
elf@ubuntu:~/work/example/hello$ ./a.out
Hello World!
elf@ubuntu:~/work/example/hello$
```

1.GCC 编译过程

GCC 编译器的编译过程要经过预处理，编译，汇编，链接等 4 步才能够生成可执行文件。

预处理：主要实现将头文件展开，宏定义替换以及解析条件编译到文件中；

编译：将预处理的代码编译转换为汇编语言；

汇编：将汇编语言翻译成二进制文件；

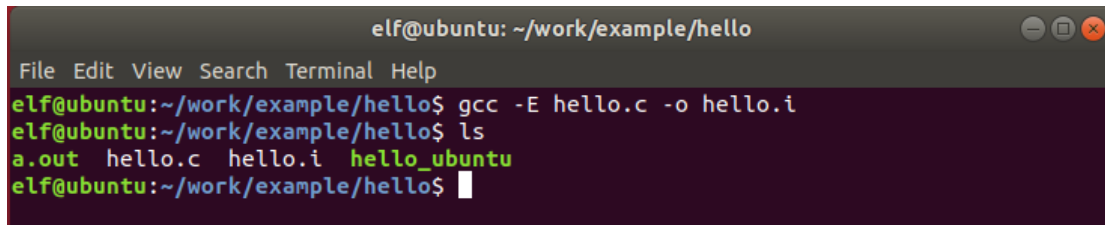
链接：将汇编出来的多个二进制目标文件和库文件进行链接，生成可执行文件。

我们可以通过一个简单的 `hello.c` 程序的编译过程对 GCC 的整个编译过程有一个简单的了解。



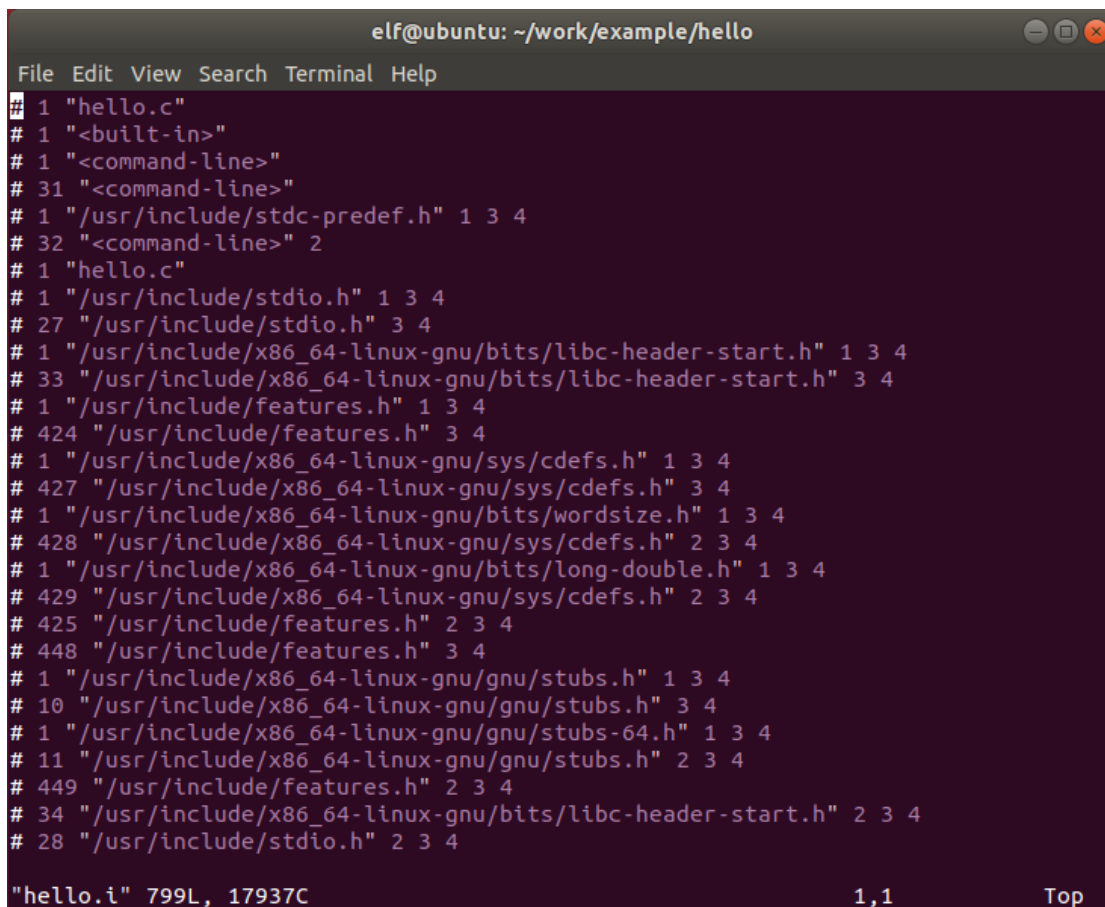
1) 预处理

```
elf@ubuntu:~/work/example/hello$ gcc -E hello.c -o hello.i
```



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/hello$ gcc -E hello.c -o hello.i
elf@ubuntu:~/work/example/hello$ ls
a.out hello.c hello.i hello_ubuntu
elf@ubuntu:~/work/example/hello$
```

我们使用 `vim hello.i` 命令可以看到预处理文件有将近 800 行，预处理过程将一个不到 10 行的代码，生成了一个将近 800 行的预处理文件。



```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
1 "hello.c"
# 1 "<built-in>"
# 1 "<command-line>"
# 31 "<command-line>"
# 1 "/usr/include/stdc-predef.h" 1 3 4
# 32 "<command-line>" 2
# 1 "hello.c"
# 1 "/usr/include/stdio.h" 1 3 4
# 27 "/usr/include/stdio.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 1 3 4
# 33 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 3 4
# 1 "/usr/include/features.h" 1 3 4
# 424 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 1 3 4
# 427 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/wordsize.h" 1 3 4
# 428 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 1 "/usr/include/x86_64-linux-gnu/bits/long-double.h" 1 3 4
# 429 "/usr/include/x86_64-linux-gnu/sys/cdefs.h" 2 3 4
# 425 "/usr/include/features.h" 2 3 4
# 448 "/usr/include/features.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 1 3 4
# 10 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 3 4
# 1 "/usr/include/x86_64-linux-gnu/gnu/stubs-64.h" 1 3 4
# 11 "/usr/include/x86_64-linux-gnu/gnu/stubs.h" 2 3 4
# 449 "/usr/include/features.h" 2 3 4
# 34 "/usr/include/x86_64-linux-gnu/bits/libc-header-start.h" 2 3 4
# 28 "/usr/include/stdio.h" 2 3 4
"hello.i" 799L, 17937C 1,1 Top
```

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help

extern int pclose (FILE *__stream);

extern char *ctermid (char *__s) __attribute__ ((__nothrow__ , __leaf__));
# 840 "/usr/include/stdio.h" 3 4
extern void flockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));

extern int ftrylockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));
);

extern void funlockfile (FILE *__stream) __attribute__ ((__nothrow__ , __leaf__));
);
# 868 "/usr/include/stdio.h" 3 4

# 2 "hello.c" 2

# 2 "hello.c"
int main(void)
{
    printf("Hello World!\n");
    return 0;
}
```

2) 编译

这个过程是将 hello.i 预处理文件编译成 hello.s 文件。

```
elf@ubuntu:~/work/example/hello$ gcc -S hello.i
```

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help

elf@ubuntu:~/work/example/hello$ gcc -S hello.i
elf@ubuntu:~/work/example/hello$ ls
a.out hello.c hello.i hello.s hello_ubuntu
elf@ubuntu:~/work/example/hello$
```

通过 vim hello.s 命令可以看到文件中都是汇编语句。

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
.file "hello.c"
.text
.section .rodata
.LC0:
.string "Hello World!"
.text
.globl main
.type main, @function
main:
.LFB0:
.cfi_startproc
pushq %rbp
.cfi_def_cfa_offset 16
.cfi_offset 6, -16
movq %rsp, %rbp
.cfi_def_cfa_register 6
leaq .LC0(%rip), %rdi
call puts@PLT
movl $0, %eax
popq %rbp
.cfi_def_cfa 7, 8
ret
.cfi_endproc
.LFE0:
.size main, .-main
.ident "GCC: (Ubuntu 7.5.0-3ubuntu1~18.04) 7.5.0"
.section .note.GNU-stack,"",@progbits
~
~
"hello.s" 27L, 461C 1,1-8 All
```

3) 汇编

在终端输入 `gcc -c hello.s` 将汇编语句生成机器码，得到 `hello.o` 文件。

```
elf@ubuntu:~/work/example/hello$ gcc -c hello.s
```

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/hello$ gcc -c hello.s
elf@ubuntu:~/work/example/hello$ ls
a.out hello.c hello.i hello.o hello.s hello_ubuntu
elf@ubuntu:~/work/example/hello$
```

4) 链接

机器码不能够直接运行，我们通过链接得到可执行文件 `hello`。

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/hello$ gcc hello.o -o hello
elf@ubuntu:~/work/example/hello$ ls
a.out hello hello.c hello.i hello.o hello.s hello_ubuntu
elf@ubuntu:~/work/example/hello$
```

5.4 静态链接和动态链接

1.静态链接

静态链接通过静态库进行链接，生成的目标程序中包含运行需要的所有库，可以直接运行，不过就是

文件比较大。静态库是汇编产生的.o 文件的集合，一般以.a 文件形式出现。gcc 在使用静态链接的时候需要加上-static 参数。

```
elf@ubuntu: ~/work/example/hello
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/hello$ gcc hello.o -static -o hello_static
elf@ubuntu:~/work/example/hello$ ls -l
total 896
-rwxrwxr-x 1 elf elf 8304 5月 12 17:39 a.out
-rwxrwxr-x 1 elf elf 8304 5月 12 17:46 hello
-rw-rw-r-- 1 elf elf 75 5月 12 17:01 hello.c
-rw-rw-r-- 1 elf elf 17937 5月 12 17:40 hello.i
-rw-rw-r-- 1 elf elf 1544 5月 12 17:45 hello.o
-rw-rw-r-- 1 elf elf 461 5月 12 17:43 hello.s
-rwxrwxr-x 1 elf elf 845240 5月 12 19:21 hello_static
-rwxrwxr-x 1 elf elf 8304 5月 12 17:37 hello_ubuntu
elf@ubuntu:~/work/example/hello$
```

2.动态链接

动态链接使用动态链接库进行链接，生成的程序在执行的时候需要加载所需要的动态库才能运行，动态链接程序轻便，但必须依赖动态库。Linux 下的动态链接库实际上是共享目标文件，一般是以.so 文件形式存在。GCC 默认是动态链接不需要加参数。

举例子看一下动态链接库的使用方法，创建三个 C 语言文件：mymath.c、mymath.h、test.c。

```
elf@ubuntu: ~/work/example/compile_test
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/compile_test$ ls
mymath.c mymath.h test.c
elf@ubuntu:~/work/example/compile_test$
```

通过两个变量相加的方式来举例说明。

mymath.h 添加内容如下：

```
#ifndef __MYMATH_H_
#define __MYMATH_h_
#include<stdio.h>

extern int add(int a,int b);
#endif
```

mymath.c 添加内容如下：

```
#include "mymath.h"

int add(int a,int b)
{
    return a+b;
}
```

最后来一个测试文件 test.c(在这里需要注意的一个小细节，调用自己写的.h 文件时，必须使用双引号，“xxx.h”)：

```
#include<stdio.h>
#include"mymath.h"
```




```
void main()
{
    printf("2+3=%d\n",add(2,3));
}
```

然后进行编译，因为 `test` 引用了 `mymath.h`，所以不能单独编译，`gcc test.c mymath.c` 一起进行，否则就会报错。（如下编译正确）

```
elf@ubuntu: ~/work/example/compile_test
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/compile_test$ ls
mymath.c mymath.h test.c
elf@ubuntu:~/work/example/compile_test$ gcc test.c mymath.c -o test
elf@ubuntu:~/work/example/compile_test$ ls
mymath.c mymath.h test test.c
elf@ubuntu:~/work/example/compile_test$ ./test
2+3=5
elf@ubuntu:~/work/example/compile_test$
```

接下来是生成动态链接库的方法，`gcc -shared xx -o xxx.so`，当我们使用 `cat` 去进行查看的时候会发现，`so` 文件里全是乱码，这就是使用动态链接库的一个好处，可以防止核心算法的泄露。

[illegible]

现在就可以将 `mymath.c` 删除掉了:

```
elf@ubuntu: ~/work/example/compile_test
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/compile_test$ ls
mymath.c mymath.h mymath.so test test.c
elf@ubuntu:~/work/example/compile_test$ rm mymath.c
elf@ubuntu:~/work/example/compile_test$ rm test
elf@ubuntu:~/work/example/compile_test$ ls
mymath.h mymath.so test.c
elf@ubuntu:~/work/example/compile_test$
```

直接调用库编译发现程序不能运行。

```
elf@ubuntu: ~/work/example/compile_test
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/compile_test$ gcc test.c -o test -L. mymath.so
elf@ubuntu:~/work/example/compile_test$ ls
mymath.h mymath.so test test.c
elf@ubuntu:~/work/example/compile_test$ ./test
./test: error while loading shared libraries: mymath.so: cannot open shared object file: No such file or directory
elf@ubuntu:~/work/example/compile_test$
```

下面将我们自己建立的库移动到/usr/lib 目录下试试，然后发现代码运行成功。

```
elf@ubuntu: ~/work/example/compile_test
File Edit View Search Terminal Help
elf@ubuntu:~/work/example/compile_test$ ls
mymath.h mymath.so test test.c
elf@ubuntu:~/work/example/compile_test$ sudo mv mymath.so /usr/lib
[sudo] password for elf:
elf@ubuntu:~/work/example/compile_test$ ls
mymath.h test test.c
elf@ubuntu:~/work/example/compile_test$ ./test
2+3=5
elf@ubuntu:~/work/example/compile_test$
```

可以在 Linux 系统下自己尝试下。

第六章 文件传输与网络服务的搭建

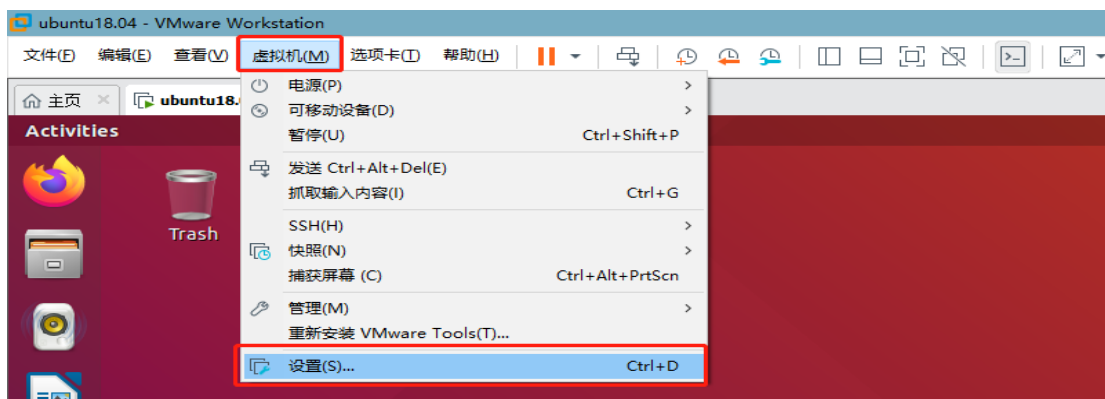
说明：本章内容为通用知识，关于文件传输与网络服务的搭建的讲解将在第二章搭建的环境中进行演示，适用于 ElfBoard 提供的所有环境。

6.1 windows 与 ubuntu 系统互传文件

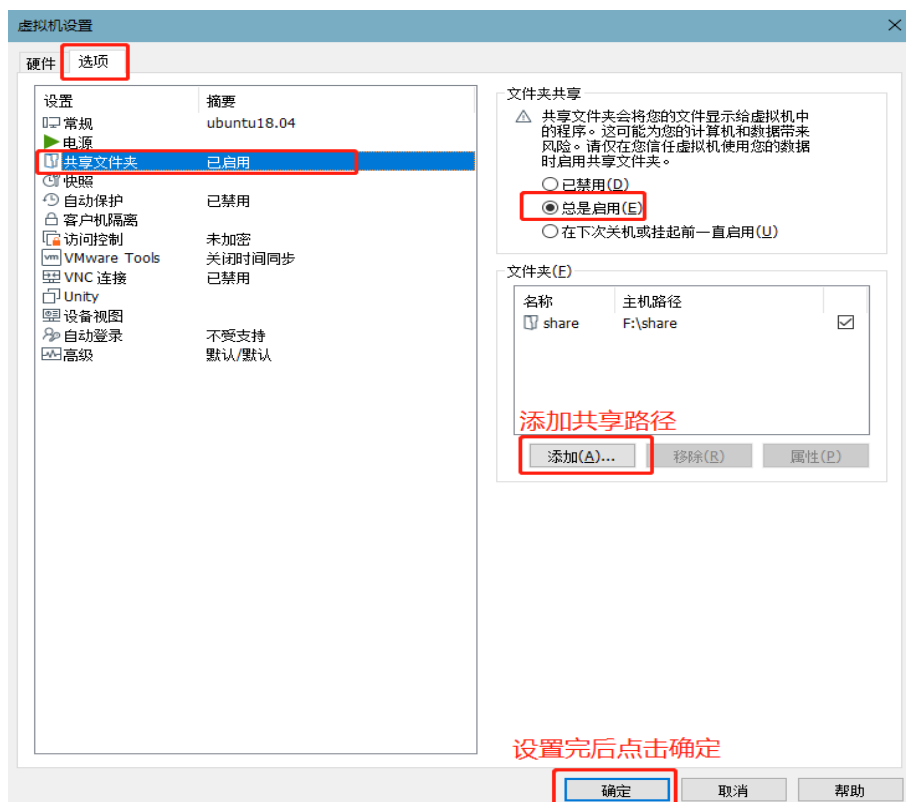
6.1.1 共享文件夹

ubuntu 和 Windows 主机之间的文件传输有很多种，安装 VMware Tools 后，可以设置虚拟机共享文件夹，将 Windows 主机的文件目录挂载到 ubuntu 中，实现文件共享。

设置方法如下，点击菜单栏的“虚拟机”，选择“设置”



点击“选项”，启用“共享文件夹”，设置 Windows 主机上的共享目录，点击“确定”。



虚拟机的文件共享设置完成后，将测试的所用的：内核源码 linux-4.1.15-elf1.tar.bz2、文件系统（本

节以 rootfs.tar.bz2 为例)、命令行测试程序 elf1_cmd_wdt.c 放到 Windows 主机的共享文件夹后。

共享文件夹在 ubuntu 中的挂载目录/mnt/hgfs/share, 查看挂载目录下文件。

```
elf@ubuntu:~$ ls /mnt/hgfs/share/  
elf1_cmd_wdt.c  linux-4.1.15-elf1.tar.bz2  rootfs.tar.bz2
```

将共享文件夹中的源码拷贝到 ubuntu 的/home/elf/work 目录下

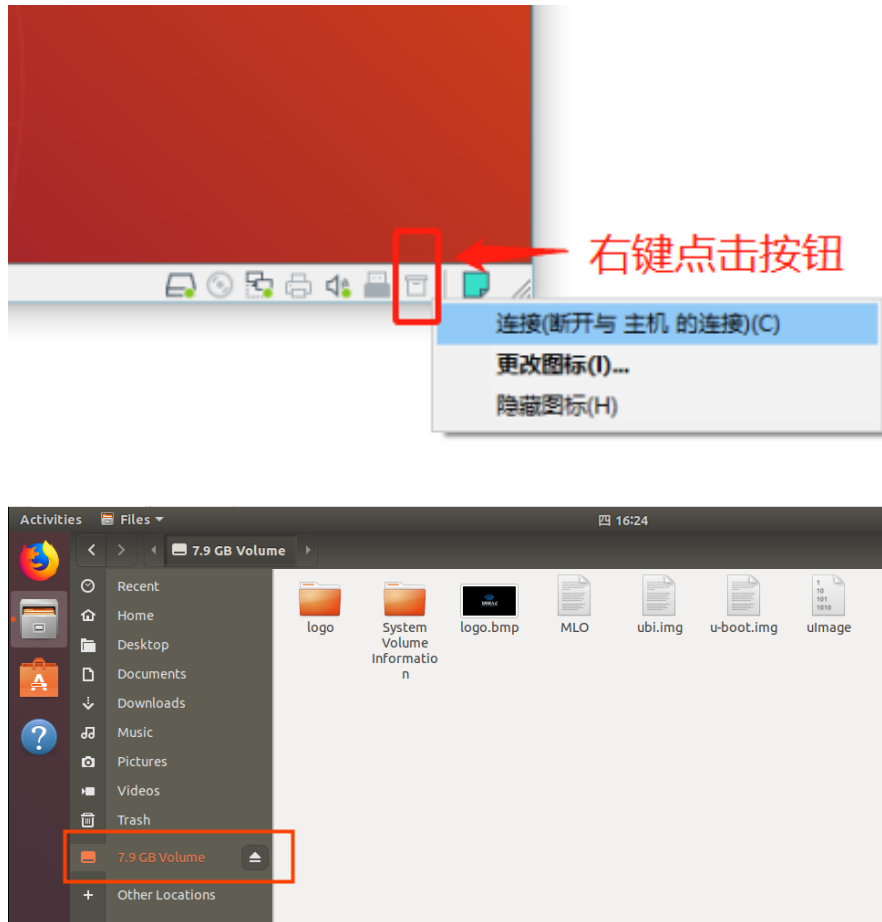
```
elf@ubuntu:~$ sudo cp -r /mnt/hgfs/share/linux-4.1.15-elf1.tar.bz2 /home/elf/work/  
[sudo] password for elf:  
elf@ubuntu:~$ sudo cp -r /mnt/hgfs/share/rootfs.tar.bz2 /home/elf/work/  
elf@ubuntu:~$ sudo cp -r /mnt/hgfs/share/elf1_cmd_wdt.c /home/elf/work/  
elf@ubuntu:~$ cd /home/elf/work/  
elf@ubuntu:~$ ls  
elf1_cmd_wdt.c  linux-4.1.15-elf1.tar.bz2  rootfs.tar.bz2      //文件拷贝成功
```

6.1.2 U 盘的加载

打开虚拟机设置, USB 控制器, 在兼容性里面选择 USB3.0, 然后确定。如下图, 因为目前大多数电脑都支持 USB3.0 的接口, 如果不设置, 当我们插入 USB3.0 接口, 是不能连接到虚拟机的。如下图:



虚拟机启动后, 插入 U 盘, 虚拟机右下角会多出一个类似“U 盘”的图标, 右击-->连接即可, 然后就可以在文件系统看到多一个目录, 说明 U 盘加载成功, 如图:



6.2 常用的网络服务的搭建和使用

在网络服务搭建和测试时，ubuntu 的网络模式需要设置为桥接模式，电脑主机、ubuntu 主机、开发板三者的 ip 要处于同一网段。关闭 ubuntu 和 windows 防火墙（关闭 ubuntu 防火墙:sudo ufw disable）

教程中使用的三者的 ip 分别如下：

电脑主机 ip: 192.168.0.64

主机 ip:192.168.0.51

开发板 ip:192.168.0.232

6.2.1 TFTP 服务搭建

TFTP（Trivial File Transfer Protocol，简单文件传输协议），是 TCP/IP 协议族中用来在客户机和服务器之间进行简单文件传输的协议，开销很小。TFTP 通常用于内核调试。在嵌入式 Linux 开发过程中，内核调试是其中一个基础、重要的环节。调试内核通常是与 Bootloader 配合使用，只需在 Bootloader 中实现了网卡驱动和 TFTP 客户端，就可以使用 TFTP 进行传输内核。使用 TFTP 协议传输文件，还需要在主机安装 TFTP 服务端，可以在 Linux 系统下实现，也可以在 Windows 系统下实现，下面在 Linux(ubuntu)系统下安装 TFTP 服务端进行测试。

1.安装服务器、客户端和守护进程

```
elf@ubuntu:~$ sudo apt-get install tftp-hpa tftpd-hpa xinetd
```

2.服务器配置

首先，在/home/elf目录下建一个tftpboot，并把属性改成任意用户可读写：

```
elf@ubuntu:~$ mkdir tftpboot
elf@ubuntu:~$ chmod 777 tftpboot
```

然后，进入目录 /etc/xinetd.d/，并在其中新建文件tftp，把指定的内容加入到tftp文件中：

```
elf@ubuntu:~$ cd /etc/xinetd.d/
elf@ubuntu:/etc/xinetd.d$ sudo vim tftp
```

添加以下内容到tftp文件：

```
service tftp
{
    disable = no 138
    socket_type = dgram
    protocol = udp
    wait = yes
    user = elf
    server = /usr/sbin/in.tftpd
    server_args = -s /home/elf/tftpboot -c
    per_source = 11
    cps = 100 2
}
```

最后，修改配置文件/etc/default/tftpd-hpa:

```
elf@ubuntu:~$ sudo vim /etc/default/tftpd-hpa
```

修改为：

```
TFTP_USERNAME="tftp"
TFTP_DIRECTORY="/home/elf/tftpboot"
TFTP_ADDRESS="0.0.0.0:69"
TFTP_OPTIONS="--secure -l -c -s"
```

⚠注意：将“TFTP_DIRECTORY” 改为新建tftpboot目录所在的路径。

3.重新启动服务

```
elf@ubuntu:~$ sudo /etc/init.d/xinetd reload
[ ok ] Reloading xinetd configuration (via systemctl): xinetd.service.
elf@ubuntu:~$ sudo /etc/init.d/xinetd restart
[ ok ] Restarting xinetd (via systemctl): xinetd.service.
elf@ubuntu:~$ sudo /etc/init.d/tftpd-hpa restart
[ ok ] Restarting tftpd-hpa (via systemctl):tftpd-hpa.service.
```

4.测试服务器

测试一下，在/tftpboot文件夹下新建建立一个文件：

```
elf@ubuntu:~/tftpboot$ touch abc
```

进入另外一个文件夹：

```
elf@ubuntu:~/tftpboot$ cd /home/
elf@ubuntu:/home$ sudo tftp 192.168.0.51 //192.168.0.51 为本
机 IP
tftp> get abc
tftp> quit
elf@ubuntu:/home$ ls
```

```
abc elf
```

如果可以下载abc文件说明服务器已经安装成功

6.2.2 NFS 服务搭建

在嵌入式 Linux 开发中，需要在 Linux 主机编写程序代码，然后编译程序，生成的二进制程序文件要传输到目标机上才能调试、运行。那么如何更快、更便捷地传输文件，提高开发工作的效率呢？NFS 无疑是最好的选择。通过 NFS 服务，主机将用户指定的目录通过网络共享给目标机（和 windows 的文件网络共享类似）。目标机可以直接运行存放于 Linux 主机共享目录下的二进制程序，这样调试程序时十分方便、高效。

NFS 即网络文件系统（Network File-System），可以通过网络让不同机器、不同系统之间可以实现文件共享。通过 NFS，可以访问远程共享目录，就像访问本地磁盘一样。

1.ubuntu 下搭建 nfs 服务器的方法

软件下载安装：

```
elf@ubuntu:~$ sudo apt-get install nfs-kernel-server nfs-common portmap
```

创建NFS的目录为nfs_rootfs，并在该目录下新建测试文件test；

```
elf@ubuntu:~$ mkdir nfs_rootfs
elf@ubuntu:~$ cd nfs_rootfs/
elf@ubuntu:~/nfs_rootfs$ touch test
elf@ubuntu:~/nfs_rootfs$ ls
test
```

修改配置文件：

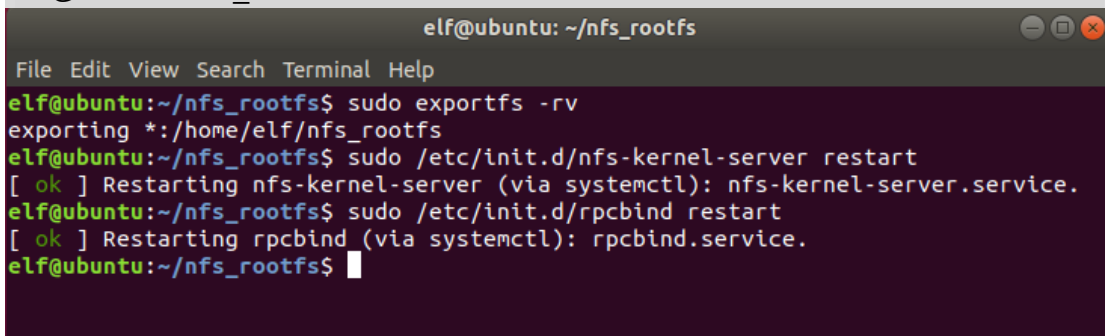
```
elf@ubuntu:~/nfs_rootfs$ sudo vim /etc/exports
```

在文件中添加以下配置：

```
/home/elf/nfs_rootfs *(rw,sync,no_root_squash,no_subtree_check)
```

重启配置文件和服务：

```
elf@ubuntu:~/nfs_rootfs$ sudo exportfs -rv
elf@ubuntu:~/nfs_rootfs$ sudo /etc/init.d/rpcbind restart
elf@ubuntu:~/nfs_rootfs$ sudo /etc/init.d/nfs-kernel-server restart
```



2.验证

在开发板上验证nfs服务器，执行完以下命令将nfs服务器挂载到开发板的/mnt目录：

```
root@ELF1:~# mount -t nfs4 -o vers=4 172.20.8.9:/home/elf/nfs_rootfs /mnt/
```

挂载成功后，查看/mnt目录，会看到刚才新建的测试文件：

```
root@ELF1:~# ls /mnt/
test
```

6.2.3 SSH 服务搭建

SSH 为 Secure Shell 的缩写，由 IETF 的网络小组（Network Working Group）所制定的建立在应用层基础上的安全协议。SSH 是较可靠，专为远程登录会话和其他网络服务提供安全性的协议，利用 SSH 协议可以有效防止远程管理过程中的信息泄露问题。SSH 最初是 UNIX 系统上的一个程序，后来又迅速扩展到其他操作平台，目前为止，几乎所有 UNIX 平台，包括 HP-UX、Linux、AIX、Solaris、Digital UNIX、Irix 等，都可运行。

1.在系统开发过程中经常会遇到下列情形

- 1) Linux 主机不在本地，但又要使用或者维护这台计算机；
- 2) 一个嵌入式 Linux 产品不方便接调试串口，需要进行维护；
- 3) 在远程机器和本地机器之间进行文件传输。

如果远程目标系统已经开启了 SSH 服务，通过 SSH 可以轻松解决以上问题。使用 SSH 服务，一方面需要在远程系统上安装 SSH 服务，另一方面要在本地系统上安装 SSH 客户端，常见的 SSH 客户端有 putty、SSH Secure Shell Client 等。

2.SSH 有以下几个知识要点

- 1) SSH 是安全的加密协议，用于远程连接 Linux 服务器；
- 2) SSH 默认端口是 22 号端口，安全协议版本 SSHv2，出了 SSHv2 版本之外还有 SSHv1 版本(有漏洞，不安全)；
- 3) SSH 服务端主要包含两个服务功能 SSH 远程连接和 SFTP 服务；
- 4) Linux SSH 客户端包含 SSH 远程连接命令以及远程拷贝 scp 命令等。

3.安装 SSH

- 1) 在 ubuntu(Linux 主机)终端键入以下指令，安装 SSH 服务：

```
elf@ubuntu:/$ sudo apt-get install ssh
```

- 2) 启动 SSH 服务：

```
elf@ubuntu:/$ sudo service ssh start
```

- 3) 查看 SSH 服务的状态：

```
elf@ubuntu:/$ sudo service ssh status
```

```
• ssh.service - OpenBSD Secure Shell server
```

```
Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)
```

```
Active: active (running) since Mon 2021-08-23 17:19:57 CST; 45s ago
```

```
Main PID: 7383 (sshd)
```

```
Tasks: 1 (limit: 2292)
```

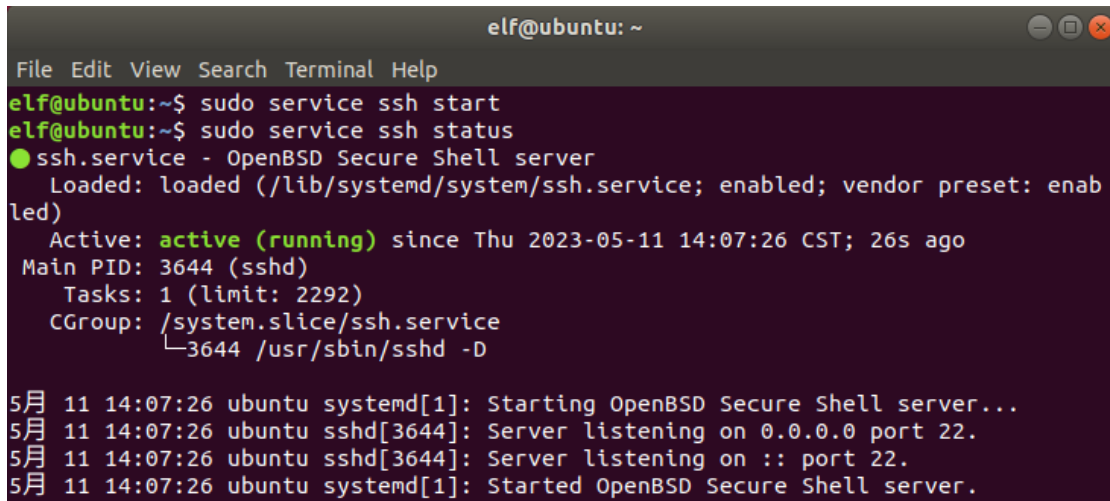
```
CGroup: /system.slice/ssh.service
```

```
└─7383 /usr/sbin/sshd -D
```

```
8月 23 17:19:57 ubuntu systemd[1]: Starting OpenBSD Secure Shell server...
```

```
8月 23 17:19:57 ubuntu sshd[7383]: Server listening on 0.0.0.0 port 22.
```

```
8月 23 17:19:57 ubuntu sshd[7383]: Server listening on :: port 22.  
8月 23 17:19:57 ubuntu systemd[1]: Started OpenBSD Secure Shell server.
```



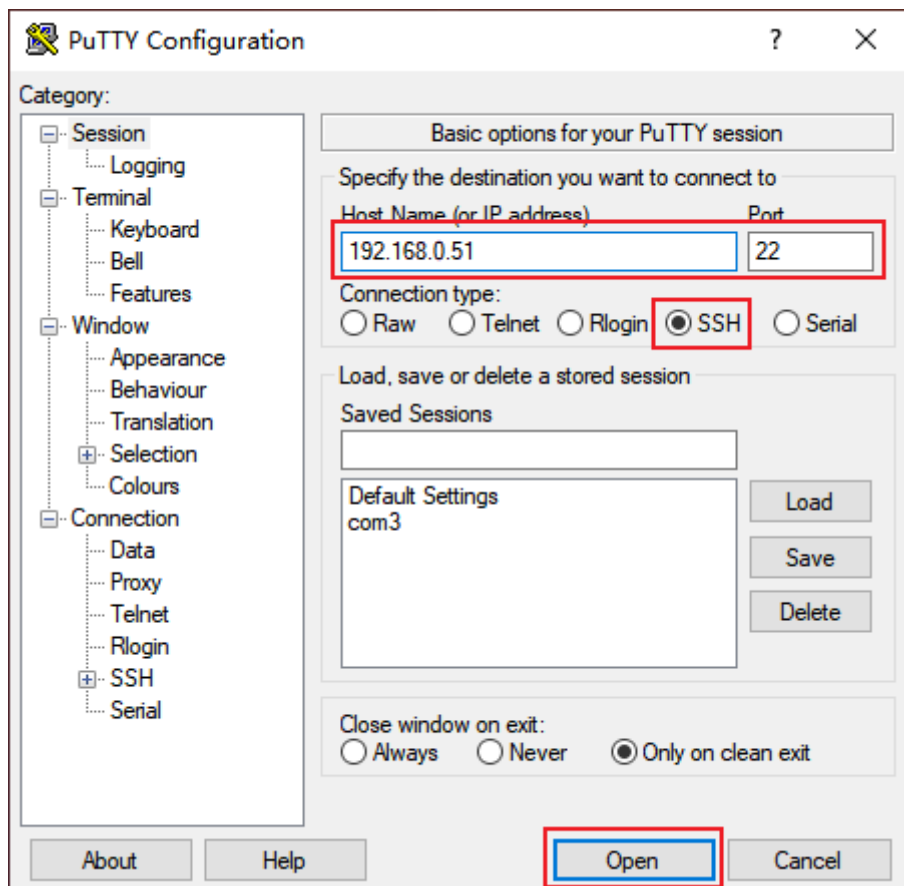
```
elf@ubuntu: ~  
File Edit View Search Terminal Help  
elf@ubuntu:~$ sudo service ssh start  
elf@ubuntu:~$ sudo service ssh status  
● ssh.service - OpenBSD Secure Shell server  
   Loaded: loaded (/lib/systemd/system/ssh.service; enabled; vendor preset: enabled)  
   Active: active (running) since Thu 2023-05-11 14:07:26 CST; 26s ago  
     Main PID: 3644 (sshd)  
       Tasks: 1 (limit: 2292)  
      CGroup: /system.slice/ssh.service  
             └─3644 /usr/sbin/sshd -D  
  
5月 11 14:07:26 ubuntu systemd[1]: Starting OpenBSD Secure Shell server...  
5月 11 14:07:26 ubuntu sshd[3644]: Server listening on 0.0.0.0 port 22.  
5月 11 14:07:26 ubuntu sshd[3644]: Server listening on :: port 22.  
5月 11 14:07:26 ubuntu systemd[1]: Started OpenBSD Secure Shell server.
```

4) 关闭 SSH 服务:

```
elf@ubuntu:/$ sudo service ssh stop
```

5) 测试方法:

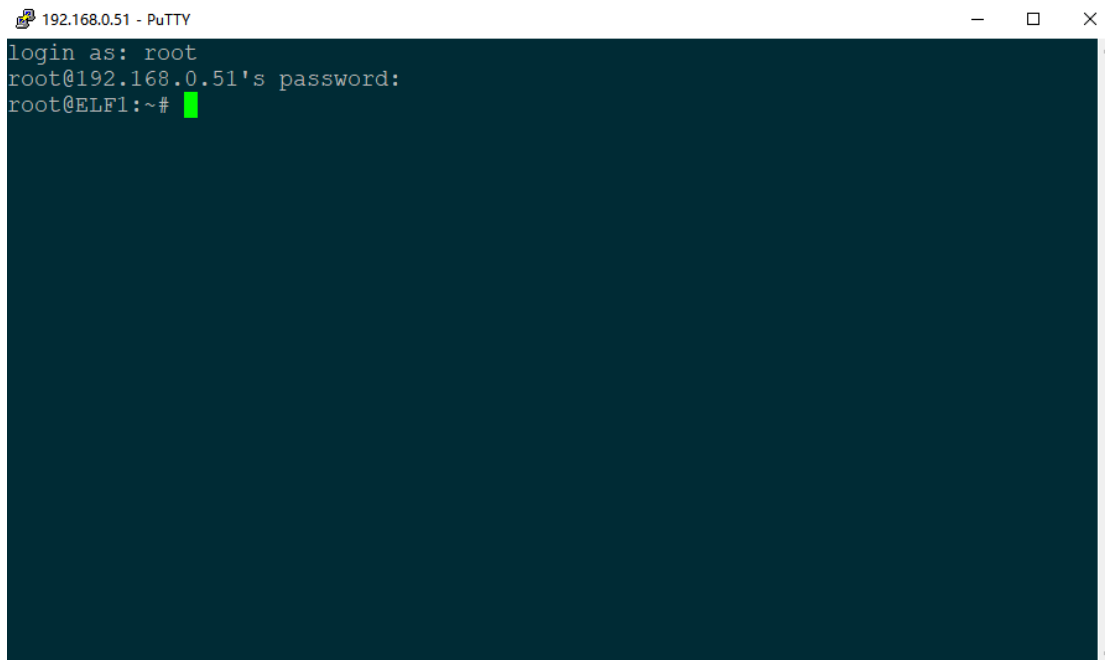
在 Windows 主机通过 putty 访问 Linux 系统, 打开 putty 界面, 使用 SSH 登录, ip 设置 Linux 主机的 ip, 端口号默认为 22。



第一次登录, 会提示如下警告, 点击“是”即可:



登录时，输入用户名与密码完成登录。



声明

本手册版权归保定飞凌嵌入式技术有限公司所有。未经本公司的书面许可，任何单位和个人无权以任何形式复制、传播、转载本手册的任何部分，违者将被追究法律责任。

保定飞凌嵌入式有限公司所提供的服务内容旨在协助用户加速产品的研发进度，在服务过程中所提供的任何程序、文档、测试结果、方案、支持等资料和信息，都仅供参考，用户有权不使用或自行参考修改，本公司不提供任何的完整性、可靠性等保证，若在用户使用过程中因任何原因造成的特别的、偶然的或间接的损失，本公司不承担任何责任。

更多帮助

注意事项与维护

- 请勿带电插拔核心板及外围模块！
- 请遵循所有标注在产品上的警示和指引信息。
- 请保持本产品干燥。如果不慎被任何液体泼溅或浸润，请立刻断电并充分晾干。
- 使用中注意本产品的通风散热，避免温度过高造成元器件损坏。
- 请勿在多尘、脏乱的环境中使用或存放本产品。
- 请勿将本产品应用在冷热交替环境中，避免结露损坏元器件。
- 请勿粗暴对待本产品，跌落、敲打或剧烈晃动都可能损坏线路及元器件。
- 请勿使用有机溶剂或腐蚀性液体清洗本产品。
- 请勿自行修理、拆卸本公司产品，如产品出现故障请及时联系本公司进行维修。
- 擅自修改或使用未经授权的配件可能损坏本产品，由此造成的损坏将不予以保修。

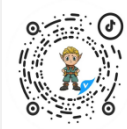


更新通知与学习资源

ElfBoard 产品相关资料会持续完善和更新，本手册内容也会及时跟进。请在使用这些资料时，确保它们为最新版本；最新的资料更新通知可通过微信公众号获取，课程视频和嵌入式知识科普则可以在抖音、哔哩哔哩等平台观看，以帮助大家更好地了解和使用 ElfBoard 产品。



官方公众号



官方抖音



哔哩哔哩

资料获取

1. 请登录 ElfBoard 官网“www.elfboard.com”，进入“资料”专区进行下载；
2. 下载前请阅读《用户资料极速下载必读》。

售后服务政策

1. 如产品使用过程中出现硬件故障可根据售后服务政策进行维修；
2. 售后服务政策：参见官网“www.elfboard.com”首页->技术服务->售后服务政策。

送修地址

1. 地址：河北省保定市高开区向阳北大街 2699 号飞凌嵌入式技术有限公司新楼五层售后维修部
2. 联系人：售后维修部
3. 电话：0312-3102650-952、953
4. 邮编：071000
5. 邮寄须知：建议使用顺丰、圆通或韵达，且不接收任何到付。

技术支持范围

1. 本公司产品的软、硬件资源提供情况咨询；
2. 本公司产品的软、硬件手册使用过程中遇到的问题。

技术讨论范围

1. 源码的修改以及理解；
 2. 操作系统如何移植；
 3. 用户自行修改以及开发中遇到的软硬件问题；
- 注：以上三点虽不属于技术支持范围，但我公司会尽力为用户提供帮助，如仍然没能解决敬请谅解。

技术支持方式

1. 论坛：bbs.elfboard.com
2. QQ 群：474552704

技术支持时间

1. 周一至周五：上午 9:00—11:30；
下午 13:30—17:00；
2. 公司按照国家法定节假日安排休息，在此期间无法提供技术支持，请将问题发送至论坛开发板专区或 QQ 群，我们会在工作日尽快给您回复。